

Contents

第十二册简介：OpenAI 大模型算法岗求职与面试作战手册	8
本书定位	8
为什么需要这本书	8
适合读者	8
学完后应具备的能力	8
阅读建议	8
与其他书的关系	8
第十二册：OpenAI 大模型算法岗求职与面试作战手册	8
本书定位	8
章节文件	8
第一章：岗位画像	9
1.1 先建立直觉：大模型算法岗不是单一岗位	9
1.2 高水平大模型团队到底看什么	9
1.2.1 研究能力	9
1.2.2 训练能力	10
1.2.3 对齐与安全意识	10
1.2.4 评估能力	10
1.2.5 系统与工程能力	10
1.2.6 产品影响	11
1.2.7 协作与表达能力	11
1.2.8 学习速度与适应能力	11
1.3 不同岗位方向的画像	11
1.3.1 基础模型研究岗	11
1.3.2 预训练与训练工程岗	12
1.3.3 Post-training 与对齐岗	12
1.3.4 评估与模型行为岗	12
1.3.5 多模态岗	13
1.3.6 推理部署岗	13
1.3.7 Agent 与应用算法岗	13
1.3.8 数据岗与数据智能岗	13
1.4 OpenAI 风格岗位可能更强调什么	14
1.5 面试官如何从回答中判断你	14
1.6 简历和面试要围绕岗位画像组织	14
1.7 候选人常见错位	15
1.7.1 把“知道”误认为“会做”	15
1.7.2 只准备热门词，不准备底层逻辑	15
1.7.3 只讲结果，不讲过程	15
1.7.4 只讲成功，不讲失败	15
1.7.5 忽略安全和合规	15
1.8 如何读 JD	15
1.9 自我定位表	16
1.10 准备优先级	16
1.11 本书后续章节如何使用	17
1.12 小结	17
第二章：简历策略	17
2.1 简历的本质：降低面试官判断成本	18
2.2 大模型算法岗简历应该突出什么	18
2.3 项目经历怎么写	18
2.4 项目条目的黄金公式	19
2.5 如何写论文经历	19
2.6 如何写工程贡献	20

2.7 指标结果怎么写	20
2.8 技术关键词怎么放	21
2.9 如何写影响力	21
2.10 不同背景候选人的简历策略	22
2.10.1 应届生	22
2.10.2 有算法经验但非大模型背景	22
2.10.3 工程背景转算法岗	22
2.10.4 已有大模型工业经验	22
2.11 一页还是两页	23
2.12 常见简历问题	23
2.12.1 只写职责，不写结果	23
2.12.2 只写结果，不写方法	23
2.12.3 技术词过密	23
2.12.4 贡献边界不清	23
2.12.5 项目太多但都很浅	23
2.12.6 没有目标岗位意识	24
2.13 简历和面试是一体的	24
2.14 简历自检清单	24
2.15 本章小结	24
第三章：项目包装	24
3.1 什么是好的项目讲述	25
3.2 项目讲述的标准结构	25
3.3 背景：不要讲太久，但要讲清意义	25
3.4 问题：从“做了什么”转向“解决什么”	26
3.5 难点：展示技术判断的入口	26
3.6 方法：讲决策链，不要只讲技术栈	27
3.7 实验：让故事从“我觉得”变成“我验证过”	27
3.8 结果：讲清提升，也讲清代价	27
3.9 失败：失败经历是加分项，不是扣分项	28
3.10 反思和下一步：展示成长性	28
3.11 如何准备 1 分钟、3 分钟、10 分钟版本	28
3.11.1 1 分钟版本	29
3.11.2 3 分钟版本	29
3.11.3 10 分钟版本	29
3.12 不同类型项目怎么讲	29
3.12.1 训练项目	29
3.12.2 Post-training 项目	29
3.12.3 RAG 项目	30
3.12.4 推理部署项目	30
3.12.5 评估项目	30
3.13 面试官常见追问	30
3.14 如何把普通项目讲出深度	31
3.15 如何避免过度包装	31
3.16 项目准备模板	31
3.17 本章小结	32
第四章：Coding Interview	32
4.1 大模型算法岗 coding 面试考什么	32
4.2 面试官在观察什么	33
4.3 Python 基本功	33
4.4 通用数据结构与算法	34
4.5 张量操作是核心基本功	35
4.6 Attention 实现	35
4.7 Multi-Head Attention	36
4.8 Sampling 实现	37

4.9 DPO Loss 实现	38
4.10 Debug 代码题	39
4.11 数值稳定性	40
4.12 性能与内存意识	40
4.13 手写小模块训练	41
4.14 面试中的代码风格	41
4.15 如何准备	42
4.16 常见现场失误	42
4.16.1 不澄清题意	42
4.16.2 Shape 不写清楚	42
4.16.3 忽略 mask	42
4.16.4 忽略 dtype 和 device	43
4.16.5 只会调用库，不会解释原理	43
4.16.6 过早优化	43
4.17 本章小结	43
第五章：ML 与 LLM 基础面试	43
5.1 基础面试到底考什么	43
5.2 回答基础题的三层结构	44
5.3 概率基础：语言模型到底在建模什么	44
5.4 最大似然、负对数似然和交叉熵	44
5.5 KL 散度和交叉熵的关系	45
5.6 优化基础：梯度下降在做什么	45
5.7 SGD、Adam 和 AdamW	45
5.8 Batch Size、梯度累积和泛化	46
5.9 泛化：为什么大模型不只是记忆	46
5.10 Bias-Variance、过拟合和欠拟合	46
5.11 Transformer 的核心直觉	47
5.12 Self-Attention 的机制	47
5.13 Multi-Head Attention 为什么需要多头	48
5.14 位置编码：为什么 Transformer 需要顺序信息	48
5.15 LayerNorm、RMSNorm 和残差连接	48
5.16 MLP、激活函数和 SwiGLU	48
5.17 Decoder-only、Encoder-only 和 Encoder-Decoder	49
5.18 Tokenization：文本如何变成模型输入	49
5.19 特殊 Token 和 Padding	49
5.20 LLM 训练目标：Pretraining、SFT 和偏好学习	50
5.21 Causal LM Loss 的细节	50
5.22 面试高频基础题与回答要点	50
5.22.1 为什么 Transformer 比 RNN 更适合大模型	50
5.22.2 为什么 attention 要加 mask	51
5.22.3 为什么 softmax 后能表示概率	51
5.22.4 为什么需要 validation set	51
5.22.5 为什么模型会 hallucinate	51
5.22.6 为什么 instruction tuning 有用	51
5.23 如何表现出“基础扎实”	51
5.24 常见失分点	52
5.24.1 背概念但讲不出关系	52
5.24.2 公式会背但不会解释直觉	52
5.24.3 忽略工程细节	52
5.24.4 绝对化表达	52
5.24.5 不承认不确定性	52
5.25 准备清单	52
5.26 本章小结	52
第六章：Training 面试	53

6.1 Training 面试到底考什么	53
6.2 回答 Training 题的通用结构	53
6.3 预训练: 模型从哪里获得能力	54
6.4 数据是训练的第一变量	54
6.4.1 去重为什么重要	54
6.4.2 质量过滤不是越严越好	55
6.4.3 数据混合比例决定能力形状	55
6.5 SFT 与预训练的区别	55
6.6 Optimizer、学习率和 batch size	55
6.7 分布式训练: 为什么不能只用 Data Parallel	56
6.7.1 Data Parallel	56
6.7.2 Tensor Parallel	56
6.7.3 Pipeline Parallel	56
6.7.4 ZeRO 与 FSDP	57
6.8 混合精度、数值稳定性与 loss spike	57
6.8.1 什么是 loss spike	57
6.8.2 loss spike 怎么排查	57
6.8.3 不要只说 “调小学习率”	57
6.9 训练监控: 哪些指标必须看	58
6.10 Evaluation: 训练中如何判断模型真的变好了	58
6.11 Checkpoint: 训练工程的安全绳	59
6.11.1 checkpoint 保存频率怎么定	59
6.11.2 resume 后要检查什么	59
6.12 成本估算: 训练到底要多少钱	59
6.13 Scaling Law: 模型、数据和算力怎么权衡	60
6.14 高频题: 如何设计一次继续预训练	60
6.15 高频题: 训练慢怎么排查	61
6.16 高频题: 如何判断训练是否发散	61
6.17 高频题: 预训练、SFT、RLHF 的训练差异	61
6.18 面试中的常见失分点	62
6.19 一套完整 Training 面试回答模板	62
6.20 准备清单	62
第七章: Deployment 面试	63
7.1 Deployment 面试到底考什么	63
7.2 回答 Deployment 题的通用结构	63
7.3 从 checkpoint 到线上服务	64
7.4 Prefill 与 Decode	64
7.5 KV Cache: 推理服务的核心状态	64
7.6 PagedAttention 与 vLLM	65
7.7 Dynamic Batching 与 Continuous Batching	65
7.8 Chunked Prefill: 长 prompt 怎么调度	65
7.9 量化: 省显存不等于一定更快	66
7.10 Speculative Decoding 与多 token 预测	66
7.11 RAG 部署不是向量数据库加 LLM	67
7.12 Agent 与工具调用部署	67
7.13 延迟、吞吐和用户体验	68
7.14 线上故障怎么排查	68
7.14.1 TTFT 变高	68
7.14.2 TPOT 变高	68
7.14.3 OOM 增加	69
7.14.4 回答质量突然下降	69
7.15 成本优化: 不要只说换小模型	69
7.16 高频题: 如何设计一个 LLM Serving 系统	69
7.17 高频题: 如何优化首 token 延迟	70
7.18 高频题: 如何做容量规划	70

7.19 高频题：RAG 系统上线前评估什么	70
7.20 面试中的常见失分点	71
7.21 一套完整 Deployment 面试回答模板	71
7.22 准备清单	71
第八章：Alignment 与 Safety 面试	72
8.1 Safety 面试到底考什么	72
8.2 回答 Safety 题的七层框架	73
8.3 Alignment 和 Safety 的区别	73
8.4 Helpful、Honest、Harmless 的冲突	73
8.5 RLHF 的价值与根本局限	74
8.6 DPO 简化了什么，没有解决什么	74
8.7 Reward Hacking 和 Goodhart 定律	74
8.8 Jailbreak：绕过模型安全边界	75
8.9 Prompt Injection：LLM 应用的指令边界问题	75
8.10 Safety Eval 怎么设计	75
8.11 Over-Refusal：安全模型也会伤害体验	76
8.12 Red Teaming：主动发现未知失败模式	76
8.13 Dangerous Capability Eval	76
8.14 Privacy、Memorization 与数据治理	77
8.15 诚实性、不确定性和幻觉	77
8.16 Mechanistic Interpretability 与 Safety	77
8.17 Model Editing 与 Unlearning	78
8.18 Model Card、System Card 和发布门禁	78
8.19 Agent Safety 系统设计题	78
8.20 Safety 面试中的常见失分点	79
8.21 高频题回答模板	79
8.21.1 RLHF 为什么不是最终解	79
8.21.2 如何防 jailbreak	79
8.21.3 如何设计 safety eval	79
8.21.4 如何防 prompt injection	79
8.21.5 如何平衡 helpfulness 和 safety	79
8.22 一套完整 Safety 面试回答模板	80
8.23 准备清单	80
第九章：Multimodal 与 Research 面试	80
9.1 Multimodal 与 Research 面试到底考什么	81
9.2 回答多模态题的通用结构	81
9.3 回答研究讨论题的通用结构	81
9.4 CLIP 如何训练	82
9.5 Vision Encoder 面试怎么答	82
9.6 VLM 架构：图像如何接入 LLM	83
9.7 多模态 Instruction Tuning	83
9.8 Diffusion Model 如何生成图像	83
9.9 视频生成为什么难	84
9.10 Whisper 与语音模型	84
9.11 多模态评估怎么设计	84
9.12 多模态安全	85
9.13 如何介绍一篇多模态论文	85
9.14 如何判断多模态论文贡献是否真实	85
9.15 如何复现一个 VLM 项目	86
9.16 如何设计多模态 ablation	86
9.17 开放研究题怎么回答	86
9.18 高频研究题：最近哪篇论文重要	87
9.19 高频研究题：如何解释负结果	87
9.20 面试中的常见失分点	87

9.21 高频题回答模板	87
9.21.1 CLIP 如何训练	87
9.21.2 VLM 如何接入图像	88
9.21.3 Diffusion 如何生成图像	88
9.21.4 视频生成为什么难	88
9.21.5 如何评价一篇论文贡献	88
9.22 一套完整 Multimodal + Research 面试回答模板	88
9.23 准备清单	88
第十章: System Design 面试	89
10.1 System Design 面试到底考什么	89
10.2 一套通用答题流程	89
10.3 大模型系统的通用架构积木	90
10.3.1 接入层	90
10.3.2 编排层	90
10.3.3 模型服务层	90
10.3.4 数据与存储层	90
10.3.5 安全与权限层	90
10.3.6 观测与评估层	91
10.4 设计 ChatGPT 类服务	91
10.5 设计大模型推理平台	92
10.6 设计训练平台	92
10.7 设计 RAG 系统	93
10.8 设计 Agent 平台	93
10.9 设计多模态助手	94
10.10 设计评估平台	95
10.11 设计安全审核系统	95
10.12 面试追问怎么应对	96
10.12.1 流量扩大十倍怎么办	96
10.12.2 成本太高怎么办	96
10.12.3 质量下降怎么办	96
10.12.4 数据泄露怎么办	97
10.13 常见失分点	97
10.14 高频题回答模板	97
10.14.1 设计 ChatGPT 服务	97
10.14.2 设计 RAG 系统	97
10.14.3 设计推理平台	97
10.14.4 设计评估平台	97
10.15 一套完整 System Design 回答模板	97
10.16 面试前一页速记	98
第十一章: Behavioral Interview	98
11.1 Behavioral 面试到底考什么	99
11.2 STAR 框架: 行为题的基础结构	99
11.3 自我介绍怎么讲	99
11.4 为什么选择大模型方向	100
11.5 讲一次失败经历	100
11.6 讲一次团队冲突	101
11.7 讲一次主动性	101
11.8 讲一次快速学习	101
11.9 讲一次压力和优先级管理	102
11.10 讲一次伦理和安全判断	102
11.11 讲一次和非技术团队协作	102
11.12 如何回答“不确定”的问题	102
11.13 英文 Behavioral 面试表达	103
11.13.1 失败经历英文模板	103

11.13.2 冲突经历英文模板	103
11.13.3 为什么加入这个团队英文模板	103
11.14 反向提问怎么问	103
11.15 Behavioral 面试常见失分点	103
11.16 准备自己的故事库	104
11.17 高频题回答模板	104
11.17.1 你最大的失败是什么	104
11.17.2 你如何处理团队分歧	104
11.17.3 你如何快速学习新方向	104
11.17.4 如果发现安全风险但上线压力很大怎么办	104
11.18 最后一页速记	105
第十二章: Mock Interview 与复盘	105
12.1 Mock Interview 到底训练什么	105
12.2 一次完整 Mock 的标准流程	105
12.3 Mock 前怎么准备	106
12.4 面试官脚本: 如何给自己做 Mock	106
12.5 评分表: 不要只凭感觉判断好坏	107
12.6 错题归因: 不要只写 “不会”	107
12.7 表达问题怎么改	108
12.8 项目深挖怎么练	108
12.9 系统设计 Mock 怎么练	109
12.10 Research Mock 怎么练	109
12.11 Behavioral Mock 怎么练	110
12.12 英文 Mock 怎么练	110
12.13 Mock 频率怎么安排	111
12.14 如何判断自己准备好了	111
12.15 复盘记录模板	111
12.16 一周 Mock 闭环示例	112
12.17 最后一页速记	112
第十三章: 六个月冲刺计划	113
13.1 六个月冲刺的最终交付物	113
13.2 六个月路线总览	113
13.3 每周时间分配	114
13.4 每日最小闭环	114
13.5 第 1 月: 基础、数学和 PyTorch	114
13.6 第 2 月: Transformer 与 miniGPT	115
13.7 第 3 月: SFT、DPO、RAG 和推理实战	116
13.8 第 4 月: Training、Deployment、Alignment 专项突破	116
13.9 第 5 月: 多模态、Agent、论文和系统设计	117
13.10 第 6 月: 简历、项目包装和 Mock Interview	118
13.11 三类候选人的不同路线	119
13.11.1 应届生或基础薄弱候选人	119
13.11.2 有算法经验但非大模型背景	119
13.11.3 有工程经验但算法不强	119
13.12 每月验收表	119
13.13 最容易失败的准备方式	120
13.14 一个月压缩路线	120
13.15 三个月压缩路线	120
13.16 最后一周安排	120
13.17 最终面试准备 Checklist	121
13.18 本书的使用方式	121
13.19 最后一页速记	121

第十二册简介：OpenAI 大模型算法岗求职与面试作战手册

本书定位

第十二册服务求职和面试，不再新增单点知识，而是把知识、项目、表达和策略转化为真实面试竞争力。

为什么需要这本书

懂技术不等于能通过面试。候选人还需要知道岗位画像、简历如何写、项目如何讲、coding 怎么准备、英文表达如何组织、行为面试如何回答、mock interview 如何复盘。

适合读者

1. 准备投递大模型算法岗、训练岗、部署岗、Agent 岗和研究岗的读者。
2. 已经有项目，但不会包装和表达的人。
3. 面试前需要冲刺和查漏补缺的人。

学完后应具备的能力

1. 能写出有竞争力的大模型算法岗简历。
2. 能把项目讲成背景、目标、方案、指标、结果、bad case 和改进。
3. 能准备 coding、ML/LLM 基础、training、deployment、alignment、安全、多模态和 system design 面试。
4. 能完成 mock interview 并复盘错题。

阅读建议

建议在完成至少 2-3 个第三册项目后重点阅读，本书会帮助你把项目转成面试语言。

与其他书的关系

第十二册是全书系的求职出口，把前面所有知识和项目组织成面试策略。

第十二册：OpenAI 大模型算法岗求职与面试作战手册

本书定位

本书专门服务求职和面试。目标不是再学一个知识点，而是把知识、项目、表达和策略转化为面试竞争力。

章节文件

1. chapters/01-岗位画像.md
2. chapters/02-简历策略.md
3. chapters/03-项目包装.md
4. chapters/04-coding-interview.md
5. chapters/05-ml 与 llm 基础面试.md
6. chapters/06-training 面试.md
7. chapters/07-deployment 面试.md
8. chapters/08-alignment 与 safety 面试.md
9. chapters/09-multimodal 与 research 面试.md
10. chapters/10-system-design 面试.md
11. chapters/11-behavioral-interview.md
12. chapters/12-mock-interview 与复盘.md
13. chapters/13-六个月冲刺计划.md

第一章：岗位画像

很多人准备大模型算法岗面试时，会犯一个根本错误：把目标理解成“我要证明自己懂很多知识点”。于是疯狂背 Transformer、RLHF、LoRA、RAG、KV cache、MoE、FlashAttention、SFT、DPO、benchmark。知识当然重要，但真正的面试目标不是展示知识点数量，而是证明你能在一个高标准大模型团队里创造价值。

岗位画像就是回答一个问题：对方到底在招什么样的人？

如果你不知道岗位画像，就很容易准备错方向。比如你把大部分时间花在刷 LeetCode，但目标岗位更看重研究判断和训练经验。或者你拼命讲论文细节，但面试官更想知道你是否能把模型能力落到产品影响。又或者你只强调模型效果，却没有体现安全意识、实验严谨性和工程可靠性。

本章重点：OpenAI 当前大模型算法岗可能关注的能力：研究、训练、对齐、评估、多模态、系统、产品影响和安全意识。

说明：本章不代表任何公司的官方招聘标准，而是基于公开岗位描述、大模型行业通用要求和高水平 AI 团队的能力模型，总结一套求职准备视角。准备时应以目标岗位的最新 JD、公开博客、论文、产品和招聘沟通为准。

1.1 先建立直觉：大模型算法岗不是单一岗位

“大模型算法岗”这个词很宽。不同公司、不同团队、不同阶段，对它的定义完全不同。

有的岗位偏研究，希望你提出新方法、设计实验、写论文、推动模型能力边界。有的岗位偏训练工程，希望你能把万卡训练跑稳，定位 loss spike，优化吞吐和显存。有的岗位偏对齐，希望你理解人类偏好、安全、RLHF、DPO、红队和行为评估。有的岗位偏应用算法，希望你把模型能力转化为产品功能，比如代码助手、语音助手、企业知识库、Agent。有的岗位偏评估，希望你建立可靠 benchmark、自动评测和线上质量闭环。

所以准备面试之前，第一步不是打开资料开始学，而是判断目标岗位属于哪类画像。

可以把大模型相关岗位粗略分为八类：

岗位方向	核心价值
基础模型研究	提升模型能力边界，探索新架构、新训练方法、新对齐方法
预训练与训练系统	构建数据、训练、并行、稳定性、效率和大规模实验能力
Post-training / 对齐	让模型更有用、更安全、更符合人类偏好和产品目标
评估与模型行为	定义、测量、解释和改进模型能力与风险
多模态	处理文本、图像、语音、视频和跨模态推理
推理与部署	降低延迟和成本，提高吞吐、可靠性和可扩展性
Agent 与应用算法	把模型接入工具、记忆、规划、检索和业务 workflow
数据与数据智能	构建高质量训练、微调、偏好、评估和安全数据闭环

这些方向有交集，但面试重点不同。你必须把自己的简历、项目、表达和复习时间对齐目标方向。

1.2 高水平大模型团队到底看什么

顶级大模型团队通常不只看“你会不会某个技术”。他们更看重你是否具备长期产生高质量技术判断的能力。

可以把能力拆成八个维度。

1.2.1 研究能力

研究能力不是“读过很多论文”，而是能提出有价值的问题、设计合理实验、解释结果，并从失败中获得信息。

面试中会观察：

1. 你能否讲清一个方法为什么被提出。
2. 你能否解释它解决了什么问题。
3. 你能否说出它的适用边界。

4. 你能否指出它和相邻方法的关系。
5. 你能否设计实验验证一个假设。
6. 你能否在结果不符合预期时提出排查路径。

一个强信号是：你不是背论文结论，而是能讲清 “如果让我复现或改进这篇工作，我会怎么做”。

1.2.2 训练能力

训练能力包括数据、模型、优化器、并行、稳定性、监控、checkpoint、评估和故障排查。

面试中可能会问：

1. 训练 loss 突然 spike 怎么排查？
2. 数据重复率过高会有什么影响？
3. batch size、learning rate、warmup、weight decay 怎么影响训练？
4. 混合精度训练为什么可能不稳定？
5. ZeRO、tensor parallel、pipeline parallel、sequence parallel 分别解决什么问题？
6. 如何判断一个训练 run 是否值得继续？

高水平团队很重视训练直觉，因为大模型训练成本极高。一次错误实验可能浪费大量 GPU 时间。

1.2.3 对齐与安全意识

大模型不是只要聪明就够了，还要可控、有用、安全、诚实、稳健。

对齐与安全能力包括：SFT、RLHF、RLAIF、DPO、偏好数据、安全数据、拒答策略、红队、越狱鲁棒性、误杀控制、滥用防护、隐私保护和评估。

面试中会观察你是否能平衡两个目标：

1. 不能让模型轻易产生高风险输出。
2. 也不能让模型对正常请求过度拒答。

一个成熟候选人不会只说 “加安全分类器”，而会讨论 policy、数据、评估、红队、人工复核、线上反馈和审计闭环。

1.2.4 评估能力

评估能力是大模型岗位的核心分水岭。

因为大模型很多能力没有简单准确率。写作质量、推理能力、工具使用、幻觉、安全、长上下文、多轮一致性，都需要设计合适评估。

面试中会关注：

1. 你能否区分离线 benchmark 和线上用户体验。
2. 你是否知道 benchmark 可能被污染或过拟合。
3. 你能否设计人工评审 rubric。
4. 你能否使用 LLM-as-judge，同时理解它的偏差。
5. 你能否把评估结果转化为下一轮改进。
6. 你能否处理指标提升但用户体验下降的情况。

强候选人会把评估视为研发闭环，而不是项目最后的打分步骤。

1.2.5 系统与工程能力

很多大模型岗位要求研究和工程兼备。

你不一定要是基础设施专家，但必须理解你的算法如何在真实系统中运行。

系统能力包括：推理延迟、吞吐、KV cache、batching、模型路由、缓存、RAG、数据管道、监控、灰度、回滚和可观测性。

面试中常见观察点：

1. 你是否知道模型质量不是唯一目标。
2. 你是否考虑成本和延迟。
3. 你是否能设计可复现实验和可上线流程。
4. 你是否懂线上失败、超时、降级和监控。
5. 你是否能和平台、产品、安全团队协作。

1.2.6 产品影响

顶级团队通常不只看技术炫技，还看技术是否产生真实影响。

产品影响可以是：用户满意度提升、任务成功率提升、幻觉下降、延迟下降、成本下降、覆盖更多语言、支持新场景、安全事故减少、人工审核量下降。

面试中讲项目时，最好不要只说“我实现了一个 RAG 系统”，而要说：

我负责把企业知识库问答从纯 `prompt stuffing` 改成 `hybrid retrieval + rerank + citation` 的 RAG 链路。离线评测中引用准确率从数字不一定都要夸张，但要体现你知道如何定义和衡量影响。

1.2.7 协作与表达能力

大模型项目通常跨团队。一个模型能力改进可能涉及数据、训练、评估、产品、安全、基础设施和法律合规。

面试官会观察你是否能：

1. 讲清复杂问题。
2. 接受追问和挑战。
3. 承认不确定性。
4. 明确区分事实、假设和猜测。
5. 给出可执行下一步。
6. 和不同角色沟通 trade-off。

如果你在面试中把每个问题都讲成绝对结论，反而可能显得不成熟。大模型领域变化快，优秀候选人通常会表达边界和证据。

1.2.8 学习速度与适应能力

大模型领域变化非常快。两年前重要的技术栈，今天可能已经变成默认组件。今天热门的方法，明年可能被更简单的方法替代。

所以高水平团队会看你是否具备快速学习和迁移能力。

强信号包括：

1. 能快速理解新论文的核心贡献。
2. 能把一个领域的方法迁移到另一个领域。
3. 能从失败实验中更新判断。
4. 能持续跟踪技术变化但不盲目追热点。
5. 能把复杂问题拆成可验证的小实验。

1.3 不同岗位方向的画像

1.3.1 基础模型研究岗

基础模型研究岗关注模型能力边界。

可能涉及：新架构、预训练目标、优化方法、数据配比、长上下文、推理能力、工具使用、可解释性、多模态融合、世界模型等。

强候选人画像：

1. 有扎实数学、机器学习和深度学习基础。
2. 能读懂并批判前沿论文。
3. 有独立提出假设和设计实验的能力。
4. 能处理实验失败和不确定结果。
5. 有高质量论文、开源复现或研究型项目更好。

面试重点：论文深挖、研究品味、实验设计、结果解释、开放问题讨论。

常见弱点：只会复述论文，不会讲为什么重要；只会调参，不会形成假设；只追 SOTA，不关心机制。

1.3.2 预训练与训练工程岗

预训练与训练工程岗关注如何把大规模训练稳定、高效、可控地跑起来。

强候选人画像：

1. 熟悉 Transformer、优化器、学习率调度和训练稳定性。
2. 理解数据质量、去重、配比和污染检测。
3. 理解分布式训练和并行策略。
4. 能定位 loss spike、NaN、吞吐下降、checkpoint 异常。
5. 有大规模训练、微调或高性能训练经验。

面试重点：训练故障排查、并行策略、算力效率、实验监控、数据问题。

常见弱点：只知道算法公式，不知道训练系统；只会单卡实验，不理解大规模失败模式。

1.3.3 Post-training 与对齐岗

Post-training 关注如何把 base model 变成可用助手。

涉及 SFT、偏好数据、RLHF、DPO、reward model、安全数据、工具调用数据、多轮对话数据、风格控制和拒答行为。

强候选人画像：

1. 理解 base model、instruction tuning、preference tuning 的区别。
2. 理解偏好数据质量和标注一致性。
3. 能设计对齐实验和评估。
4. 关注 helpfulness、honesty、harmlessness 的平衡。
5. 能处理过度拒答、奖励黑客、风格退化等问题。

面试重点：RLHF/DPO 机制、数据设计、评估、安全、失败案例分析。

常见弱点：把对齐理解成单一算法；忽略数据和 policy；只看 win rate，不看副作用。

1.3.4 评估与模型行为岗

评估岗关注“我们怎么知道模型真的变好了”。

强候选人画像：

1. 能设计 benchmark 和线上指标。
2. 理解自动评测、人工评审和 LLM-as-judge 的优缺点。
3. 能分析模型行为变化。
4. 能发现数据污染、指标过拟合和评估偏差。
5. 能把评估结果变成训练、数据或产品改进建议。

面试重点：评估集构造、rubric、统计显著性、A/B 实验、错误分析。

常见弱点：只会跑公开榜单；不会定义业务指标；不会解释指标和用户体验不一致。

1.3.5 多模态岗

多模态岗关注文本、图像、语音、视频之间的理解和生成。

强候选人画像：

1. 理解视觉编码器、语言模型、跨模态对齐和生成模型。
2. 理解图文、语音、视频数据的清洗和标注。
3. 能处理 OCR、图表理解、空间关系、时序建模等问题。
4. 熟悉多模态评估和安全风险。
5. 有实际多模态项目或论文经验。

面试重点：跨模态表示、数据构造、训练策略、评估、产品场景。

常见弱点：只懂文本 LLM；不了解视觉或语音数据特性；忽略延迟和安全。

1.3.6 推理部署岗

推理部署岗关注如何让模型低成本、低延迟、高可靠地服务用户。

强候选人画像：

1. 理解 prefill、decode、KV cache、batching 和显存瓶颈。
2. 熟悉量化、蒸馏、推测解码、模型并行和 serving 框架。
3. 能分析 p95/p99 延迟和吞吐。
4. 能设计模型路由、缓存和降级。
5. 理解线上稳定性、监控和容量规划。

面试重点：性能瓶颈分析、成本优化、系统设计、线上故障处理。

常见弱点：只会说“用 vLLM”；不知道为什么快；不理解 workload 和 SLA。

1.3.7 Agent 与应用算法岗

Agent 与应用算法岗关注把模型接入真实任务。

强候选人画像：

1. 理解 RAG、工具调用、规划、记忆、状态管理和权限。
2. 能把复杂业务流程拆成模型可执行步骤。
3. 能设计失败恢复、重试、人工接管和审计。
4. 关注任务成功率而不是单轮回答质量。
5. 理解安全、成本和用户体验。

面试重点：系统设计、任务分解、工具安全、评估、线上闭环。

常见弱点：把 Agent 当成 prompt 技巧；忽略权限和失败恢复；只展示 demo，不衡量任务成功。

1.3.8 数据岗与数据智能岗

数据方向关注训练数据、微调数据、偏好数据、评估数据和安全数据。

强候选人画像：

1. 理解数据采集、清洗、去重、质量评分和污染检测。
2. 理解 data mixture 和 curriculum。
3. 能构建高质量 instruction、preference、reasoning、code、math 数据。
4. 能评估数据对模型能力的影响。
5. 关注版权、隐私、合规和治理。

面试重点：数据质量、数据实验、标注体系、数据闭环、治理。

常见弱点：只说“数据越多越好”；不理解数据分布和能力之间的关系。

1.4 OpenAI 风格岗位可能更强调什么

如果目标是 OpenAI 这类前沿大模型团队，需要特别关注几个倾向。

第一，研究和工程结合。

只会写论文但不能落地，或者只会工程实现但没有研究判断，都可能不够。前沿团队需要能把想法变成实验，把实验变成系统，把系统结果反过来推动研究判断的人。

第二，安全和社会影响意识。

大模型能力越强，风险越大。候选人需要理解模型可能被误用、可能产生偏见、可能泄露隐私、可能在高风险领域给出错误建议。安全不是面试附加题，而是核心能力之一。

第三，评估严谨性。

模型能力提升常常很微妙。没有严谨评估，就容易被 cherry-pick demo 误导。强团队会看重你是否能建立可靠证据。

第四，产品影响。

研究最终要服务模型能力和用户价值。你需要能把技术指标和真实用户问题连接起来。

第五，处理不确定性的能力。

前沿问题没有标准答案。面试官可能故意问开放问题，看你如何思考、如何分解、如何承认未知、如何提出实验。

第六，沟通清晰。

复杂项目需要跨团队协作。表达混乱会降低可信度，即使技术很强。

1.5 面试官如何从回答中判断你

同一个问题，不同回答暴露完全不同的层次。

例如面试官问：

如果一个模型在数学推理 **benchmark** 上提升了，但线上用户反馈变差，你怎么分析？

弱回答：

可能 **benchmark** 不准，再换一个 **benchmark**。

中等回答：

我会看线上问题类型是否和 **benchmark** 分布不同，也会看模型是否变得更啰嗦或更容易拒答。

强回答：

我会先确认变化是否显著，并分桶分析用户反馈：任务类型、语言、上下文长度、用户群体、模型版本、**prompt** 版本和路由策略。然

强回答体现了实验、评估、产品、系统和排查能力。

1.6 简历和面试要围绕岗位画像组织

岗位画像不是只用于理解 JD，还要反向指导你的简历和面试表达。

如果你投训练方向，简历应该突出：训练规模、数据规模、并行策略、稳定性问题、吞吐优化、loss 曲线、checkpoint、实验管理。

如果你投对齐方向，简历应该突出：SFT/RLHF/DPO、偏好数据、reward model、安全评估、红队、拒答策略、人工标注和线上反馈。

如果你投 RAG/Agent 方向，简历应该突出：检索质量、工具调用、任务成功率、权限、安全、延迟、成本和产品指标。

如果你投评估方向，简历应该突出：benchmark、rubric、LLM-as-judge、人工评审一致性、A/B、回归测试和错误分析。

如果你投多模态方向，简历应该突出：数据类型、模型结构、跨模态对齐、生成质量、时序理解、评估和延迟。

一个项目可以有多个讲法。比如同一个企业知识库项目：

训练方向讲数据质量和微调。

系统方向讲索引、检索、延迟和缓存。

评估方向讲 benchmark、citation accuracy 和人工评审。

产品方向讲客服转接率下降和用户满意度提升。

面试准备的关键，就是把同一段经历重组为目标岗位最关心的证据。

1.7 候选人常见错位

1.7.1 把“知道”误认为“会做”

知道 RLHF 的流程，不等于会设计偏好数据实验。

知道 FlashAttention，不等于会分析推理瓶颈。

知道 RAG，不等于会处理权限、引用、更新和幻觉。

面试官通常会通过追问把“知道概念”和“做过事情”区分开。

1.7.2 只准备热门词，不准备底层逻辑

热门词会变化，但底层逻辑稳定。

例如 LoRA、QLoRA、DPO、RAG、MoE 都是具体方法。更底层的问题是：参数高效适配、偏好优化、知识注入、条件计算、数据效率、推理效率。

如果你能讲底层逻辑，就算遇到新方法也能现场推理。

1.7.3 只讲结果，不讲过程

“准确率提升 5%”不够。面试官想知道你怎么发现问题、怎么提出假设、怎么设计实验、怎么排除干扰、怎么确认结果可信。

过程体现能力，结果体现影响。两者都要讲。

1.7.4 只讲成功，不讲失败

真实项目都会失败。强候选人能讲清失败实验带来了什么信息。

例如：某个 reranker 离线提升但线上变慢，最后改成只对 top-50 候选重排；某个 DPO 数据提升 win rate 但导致过度拒答，后来重新分桶安全样本和 helpfulness 样本。

1.7.5 忽略安全和合规

在大模型岗位中，安全和合规不是“加分项”，而是底线。尤其是面向真实用户、企业数据、多模态输入、工具调用和自动化执行的系统。

1.8 如何读 JD

读 JD 时不要只看关键词，要拆成能力要求。

例如 JD 写：

Experience with large-scale training and post-training of foundation models.

这背后可能意味着：

1. 你懂预训练和 post-training 的区别。
2. 你有训练实验经验。
3. 你理解大规模训练的稳定性和效率。
4. 你知道如何评估模型能力变化。
5. 你能在数据、算法和系统之间做 trade-off。

JD 写：

Build evaluations for model behavior and safety.

这背后可能意味着：

1. 你能定义行为类别。
2. 你能构造评估数据。
3. 你能设计 rubric。
4. 你理解自动评估和人工评估的偏差。
5. 你能把评估接入模型迭代和上线门禁。

JD 写：

Improve product experiences using frontier models.

这背后可能意味着：

1. 你能理解用户场景。
2. 你能把模型能力转化为功能。
3. 你能定义产品指标。
4. 你能处理延迟、成本、安全和质量。
5. 你能和产品、设计、工程协作。

读 JD 的目标，是把它翻译成你需要提供的证据。

1.9 自我定位表

你可以用下面表格快速判断自己的准备重点。

如果你的优势是	适合重点投递	面试要补强
论文和研究实验	研究、评估、多模态	工程落地、产品影响
分布式训练和性能优化	训练、推理部署	模型行为、评估、安全
RAG/Agent 项目	应用算法、系统设计	底层模型原理、评估严谨性
数据清洗和标注体系	数据、post-training、评估	模型训练机制、实验设计
安全审核和红队	Safety、alignment、评估	基础模型和系统实现
产品落地经验	应用算法、产品算法	论文深度、模型训练细节

没有人一开始全能。关键是找到你的主战场，同时补齐会被追问的短板。

1.10 准备优先级

如果时间有限，可以按优先级准备。

第一优先级：你的简历项目。

面试官一定会围绕你的项目深挖。你必须能讲清背景、目标、方法、指标、实验、失败、取舍和影响。

第二优先级：目标岗位核心能力。

投训练就重点准备训练。投对齐就重点准备 SFT/RLHF/DPO 和安全。投 RAG/Agent 就重点准备系统设计、检索、工具和评估。

第三优先级：大模型通用基础。

Transformer、attention、优化器、预训练、微调、推理、评估、安全，这些是底座。

第四优先级：coding 和系统设计。

不同岗位权重不同，但不能完全忽略。尤其是工程型和应用型岗位。

第五优先级：行为面试。

高水平团队非常重视协作、ownership、处理失败和价值观匹配。

1.11 本书后续章节如何使用

本书后续章节会围绕求职完整链路展开。

第二章讲简历策略，重点是如何把项目写成“岗位证据”。

第三章讲项目包装，重点是如何准备项目深挖和 STAR 技术叙事。

第四章讲 coding interview，重点是算法题、工程题和代码表达。

第五到第十章分别覆盖 ML/LLM 基础、training、deployment、alignment/safety、multimodal/research 和 system design 面试。

第十一章讲 behavioral interview。

第十二章讲 mock interview 与复盘。

第十三章给六个月冲刺计划。

你可以先用本章确定目标岗位画像，再决定后续章节的阅读顺序和投入比例。

1.12 小结

大模型算法岗不是一个单一岗位，而是一组围绕基础模型、训练、对齐、评估、多模态、部署、应用和数据的能力组合。

准备面试时，不要只问“我要学哪些知识点”，而要问：

1. 目标岗位要我证明什么能力？
2. 我的经历里哪些内容能作为证据？
3. 我在哪些追问上容易暴露短板？
4. 我的项目如何连接研究、工程、评估、安全和产品影响？
5. 我如何用清晰表达让面试官相信我能在真实团队里创造价值？

岗位画像不是求职包装术，而是准备方向盘。方向对了，后面的简历、项目、知识复习和 mock interview 才不会各自发散。

第二章：简历策略

很多候选人写大模型算法岗简历时，会犯两个相反的错误。

第一种错误是把简历写成知识点清单：Transformer、RLHF、LoRA、RAG、MoE、FlashAttention、DeepSpeed、vLLM、DPO、Agent、multimodal 全部堆上去，看起来很热闹，但面试官看不出你到底做成了什么。第二种错误是把简历写成流水账：参与某项目，负责某模块，完成某功能，优化某指标，但缺少背景、难点、方法、结果和你的个人贡献。

大模型算法岗简历的核心不是“我知道什么”，而是“我在什么约束下，用什么判断和方法，解决了什么问题，产生了什么可验证影响”。

本章重点：如何写项目、论文、工程贡献、指标结果、影响力和技术关键词。

2.1 简历的本质：降低面试官判断成本

简历不是自传，也不是作品集目录。它的作用是让筛简历的人在很短时间内判断三件事：

1. 你是否和岗位方向匹配。
2. 你是否有真实、可追问的技术深度。
3. 你是否有足够强的产出信号，值得进入面试。

大模型算法岗尤其如此。因为岗位竞争激烈，很多候选人的关键词高度相似。如果你的简历只是写“熟悉 Transformer、熟悉大模型训练、熟悉 RAG、熟悉 LoRA 微调”，它并不能有效区分你和其他人。

更好的简历应该让面试官快速看到：

1. 你做过哪类问题。
2. 你的工作处在系统的哪个环节。
3. 你面对的主要约束是什么。
4. 你做了哪些关键决策。
5. 结果如何验证。
6. 如果追问，你能不能讲清细节。

所以，简历不是越长越好，也不是关键词越多越好。好的简历是高信息密度的证据链。

2.2 大模型算法岗简历应该突出什么

第 1 章讲过，大模型算法岗不是单一岗位。简历策略也必须跟岗位画像匹配。

如果目标是基础模型研究岗，简历应该突出研究问题、方法创新、实验设计、论文产出和对失败结果的理解。如果目标是训练工程岗，简历应该突出大规模训练经验、稳定性排查、吞吐优化、显存优化、分布式并行和监控体系。如果目标是对齐岗，简历应该突出 SFT、RLHF、DPO、偏好数据、安全评估、红队、拒答策略和模型行为分析。如果目标是推理部署岗，简历应该突出延迟、吞吐、显存、KV cache、batching、quantization、服务可靠性和成本。如果目标是 Agent 或应用算法岗，简历应该突出任务成功率、工具调用、检索、规划、评估闭环和产品影响。

可以用下面这张表快速对齐：

目标方向	简历重点
基础模型研究	问题定义、方法动机、实验设计、论文或复现深度
预训练与训练工程	数据、并行、优化、稳定性、吞吐、训练规模
Post-training / 对齐	SFT、偏好数据、DPO/RLHF、安全评估、模型行为
评估与模型行为	benchmark、自动评测、人评、误差分析、评估可信度
多模态	跨模态数据、编码器、对齐、生成质量、任务指标
推理部署	延迟、吞吐、成本、显存、服务稳定性、线上流量
Agent / RAG	检索质量、工具调用、任务成功率、业务闭环
数据工程	数据采集、清洗、去重、质量评分、合规、安全数据

简历最忌讳“所有方向都想沾一点”。如果每个方向都写一点，但没有任何方向能讲深，反而会让面试官觉得你没有主线。

2.3 项目经历怎么写

项目经历是算法岗简历的核心。一个项目条目最好包含五个元素：背景、目标、方法、结果、个人贡献。

可以使用下面的结构：

1. 背景：为什么要做这个项目？原始问题是什么？
2. 目标：优化什么指标？解决什么瓶颈？满足什么约束？
3. 方法：你用了什么技术路线？为什么这么选？
4. 结果：效果提升多少？成本降低多少？稳定性改善多少？

5. 贡献：哪些是你独立负责，哪些是团队协作？

差的写法：

参与大模型微调项目，使用 LoRA 对模型进行训练，提升模型效果。

这个写法的问题是信息太少。面试官不知道你微调什么模型、什么任务、什么数据、什么指标、提升多少、怎么评估、你负责哪部分。

更好的写法：

负责企业问答场景的 7B 模型指令微调，构建 8 万条领域问答与拒答数据，基于 LoRA 进行 SFT；通过数据去重、难例补充和格式约束

这个写法更好，因为它回答了几个关键问题：场景是什么，模型规模是多少，数据规模是多少，方法是什么，指标怎么变化，贡献在哪里。

再看一个训练工程方向的例子。

差的写法：

参与大模型训练平台建设，使用 DeepSpeed 提升训练效率。

更好的写法：

参与 13B 模型预训练稳定性优化，负责训练监控与异常定位模块；基于 loss、grad norm、数据 shard、吞吐和显存曲线建立告警规则。这里的重点不是“用了 DeepSpeed”，而是“你解决了什么训练问题”。

2.4 项目条目的黄金公式

对于大多数项目条目，可以套用一個简洁公式：

在 [场景/约束] 下，为解决 [问题]，我负责 [关键工作]，采用 [方法/技术]，最终使 [指标] 从 [基线] 提升/降低到 [结果]。

例如：

在客服多轮问答场景下，为解决模型答非所问和拒答不稳定问题，我负责构建偏好数据与 DPO 训练流程，结合人评和自动评测筛选 ha

这个公式的好处是，它强迫你把项目写成一个完整闭环，而不是堆技术名词。

但也要注意，不要机械套模板。真实项目里，有些结果不能公开，有些指标不能量化，有些项目偏研究探索，不一定有线上指标。这时可以改写为：

1. 用相对变化替代绝对数字，例如“降低约 30%”。
2. 用内部排名替代具体数值，例如“在同批方案中排名第一”。
3. 用实验结论替代线上指标，例如“验证了数据质量比参数量增加更影响该任务”。
4. 用可复现结果替代业务指标，例如“在三个公开 benchmark 上稳定优于 baseline”。

不要编造数字。虚假的指标很容易在追问中暴露，而且会直接伤害可信度。

2.5 如何写论文经历

如果你有论文经历，不要只写论文标题和会议。面试官更关心你在论文中真正贡献了什么。

论文经历可以从六个角度写：

1. 问题：论文解决什么问题？为什么这个问题重要？
2. 方法：核心方法是什么？和已有方法相比差异在哪里？
3. 实验：你设计了哪些关键实验？验证了哪些假设？
4. 结果：在哪些指标上提升？有没有失败或负结果？
5. 贡献：你负责方法、实验、数据、代码、写作还是分析？
6. 影响：是否开源、被引用、被复现、被产品采用或启发后续工作？

差的写法：

发表大模型相关论文一篇，提出一种新的微调方法，在多个数据集上取得 SOTA。

这个写法过于抽象，而且“SOTA”如果没有具体任务和对比对象，价值有限。

更好的写法：

一作完成参数高效微调论文，针对低资源领域适配中 LoRA 容易过拟合的问题，引入基于验证集不确定性的 rank 分配策略；负责方法这个版本更清楚：问题、方法、贡献、结果、实验结论都有。

如果论文还没发表，也可以写，但要谨慎表述：

完成一项关于长上下文检索增强的研究项目，正在投稿；负责构建长文档问答评测集和错误分析，发现 chunk 粒度与 reranker 召回率 off，并提出基于问题类型的动态切分策略。

不要把“准备投稿”“计划投稿”包装成已经发表。面试官通常可以接受未发表研究，但不能接受夸大。

2.6 如何写工程贡献

很多候选人低估工程贡献的价值，以为算法岗简历只能写模型和论文。实际上，高水平大模型团队非常重视工程能力，因为大模型研发离不开大规模实验、训练系统、数据管线、评估平台和推理服务。

工程贡献不是“写了很多代码”，而是让系统更可靠、更快、更便宜、更可观测、更可复现。

常见高价值工程贡献包括：

1. 训练吞吐提升。
2. 显存占用降低。
3. 训练中断率下降。
4. checkpoint 恢复更可靠。
5. 数据处理速度提升。
6. 评估自动化覆盖更多任务。
7. 推理延迟降低。
8. 单位请求成本下降。
9. 服务可用性提升。
10. 实验复现和配置管理改善。

差的写法：

搭建模型评估平台，支持自动化评测。

更好的写法：

负责搭建大模型自动评估流水线，覆盖知识问答、代码生成、数学推理和安全拒答 4 类任务；统一 prompt 模板、采样参数和结果存储；工程贡献要尽量写出“前后对比”。没有前后对比，就很难体现价值。

2.7 指标结果怎么写

指标是简历的证据，但指标不是越多越好。好的指标应该和岗位目标相关，并且能经得起追问。

常见指标类型包括：

类型	示例
模型效果	accuracy、F1、BLEU、pass@k、win rate、人评可用率
训练效率	tokens/s、MFU、GPU 利用率、训练中断次数、收敛速度
推理效率	latency、throughput、QPS、显存、单位 token 成本
数据质量	去重率、低质样本过滤率、PII 命中率、人工抽检通过率
产品影响	任务完成率、用户采纳率、留存、人工客服转接率
安全表现	有害输出率、越狱成功率、误拒率、合规命中率

写指标时要注意四件事。

第一，要有基线。只写“准确率 86%”不如写“从 78% 提升到 86%”。没有基线，面试官不知道这个结果强不强。

第二，要说明评估方式。自动评测、人评、线上 A/B、离线 benchmark 的可信度不同。比如“人评可用率提升 10 个百分点”必须能解释人评样本怎么抽、标注标准是什么、是否有一致性校验。

第三，要避免堆小数点。简历不是实验日志。通常写整数、百分比或近似值即可。过度精确反而显得不自然。

第四，要能解释代价。一个指标提升可能带来另一个指标下降。比如更强的安全拒答可能提高误拒率，更大的 reranker 可能增加延迟，更长上下文可能提高成本。能讲 trade-off 的候选人更可信。

2.8 技术关键词怎么放

关键词很重要，因为简历筛选系统和招聘人员会搜索关键词。但关键词应该服务于经历，而不是替代经历。

可以把关键词分为三类。

第一类是基础能力关键词：Transformer、attention、optimization、distributed training、evaluation、alignment、retrieval、inference。

第二类是具体方法关键词：LoRA、QLoRA、SFT、DPO、RLHF、RAG、reranker、MoE、FlashAttention、KV cache、quantization、speculative decoding。

第三类是工具和框架关键词：PyTorch、JAX、DeepSpeed、Megatron-LM、FSDP、vLLM、TensorRT-LLM、Ray、Kubernetes、Hugging Face。

关键词的正确写法是嵌入项目上下文：

基于 PyTorch 和 FSDP 完成 7B 模型 SFT 训练，使用 LoRA 降低显存占用，并通过梯度累积和 sequence packing 提升训练吞吐。

关键词的错误写法是堆列表：

熟悉 PyTorch、FSDP、DeepSpeed、LoRA、DPO、RLHF、RAG、MoE、vLLM、FlashAttention、Agent。

如果一定要写技能栏，可以短一些，并且按方向组织：

技能：PyTorch / distributed training / SFT / DPO / RAG / evaluation / vLLM

不要把只是看过博客、跑过 demo 的技术写成“熟悉”。简历上的每个关键词都可能成为面试入口。

2.9 如何写影响力

影响力不等于“项目很大”。影响力是你的工作改变了什么。

影响力可以来自多个层面：

1. 研究影响：论文、开源、引用、被后续工作采用。
2. 工程影响：提高效率、降低成本、减少故障、提升复现性。
3. 产品影响：提升用户体验、转化率、任务完成率或满意度。
4. 团队影响：沉淀工具、规范、文档、平台，让其他人复用。
5. 数据影响：建立数据闭环，持续提升模型质量。

例如：

沉淀数据质量评估脚本和人工抽检规范，被团队后续 3 个微调项目复用，减少重复清洗工作并提高数据版本可追溯性。

这个影响力不一定惊天动地，但真实、有价值、可追问。

如果是学生或刚入门候选人，没有线上产品影响，也可以写学习和研究影响：

复现 DPO 训练流程并完成 6 组消融实验，整理训练配置、失败案例和调参经验，形成团队内部分享材料。

关键是不要把“参与过”写成“主导过”，也不要把“个人学习项目”包装成“生产系统”。诚实不等于弱，清晰表达真实贡献反而更容易获得信任。

2.10 不同背景候选人的简历策略

2.10.1 应届生

应届生通常缺少工业级项目，但可以用论文、课程项目、开源复现、竞赛和实习证明潜力。

应届生简历重点是：

1. 数学和机器学习基础是否扎实。
2. 是否能独立读论文、复现和做实验。
3. 是否有清晰的问题意识。
4. 是否能写可靠代码。
5. 是否有持续学习能力。

不要只写“阅读多篇大模型论文”。更好的写法是：

系统复现 LoRA、QLoRA 和 DPO 在 7B 模型上的训练流程，比较不同 rank、学习率和数据规模对效果的影响，整理 20 余个失败实验并分析原因。这比空泛地写“熟悉大模型微调”强很多。

2.10.2 有算法经验但非大模型背景

这类候选人的优势是机器学习、实验方法和工程经验，劣势是大模型关键词和最新技术栈可能不够。

简历策略是把过去经验迁移到大模型语境：

1. 推荐系统经验可以对应召回、排序、评估、在线实验和数据闭环。
2. NLP 经验可以对应文本建模、预训练、信息抽取、问答和评估。
3. CV 经验可以对应多模态、表示学习、数据增强和生成模型。
4. 系统经验可以对应训练平台、推理服务和分布式优化。

例如，推荐系统背景可以这样写：

负责推荐排序模型迭代，建立离线 AUC、线上 CTR 和用户分层分析闭环；该经验可迁移到大模型评估中的自动指标、人评校准和线上 A/B 测试。简历中不一定要直接写“可迁移到”，但你需要在项目表达里体现这种迁移能力。

2.10.3 工程背景转算法岗

工程背景候选人不要掩盖工程优势。大模型团队非常需要能把研究想法变成可靠系统的人。

你可以突出：

1. 分布式系统经验。
2. 高并发服务经验。
3. GPU 性能优化经验。
4. 数据平台经验。
5. 可观测性和稳定性经验。
6. 实验平台和自动化经验。

同时补上算法信号：

1. 至少一个完整的大模型训练或微调项目。
2. 至少一个评估和误差分析项目。
3. 能讲清 Transformer、SFT、DPO、RAG、推理优化等核心机制。

工程转算法的简历不能只证明“我会写代码”，还要证明“我理解模型行为和算法目标”。

2.10.4 已有大模型工业经验

如果你已经有大模型工业经验，简历要避免写得过于保守或过于笼统。

重点应该放在：

1. 规模：模型规模、数据规模、训练规模、线上流量。

2. 复杂度：稳定性、成本、安全、评估、协作约束。
3. 决策：你做过哪些关键技术选择。
4. 影响：你的工作如何影响模型能力、成本或产品。
5. 深度：你能否解释失败案例和 trade-off。

注意保密边界。不能公开的模型规模、业务数据和内部指标，可以用区间、相对变化或抽象描述。

2.11 一页还是两页

大多数候选人一页足够。如果经历较多、论文较多、工业项目较复杂，可以两页。但无论一页还是两页，都要保证第一页最强。

建议顺序如下：

1. 姓名和联系方式。
2. 一句话定位。
3. 教育或工作经历。
4. 最匹配目标岗位的 2 到 4 个项目。
5. 论文、开源、竞赛或技能。

“一句话定位”可以很有用，但不要写空话。

差的定位：

热爱人工智能，具有扎实算法基础和强烈学习能力。

更好的定位：

大模型 post-training 方向候选人，重点经历包括 7B 模型 SFT/DP0、偏好数据构建和模型安全评估。

或者：

偏训练与推理工程方向，熟悉 PyTorch/FSDP/vLLM，做过模型微调、自动评估和推理延迟优化。

一句话定位的作用是告诉面试官“请用什么视角读我的简历”。

2.12 常见简历问题

2.12.1 只写职责，不写结果

负责模型训练、数据处理和效果评估。

这句话几乎没有信息。它没有说明规模、难点、方法和结果。

2.12.2 只写结果，不写方法

将模型准确率提升 15%。

这句话看起来强，但如果没有说明怎么做到，面试官会怀疑可信度。

2.12.3 技术词过密

一条经历里塞太多技术词，会显得像搜索引擎优化，而不是实际项目。

2.12.4 贡献边界不清

“参与”“负责”“主导”含义不同。不要把团队成果全部写成个人成果。

2.12.5 项目太多但都很浅

与其写 8 个浅项目，不如写 3 个能讲深的项目。面试通常会围绕你最强的项目追问。

2.12.6 没有目标岗位意识

投训练岗却把主要篇幅写成前端产品项目，投对齐岗却完全不提偏好数据和安全评估，都会降低匹配度。

2.13 简历和面试是一体的

写简历时要提前想清楚：面试官会怎么追问？

每个项目至少准备回答以下问题：

1. 为什么做这个项目？
2. 最初 baseline 是什么？
3. 你尝试过哪些方案？
4. 哪些方案失败了？为什么？
5. 最关键的改进来自哪里？
6. 指标是否可靠？有没有数据泄漏或评估偏差？
7. 如果再做一次，你会怎么改？
8. 这个项目和目标岗位有什么关系？

如果一个项目写在简历上，但你无法回答这些问题，就应该删掉或弱化。简历不是越满越好，而是每一项都能成为高质量对话的入口。

2.14 简历自检清单

投递前可以逐条检查：

1. 第一眼能否看出我的目标方向？
2. 最强的 2 到 3 个项目是否放在最显眼位置？
3. 每个项目是否包含背景、方法、结果和个人贡献？
4. 是否有可验证指标或清晰实验结论？
5. 技术关键词是否嵌入真实经历，而不是孤立堆砌？
6. 是否删除了和目标岗位无关的低价值内容？
7. 是否避免夸大论文、开源、指标和个人贡献？
8. 是否能对每个关键词进行 5 分钟以上追问？
9. 是否遵守保密和合规边界？
10. 是否针对不同岗位版本做过调整？

2.15 本章小结

大模型算法岗简历的核心是证据链。

你不需要证明自己懂所有热门概念，而要证明自己能在具体问题中做出有效判断。项目经历要讲清背景、目标、方法、结果和贡献。论文经历要讲清问题、方法、实验和你的真实角色。工程贡献要讲清效率、稳定性、成本和可复用影响。指标要有基线、评估方式和 trade-off。关键词要服务于经历，而不是替代经历。

一份好的简历会让面试官产生明确判断：这个候选人有主线，有真实经历，有可追问的技术深度，并且和岗位匹配。

下一章我们会继续讨论项目包装。简历解决的是“如何让别人愿意面试你”，项目包装解决的是“进入面试后，如何把一个项目讲成有深度、有逻辑、有说服力的技术故事”。

第三章：项目包装

简历让你进入面试，项目讲述决定面试官是否相信你真的做过、做懂了、做出了价值。

很多候选人的项目本身并不差，但讲出来很弱。常见表现是：从背景讲到技术栈，再讲到结果，中间没有问题意识，没有关键决策，没有失败和反思。面试官听完只能知道“你参与过一个项目”，但不知道你是不是核心贡献者，也不知道你是否具备独立解决复杂问题的能力。

项目包装不是夸大，也不是把普通项目讲成惊天动地的大项目。项目包装的本质是把真实经历组织成清晰、可信、可追问的技术故事。

本章重点：把项目讲成问题、方法、实验、结果、失败、反思和下一步。

3.1 什么是好的项目讲述

一个好的项目讲述，应该让面试官在 3 到 5 分钟内听懂七件事：

1. 你解决的是什么问题。
2. 这个问题为什么重要。
3. 难点和约束在哪里。
4. 你尝试了哪些方案。
5. 为什么最终选择这个方案。
6. 结果如何验证。
7. 你从中学到了什么。

如果只能讲清“用了什么技术”，说明项目还停留在工具层。如果能讲清“为什么这样做、为什么不是别的方案、失败在哪里、指标为什么可信”，才说明你真正理解项目。

大模型算法岗的项目讲述尤其看重三类能力：

1. 技术判断：能不能在很多方法中选择合适方案。
2. 实验能力：能不能设计实验、解释结果、排查异常。
3. 系统意识：能不能考虑数据、模型、评估、成本、安全和产品约束。

3.2 项目讲述的标准结构

推荐使用一个七段式结构：

1. 背景：业务、研究或工程场景是什么。
2. 问题：原系统或 baseline 有什么不足。
3. 难点：为什么这个问题不好解决。
4. 方法：你做了哪些关键设计。
5. 实验：你如何验证方法有效。
6. 结果：指标、现象和影响是什么。
7. 反思：失败、边界和下一步是什么。

这不是固定模板，而是一种组织逻辑。实际面试中，你可以先用 1 分钟讲概览，再根据面试官兴趣展开。

一个简洁开场可以这样说：

我想讲一个 7B 模型在企业问答场景中的 post-training 项目。这个项目的目标不是单纯提高 benchmark 分数，而是降低幻觉和答非所

这段开场包含场景、目标、角色、方法和结果。面试官可以立刻决定追问数据、训练、评估或失败案例。

3.3 背景：不要讲太久，但要讲清意义

背景的作用是让面试官知道项目为什么存在。背景不需要讲公司历史、团队架构或无关业务细节，只需要交代和技术问题相关的信息。

好的背景通常包括：

1. 使用场景：模型用于什么任务。
2. 用户或系统需求：为什么现有方案不够。
3. 约束条件：数据、算力、延迟、安全、成本或上线周期。
4. 项目目标：希望改善什么。

差的背景：

我们公司有一个客服系统，业务比较复杂，有很多用户问题，所以我们做了一个大模型问答项目。

更好的背景：

项目面向企业客服知识库问答。原系统基于检索和模板回复，覆盖率不足；直接接入通用大模型后，回答流畅但容易出现知识库外编。这个背景讲清了场景、原方案、问题、约束和目标。

3.4 问题：从“做了什么”转向“解决什么”

项目讲述最重要的转变，是从“我做了什么”转向“我解决了什么问题”。

同样是 LoRA 微调，背后的问题可能完全不同：

- 1. 领域知识不足。
- 2. 输出格式不稳定。
- 3. 多轮对话上下文利用差。
- 4. 安全拒答不稳定。
- 5. 训练成本受限。
- 6. 小样本数据下容易过拟合。

如果你只说“我用了 LoRA”，面试官无法判断技术含量。如果你说“我们在小样本领域数据下发现全参微调成本高、LoRA 容易过拟合，所以重点做了数据去重、rank 选择和验证集早停”，技术深度就出现了。

可以用下面的问题帮助自己提炼项目核心问题：

- 1. 如果没有这个项目，会有什么损失？
- 2. 原方法最主要的失败模式是什么？
- 3. 这个问题为什么不能用简单方案解决？
- 4. 你的方案针对的是哪个失败模式？
- 5. 面试官如果只记住一句话，应该记住什么？

项目的核心问题越清楚，后面的技术细节越容易展开。

3.5 难点：展示技术判断的入口

很多候选人不敢讲难点，担心暴露项目不完美。实际上，难点是项目最有价值的部分。没有难点的项目，很难体现候选人能力。

大模型项目常见难点包括：

方向	常见难点
数据	噪声、重复、覆盖不足、标注不一致、数据泄漏、PII 和版权边界
训练	显存不足、loss spike、收敛慢、过拟合、混合精度不稳定
对齐	偏好数据难构造、奖励黑客、安全和有用性冲突、误拒率上升
评估	自动指标不可信、人评成本高、样本选择偏差、线上离线不一致
推理	延迟高、吞吐低、KV cache 占用大、batching 策略复杂
RAG	召回不足、chunk 粒度、rerank 成本、上下文污染、引用不稳定
Agent	工具调用失败、规划不稳定、状态管理复杂、长任务评估困难

讲难点时，不要只列问题，要讲你如何判断优先级。

例如：

我们最初以为主要问题是模型不懂领域知识，但错误分析发现，约一半错误来自检索召回不足和上下文冲突，只有一部分来自模型生成。这段话能体现你不是看到效果差就堆训练，而是做了错误分析和问题归因。

3.6 方法：讲决策链，不要只讲技术栈

方法部分最常见的问题是变成技术名词串联：用了 PyTorch、LoRA、DeepSpeed、RAG、FAISS、reranker、vLLM。工具本身不是重点，重点是你为什么这么设计。

好的方法讲述应该包含决策链：

1. 候选方案有哪些。
2. 选择标准是什么。
3. 为什么选当前方案。
4. 如何控制风险。
5. 如何验证是否有效。

例如，讲 SFT 和 DPO 项目时，可以这样组织：

我们先做 SFT，因为基础模型在领域格式和任务指令上不稳定。SFT 后可用率提升，但模型仍然会在知识不足时强答，所以第二阶段构造这段话回答了为什么 SFT、为什么 DPO、为什么不是 RLHF。

再比如讲推理优化：

最初瓶颈不是模型计算，而是请求长度差异导致 batch padding 浪费。因此我们先按输入长度分桶并调整 continuous batching 策略，bit 权重。

这里体现了对瓶颈定位、方案排序和质量 trade-off 的理解。

3.7 实验：让故事从“我觉得”变成“我验证过”

项目讲述必须有实验或验证。不管是研究、训练、对齐、RAG、推理还是产品项目，都需要说明你如何知道方案有效。

实验部分要回答：

1. baseline 是什么。
2. 对比方案是什么。
3. 指标是什么。
4. 数据集或样本怎么来。
5. 是否做了消融。
6. 是否排除了数据泄漏和评估偏差。
7. 结果是否稳定。

大模型项目里，评估尤其容易出问题。比如自动指标可能无法反映真实质量，LLM-as-judge 可能偏向长答案，人评可能标注不一致，benchmark 可能和训练数据污染。

因此，讲实验时最好同时讲指标和误差分析。

差的说法：

我们模型效果提升了 10%。

更好的说法：

我们用 500 条人工构造的领域问答作为固定验证集，同时每周从线上日志抽样 200 条做人工评测。自动指标主要看引用命中和格式正确。这个说法更可信，因为它说明了评估来源、指标和稳定性。

3.8 结果：讲清提升，也讲清代价

结果不是只报一个好看的数字。面试官会关心结果是否真实、是否稳定、是否有代价。

可以从四个层面讲结果：

1. 效果指标：准确率、任务完成率、win rate、人评可用率。
2. 效率指标：训练时间、吞吐、延迟、显存、成本。
3. 稳定性指标：失败率、中断次数、线上报警、回滚次数。

4. 负面影响：误拒、延迟上升、成本增加、某些子任务下降。

强候选人不会只讲“提升”，还会讲 trade-off。

例如：

DP0 后模型在安全拒答上更稳定，有害请求拒答率提升明显，但早期版本误拒率也上升了。我们后续把安全数据按风险等级拆分，并加了这个回答比单纯说“安全效果提升”更高级，因为它展示了对齐任务中常见的有用性和安全性冲突。

3.9 失败：失败经历是加分项，不是扣分项

很多面试官会问：“这个项目里最大的失败是什么？”

这不是陷阱题。面试官想看你是否真的做过项目，以及你如何从失败中学习。

一个好的失败案例应该包含：

1. 当时的假设是什么。
2. 为什么这个假设看起来合理。
3. 实验结果为什么不符合预期。
4. 你如何定位原因。
5. 后续如何调整。
6. 你从中得到什么原则。

差的回答：

项目比较顺利，没有什么失败。

这个回答通常不可信。真实项目几乎一定有失败、反复和取舍。

更好的回答：

我们一开始认为增加领域 SFT 数据一定能降低幻觉，所以扩充了很多问答对。但验证发现幻觉下降不明显，部分答案还变得更自信。这类失败案例能体现问题归因能力。

3.10 反思和下一步：展示成长性

项目讲完结果后，不要突然结束。可以用反思和下一步收尾。

反思可以包括：

1. 当前方案的边界是什么。
2. 哪些指标还不够可靠。
3. 哪些问题没有解决。
4. 如果资源更多，你会怎么做。
5. 如果重新开始，你会少走哪些弯路。

例如：

这个项目最大的局限是评估集覆盖还不够，尤其是长尾问题和多轮上下文。下一步我会优先建设更系统的错误 taxonomy，并把线上失败案例加入评估集。这个收尾能让面试官看到你不仅是只会执行任务，而是能从系统角度继续改进。

3.11 如何准备 1 分钟、3 分钟、10 分钟版本

同一个项目要准备三个版本。

3.11.1 1 分钟版本

用于自我介绍或面试官快速扫项目。

结构：场景、目标、你的角色、方法、结果。

示例：

我做过一个企业知识库问答模型优化项目，目标是降低幻觉并提高答案可用率。我主要负责数据构建、SFT/DPO 训练和评估闭环。我们

3.11.2 3 分钟版本

用于正式介绍项目。

结构：背景、问题、难点、方法、实验、结果、反思。

3 分钟版本要比 1 分钟版本多讲难点和实验，但不要展开所有实现细节。目标是给面试官留下追问入口。

3.11.3 10 分钟版本

用于深挖。

10 分钟版本要准备细节：

1. 数据怎么构造。
2. 模型和训练参数怎么选。
3. baseline 是什么。
4. 消融实验怎么做。
5. 失败案例是什么。
6. 指标如何定义。
7. 成本和延迟如何变化。
8. 线上和离线是否一致。
9. 安全和合规如何处理。
10. 如果重做会怎么改。

项目准备到 10 分钟版本，才算真正能扛住追问。

3.12 不同类型项目怎么讲

3.12.1 训练项目

训练项目要突出规模、稳定性、效率和实验判断。

重点问题包括：

1. 模型规模、数据规模和训练资源。
2. 训练目标和 tokenizer。
3. batch size、learning rate、warmup、weight decay 等超参。
4. 并行策略和显存优化。
5. loss spike、NaN、梯度异常如何处理。
6. checkpoint、恢复和监控体系。
7. 如何判断训练是否有效。

训练项目不要只说“我跑通了训练”。跑通只是起点，能解释训练过程中的异常和取舍才是重点。

3.12.2 Post-training 项目

Post-training 项目要突出数据、偏好、评估和模型行为。

重点问题包括：

1. SFT 数据从哪里来，质量如何控制。

2. 偏好数据如何构造，chosen 和 rejected 的标准是什么。
3. 为什么用 DPO、RLHF、RLAIF 或其他方法。
4. 如何平衡 helpfulness、harmlessness 和 honesty。
5. 如何评估幻觉、拒答、越狱鲁棒性和误拒。
6. 是否出现 reward hacking 或过度拒答。

讲 post-training 时，不能只讲训练 loss。模型行为变化和评估可信度更重要。

3.12.3 RAG 项目

RAG 项目要突出检索、上下文组织和答案可信度。

重点问题包括：

1. 文档如何切分和清洗。
2. embedding 模型和 reranker 如何选择。
3. 召回率、引用准确率和最终答案质量如何评估。
4. 如何处理检索不到、检索冲突和过期知识。
5. 如何控制上下文长度和延迟。
6. 是否做了 query rewrite、hybrid search 或多轮检索。

RAG 项目的常见误区是只讲向量数据库。真正难点通常在数据、评估和失败案例处理。

3.12.4 推理部署项目

推理部署项目要突出瓶颈定位、性能指标和质量 trade-off。

重点问题包括：

1. 请求模式是什么，输入输出长度分布如何。
2. 延迟和吞吐瓶颈在哪里。
3. batching、KV cache、量化、并行和 speculative decoding 如何选择。
4. 优化后质量是否变化。
5. 峰值流量、扩缩容和故障恢复如何处理。
6. 单位 token 成本如何下降。

推理项目中，面试官通常会追问具体数字和瓶颈分析。要准备好 p50、p95、QPS、显存和成本口径。

3.12.5 评估项目

评估项目要突出指标设计、可信度和闭环。

重点问题包括：

1. 评估集如何构建。
2. 自动评测和人评如何结合。
3. LLM-as-judge 如何校准。
4. 如何避免数据污染和 prompt 过拟合。
5. 如何做错误分类和回归测试。
6. 评估结果如何影响训练或产品决策。

评估项目看似不如训练项目“硬”，但在大模型团队中非常关键。能把评估讲深，是强信号。

3.13 面试官常见追问

准备项目时，可以提前模拟这些问题：

1. 这个项目里你的个人贡献是什么？
2. 你们的 baseline 是什么？
3. 为什么不用另一个方案？
4. 最大的失败是什么？

5. 指标是否可能被污染？
6. 线上效果和离线效果一致吗？
7. 如果数据规模扩大 10 倍，方案还成立吗？
8. 如果模型规模换成 70B，会有什么变化？
9. 如果算力减少一半，你会怎么取舍？
10. 如果安全要求更高，你会怎么改？
11. 这个方案的边界在哪里？
12. 你从这个项目中学到的最重要原则是什么？

这些问题的目的不是为难你，而是判断你是否真正理解项目。如果只能背准备稿，一追问就断，说明项目包装还不够。

3.14 如何把普通项目讲出深度

并不是每个人都有顶级实验室或万卡训练经历。普通项目也可以讲出深度，关键是挖掘真实问题。

比如一个简单的 LoRA 微调项目，可以从这些角度深化：

1. 数据质量：数据从哪里来，如何去重、过滤、标注。
2. 训练稳定性：学习率、rank、batch size 如何影响结果。
3. 泛化能力：是否只对验证集有效，是否有跨域测试。
4. 评估可信度：自动指标和人工评价是否一致。
5. 失败案例：模型在哪些问题上仍然失败。
6. 成本约束：为什么选择 LoRA 而不是全参微调。
7. 安全边界：是否出现不该回答的问题。

一个项目是否有深度，不完全取决于资源规模，而取决于你是否进行了认真归因和验证。

3.15 如何避免过度包装

项目包装的底线是真实。可以优化表达，但不能虚构事实。

常见过度包装包括：

1. 把参与写成主导。
2. 把 demo 写成线上系统。
3. 把阅读论文写成复现论文。
4. 把一次实验写成系统研究。
5. 把团队指标全部写成个人贡献。
6. 把未公开、不能披露的信息直接写出。
7. 把不确定结论写成确定结论。

过度包装的风险很高。面试官只要追问一个细节，比如数据规模、baseline、错误案例、训练配置、线上监控，就能看出你是否真实参与。

正确做法是诚实说明贡献边界：

这个�项目是团队合作，我主要负责数据清洗和评估部分，训练框架由同事负责。但我参与了训练结果分析，所以可以讲清数据变化对模型的影响。这种回答不会减分，反而更可信。

3.16 项目准备模板

每个核心项目都可以填写下面这张表：

问题	你的答案
项目一句话概括	
使用场景	
原始问题或 baseline	

问题	你的答案
关键难点	
你的个人贡献	
候选方案	
最终方案	
为什么不用其他方案	
数据来源和质量控制	
训练或系统实现细节	
评估指标和评估集	
主要结果	
失败实验	
trade-off	
线上或真实影响	
当前边界	
下一步改进	

如果你能把这张表填完整，并能口头讲清楚，一个项目基本就准备到位了。

3.17 本章小结

项目包装不是把项目吹大，而是把真实经历讲清楚。

好的项目讲述要从问题出发，而不是从工具出发。你需要讲清背景、问题、难点、方法、实验、结果、失败和反思。尤其要重视失败案例、评估可信度和 trade-off，因为这些地方最能体现真实经验和技术判断。

面试官真正想知道的不是你是否碰过某个热门技术，而是你能否在复杂约束下做出合理决策，并用实验验证自己的判断。

下一章进入 Coding Interview。前面三章解决“如何让别人相信你值得面试、值得深挖”，下一章开始解决“现场写代码和调试时如何证明基本功”。

第四章：Coding Interview

很多人准备大模型算法岗 coding 面试时，只想到刷 LeetCode。刷题当然有用，但大模型算法岗的 coding 面试通常不只是考“能不能写出二叉树遍历”。它更关心你能否把算法想法写成可靠代码，能否处理张量 shape，能否实现模型核心模块，能否定位训练或推理代码中的 bug。

换句话说，coding interview 不是单独的一关，而是对你“工程化算法能力”的现场验证。

本章重点：Python、数据结构、张量操作、attention、sampling、DPO loss、debug 代码。

4.1 大模型算法岗 coding 面试考什么

大模型算法岗 coding 面试通常覆盖四类能力：

1. 通用编程基本功：Python、数据结构、复杂度、边界条件。
2. 数值计算能力：NumPy、PyTorch、张量 shape、broadcast、mask、稳定性。
3. 模型实现能力：attention、MLP、LayerNorm、sampling、loss、训练循环。
4. Debug 能力：读代码、找 bug、解释错误、修复 shape、定位数值异常。

不同岗位侧重不同。

岗位方向	Coding 面试重点
基础模型研究	张量操作、模型模块、loss、实验代码清晰度
训练工程	分布式训练、性能、显存、checkpoint、debug

岗位方向	Coding 面试重点
Post-training 推理部署	SFT/DPO loss、数据处理、采样、人评数据处理 decoding、batching、KV cache、性能优化
RAG / Agent	字符串处理、检索、排序、缓存、状态管理
多模态	图像张量、padding、mask、embedding 对齐

所以准备 coding 面试时，不要只做传统算法题，也要练大模型相关的小实现题。

4.2 面试官在观察什么

面试官不只看最终代码是否通过，还会观察过程。

他们通常看：

1. 你能否先澄清输入输出。
2. 你能否处理边界条件。
3. 你能否写出简洁、可读、可测试的代码。
4. 你是否理解时间和空间复杂度。
5. 你是否能解释张量 shape。
6. 你是否能识别数值稳定性问题。
7. 代码出错后能否系统 debug。
8. 你是否知道什么时候应该向量化，什么时候可以先写清楚。

一个常见误区是沉默写代码。coding 面试不是独立考试，而是协作问题解决。你应该边写边解释关键决策，但不要每一行都碎碎念。

推荐节奏：

1. 先复述题意。
2. 明确输入输出和边界。
3. 给出初步方案。
4. 写代码。
5. 用小例子手动跑一遍。
6. 分析复杂度。
7. 如果有时间，讨论优化。

4.3 Python 基本功

大模型算法岗大量代码仍然是 Python。Python 写不好，会直接影响面试官对你工程能力的判断。

需要熟练掌握：

1. list、tuple、dict、set。
2. defaultdict、Counter、deque、heapq。
3. 排序、自定义 key、稳定排序。
4. 迭代器、生成器、enumerate、zip。
5. 字符串处理、正则基础。
6. 文件和 JSONL 数据处理。
7. 类型标注和简单类设计。
8. 异常处理和边界判断。

例如，处理一批 JSONL 训练数据时，常见需求是统计字段缺失、长度分布和重复样本。你应该能快速写出清晰代码：

```
from collections import Counter
import json
```

```

def analyze_jsonl(path: str) -> dict:
    total = 0
    missing = Counter()
    lengths = []
    seen = set()
    duplicates = 0

    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            total += 1
            obj = json.loads(line)
            for key in ["prompt", "response"]:
                if key not in obj or not obj[key]:
                    missing[key] += 1

            text = obj.get("prompt", "") + "\n" + obj.get("response", "")
            lengths.append(len(text))
            if text in seen:
                duplicates += 1
            else:
                seen.add(text)

    return {
        "total": total,
        "missing": dict(missing),
        "duplicates": duplicates,
        "avg_length": sum(lengths) / max(len(lengths), 1),
    }

```

这个例子不复杂，但体现了数据处理、边界保护和可读性。

4.4 通用数据结构与算法

大模型算法岗不一定会大量考 hard LeetCode，但常见数据结构仍然要熟。

高频主题包括：

1. 数组和双指针。
2. 哈希表。
3. 栈和队列。
4. 堆和 Top-K。
5. 二分查找。
6. 图搜索。
7. 动态规划基础。
8. 前缀和和滑动窗口。

为什么这些和大模型有关？因为很多模型系统问题最终也会退化成这些基本结构。

例如：

1. Top-K sampling 用到 top-k 选择。
2. beam search 用到候选队列。
3. RAG 检索结果合并用到排序、去重和堆。
4. tokenizer 可能涉及 trie 或贪心匹配。
5. batch packing 可能涉及区间和贪心。
6. 日志分析常用哈希统计和滑动窗口。

以 Top-K 为例：

```
import heapq
```

```
def top_k_frequent(items: list[str], k: int) -> list[tuple[str, int]]:
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return heapq.nlargest(k, counts.items(), key=lambda x: x[1])
```

面试时不仅要能写，还要能说复杂度：统计是 $O(n)$ ，堆选择是 $O(m \log k)$ ，其中 m 是不同元素数量。

4.5 张量操作是核心基本功

大模型 coding 面试中，张量操作非常重要。很多候选人概念懂，但一写 PyTorch 就出错，尤其是 shape、mask、broadcast 和 dtype。

你需要熟练掌握：

1. reshape、view、transpose、permute。
2. unsqueeze、squeeze、expand、repeat。
3. matmul、einsum、bmm。
4. gather、scatter、index_select。
5. masked_fill、where。
6. softmax、log_softmax。
7. dtype 转换和 device 管理。
8. contiguous 的含义。

张量题一定要先写 shape 注释。

例如，给定 logits 和 labels，计算 next-token cross entropy：

```
import torch
import torch.nn.functional as F
```

```
def causal_lm_loss(logits: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
    # logits: [batch, seq_len, vocab]
    # labels: [batch, seq_len]
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()
    return F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        ignore_index=-100,
    )
```

这个题看似简单，但面试官可以追问：

1. 为什么 logits 去掉最后一个 token？
2. 为什么 labels 去掉第一个 token？
3. 为什么要 contiguous？
4. ignore_index 的作用是什么？
5. 如果 labels 中 padding 没有设成 -100，会发生什么？

能答清这些问题，说明你真正理解自回归训练目标。

4.6 Attention 实现

Attention 是大模型 coding 面试最高频的模型实现题之一。

最基础的 scaled dot-product attention:

```
import math
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(q, k, v, mask=None):
    # q: [batch, heads, q_len, dim]
    # k: [batch, heads, k_len, dim]
    # v: [batch, heads, k_len, dim]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
    # scores: [batch, heads, q_len, k_len]

    if mask is not None:
        # mask should be broadcastable to scores, True means keep.
        scores = scores.masked_fill(~mask, torch.finfo(scores.dtype).min)

    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, v)
```

这段代码必须能解释:

1. 为什么除以 $\sqrt{d}$ 。
2. mask 的 shape 如何 broadcast。
3. causal mask 和 padding mask 有什么区别。
4. 为什么不用很大的负数常量更安全地适配 dtype。
5. softmax 维度为什么是最后一维。

如果面试官要求写 causal mask:

```
def causal_mask(seq_len: int, device=None) -> torch.Tensor:
    return torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool, device=device))
```

用于 attention 时通常需要变成 [1, 1, seq_len, seq_len]:

```
mask = causal_mask(seq_len, device=q.device)[None, None, :, :]
```

常见 bug 是 mask 语义反了。有的代码里 True 表示保留, 有的代码里 True 表示屏蔽。面试时一定要说清楚。

4.7 Multi-Head Attention

如果要求写完整 multi-head attention, 可以实现一个简化版本:

```
import torch
from torch import nn

class MultiHeadAttention(nn.Module):
    def __init__(self, hidden_size: int, num_heads: int):
        super().__init__()
        if hidden_size % num_heads != 0:
            raise ValueError("hidden_size must be divisible by num_heads")
        self.hidden_size = hidden_size
        self.num_heads = num_heads
        self.head_dim = hidden_size // num_heads
        self.q_proj = nn.Linear(hidden_size, hidden_size)
        self.k_proj = nn.Linear(hidden_size, hidden_size)
        self.v_proj = nn.Linear(hidden_size, hidden_size)
        self.o_proj = nn.Linear(hidden_size, hidden_size)
```

```

def _split_heads(self, x: torch.Tensor) -> torch.Tensor:
    batch, seq_len, _ = x.shape
    x = x.view(batch, seq_len, self.num_heads, self.head_dim)
    return x.transpose(1, 2)

def _merge_heads(self, x: torch.Tensor) -> torch.Tensor:
    batch, heads, seq_len, head_dim = x.shape
    x = x.transpose(1, 2).contiguous()
    return x.view(batch, seq_len, heads * head_dim)

def forward(self, x: torch.Tensor, mask: torch.Tensor | None = None) -> torch.Tensor:
    q = self._split_heads(self.q_proj(x))
    k = self._split_heads(self.k_proj(x))
    v = self._split_heads(self.v_proj(x))
    out = scaled_dot_product_attention(q, k, v, mask)
    return self.o_proj(self._merge_heads(out))

```

这个实现适合面试讲解，但真实生产中还要考虑：

1. dropout。
2. attention bias。
3. RoPE。
4. KV cache。
5. FlashAttention。
6. grouped-query attention。
7. dtype 和数值稳定性。

面试中先写清楚正确版本，再讨论工程优化，通常比一开始追求复杂实现更好。

4.8 Sampling 实现

推理相关岗位很可能考 sampling。你需要理解 greedy、temperature、top-k、top-p 的区别。

一个基础 sampling 函数：

```

import torch
import torch.nn.functional as F

```

```

def sample_next_token(logits: torch.Tensor, temperature: float = 1.0, top_k: int | None = None) -> torch.Tensor:
    # logits: [batch, vocab]
    if temperature <= 0:
        return torch.argmax(logits, dim=-1)

    logits = logits / temperature
    if top_k is not None:
        values, _ = torch.topk(logits, k=min(top_k, logits.size(-1)), dim=-1)
        threshold = values[:, -1].unsqueeze(-1)
        logits = logits.masked_fill(logits < threshold, torch.finfo(logits.dtype).min)

    probs = F.softmax(logits, dim=-1)
    return torch.multinomial(probs, num_samples=1).squeeze(-1)

```

可以继续实现 top-p：

```

def top_p_filter(logits: torch.Tensor, top_p: float) -> torch.Tensor:
    sorted_logits, sorted_idx = torch.sort(logits, descending=True, dim=-1)
    sorted_probs = F.softmax(sorted_logits, dim=-1)

```

```

cumulative = torch.cumsum(sorted_probs, dim=-1)

remove = cumulative > top_p
remove[:, 1:] = remove[:, :-1].clone()
remove[:, 0] = False

sorted_logits = sorted_logits.masked_fill(remove, torch.finfo(logits.dtype).min)
filtered = torch.full_like(logits, torch.finfo(logits.dtype).min)
return filtered.scatter(dim=-1, index=sorted_idx, src=sorted_logits)

```

面试官可能追问：

1. temperature 越大输出越随机还是越确定？
2. top-k 和 top-p 的区别是什么？
3. 为什么 top-p 至少要保留一个 token？
4. greedy decoding 是否等价于 temperature 为 0？
5. sampling 如何影响安全性和幻觉？

4.9 DPO Loss 实现

Post-training 方向很可能考 DPO loss。你不一定要背公式，但要理解 chosen、rejected、policy、reference 的关系。

DPO 的核心直觉是：相对于 reference model，policy model 应该更偏向 chosen，而不是 rejected。

简化实现如下：

```

import torch
import torch.nn.functional as F

```

```

def dpo_loss(
    policy_chosen_logps: torch.Tensor,
    policy_rejected_logps: torch.Tensor,
    ref_chosen_logps: torch.Tensor,
    ref_rejected_logps: torch.Tensor,
    beta: float = 0.1,
) -> torch.Tensor:
    # All tensors: [batch]
    policy_logratios = policy_chosen_logps - policy_rejected_logps
    ref_logratios = ref_chosen_logps - ref_rejected_logps
    logits = beta * (policy_logratios - ref_logratios)
    return -F.logsigmoid(logits).mean()

```

面试官可能继续要求你从 token logits 计算 sequence log probability：

```

def sequence_log_probs(logits: torch.Tensor, labels: torch.Tensor) -> torch.Tensor:
    # logits: [batch, seq_len, vocab]
    # labels: [batch, seq_len], ignored tokens are -100
    shift_logits = logits[:, :-1, :]
    shift_labels = labels[:, 1:]
    log_probs = F.log_softmax(shift_logits, dim=-1)

    mask = shift_labels != -100
    safe_labels = shift_labels.masked_fill(~mask, 0)
    token_logps = log_probs.gather(dim=-1, index=safe_labels.unsqueeze(-1)).squeeze(-1)
    return (token_logps * mask).sum(dim=-1)

```

要注意，这里返回的是总 log probability，不是平均 log probability。实际实现中是否按长度归一化，需要根据训练目标和代码库约定决定。

常见追问包括：

1. 为什么需要 reference model?
2. beta 控制什么?
3. chosen 和 rejected 长度不同怎么办?
4. sequence log probability 是否应该除以长度?
5. 如果 labels shift 错了会发生什么?

4.10 Debug 代码题

大模型 coding 面试中，debug 题很常见。面试官可能给一段训练代码，让你找 bug。

常见 bug 类型：

1. shape 不匹配。
2. mask 方向错误。
3. labels 没有 shift。
4. padding token 参与 loss。
5. dtype 不一致。
6. device 不一致。
7. 梯度没有清零。
8. eval 时忘记 no_grad。
9. dropout 在 eval 中仍然开启。
10. softmax 维度错误。
11. log probabilities 和 probabilities 混用。
12. inplace 操作破坏梯度。

例如下面这段代码有多问题：

```
def train_step(model, batch, optimizer):
    logits = model(batch["input_ids"])
    loss = F.cross_entropy(logits, batch["labels"])
    loss.backward()
    optimizer.step()
    return loss.item()
```

你应该能指出：

1. 没有 optimizer.zero_grad()。
2. cross_entropy 的输入 shape 不对，需要展平成 [batch * seq, vocab]。
3. 自回归 LM 通常需要 shift logits 和 labels。
4. padding token 需要 ignore_index。
5. batch tensor 可能没搬到 model device。
6. 如果 model 输出对象不是 tensor，还要取 .logits。

修正版本：

```
def train_step(model, batch, optimizer, device):
    model.train()
    input_ids = batch["input_ids"].to(device)
    labels = batch["labels"].to(device)

    optimizer.zero_grad(set_to_none=True)
    outputs = model(input_ids)
    logits = outputs.logits if hasattr(outputs, "logits") else outputs
    loss = causal_lm_loss(logits, labels)
    loss.backward()
```

```
optimizer.step()
return loss.item()
```

Debug 时的表达很重要。不要只说 “这里错了”，要说明为什么错、会导致什么现象、如何验证。

4.11 数值稳定性

大模型训练和推理中，数值稳定性非常关键。coding 面试中常见考点包括 softmax、logsumexp、混合精度和溢出。

错误写法：

```
def naive_softmax(x):
    exp_x = torch.exp(x)
    return exp_x / exp_x.sum(dim=-1, keepdim=True)
```

如果 x 很大， $\text{torch.exp}(x)$ 会溢出。稳定写法：

```
def stable_softmax(x):
    x = x - x.max(dim=-1, keepdim=True).values
    exp_x = torch.exp(x)
    return exp_x / exp_x.sum(dim=-1, keepdim=True)
```

更推荐实际使用 `torch.softmax` 或 `F.log_softmax`，因为框架实现已经处理了稳定性。

常见数值问题包括：

1. fp16 下 attention scores 过大。
2. mask 使用 $-1e9$ 在低精度下不合适。
3. loss 出现 NaN。
4. 梯度爆炸。
5. $\log(0)$ 导致 inf。
6. softmax 后再 log 不如直接 `log_softmax`。

如果面试官问 loss NaN 怎么排查，可以按顺序回答：

1. 检查输入数据是否有 NaN 或异常 token。
2. 检查 logits、loss、grad norm 从哪一步开始异常。
3. 检查学习率、warmup、梯度裁剪和混合精度 scaler。
4. 检查 mask 是否导致整行全被屏蔽。
5. 检查 labels 是否越界。
6. 尝试缩小 batch、关闭混合精度或保存最小复现 batch。

4.12 性能与内存意识

大模型代码不仅要正确，还要有性能和内存意识。

面试中不一定要你写 CUDA kernel，但会看你是否理解基本原则：

1. 避免 Python 循环处理大量数据。
2. 尽量使用向量化操作。
3. 注意中间张量的显存占用。
4. 区分 view、expand、repeat 的内存行为。
5. 知道 batch size、seq_len、hidden_size 对显存的影响。
6. 理解 attention 的 $O(\text{seq_len}^2)$ 复杂度。
7. eval 时使用 `no_grad` 或 `inference_mode`。

例如，不推荐这样计算 batch 内 token accuracy：

```
correct = 0
total = 0
for i in range(labels.size(0)):
```



```

for j in range(labels.size(1)):
    if labels[i, j] != -100:
        correct += int(preds[i, j] == labels[i, j])
        total += 1

```

更好的写法：

```

mask = labels != -100
correct = ((preds == labels) & mask).sum().item()
total = mask.sum().item()
accuracy = correct / max(total, 1)

```

这不仅更快，也更清晰。

4.13 手写小模块训练

建议准备几个常见小模块的手写实现：

1. LayerNorm。
2. RMSNorm。
3. GELU 或 SwiGLU。
4. MLP block。
5. scaled dot-product attention。
6. causal mask。
7. token-level cross entropy。
8. sequence log probability。
9. top-k / top-p sampling。
10. DPO loss。

以 RMSNorm 为例：

```

class RMSNorm(nn.Module):
    def __init__(self, hidden_size: int, eps: float = 1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: [..., hidden_size]
        variance = x.pow(2).mean(dim=-1, keepdim=True)
        x = x * torch.rsqrt(variance + self.eps)
        return self.weight * x

```

面试官可能追问 RMSNorm 和 LayerNorm 的区别：LayerNorm 会减均值再除标准差，RMSNorm 不减均值，只按均方根归一化，计算更简单，在很多 LLM 架构中常用。

4.14 面试中的代码风格

好的 coding 面试代码不一定最短，但一定清晰。

建议：

1. 函数签名先写清楚。
2. 对张量写 shape 注释。
3. 变量命名直接表达含义。
4. 边界条件显式处理。
5. 不确定 API 时说明思路。
6. 写完用小例子手动检查。
7. 有 bug 时不要慌，先定位再修改。

不建议：

- 1. 上来就写复杂优化版本。
- 2. 不解释 shape 变化。
- 3. 大量复制模板但讲不清。
- 4. 把所有逻辑塞进一行。
- 5. 出错后随机修改。

对于大模型题，shape 注释尤其重要。比如：

```
# hidden: [batch, seq_len, hidden]
# q: [batch, heads, seq_len, head_dim]
# scores: [batch, heads, seq_len, seq_len]
```

这些注释能显著降低你自己和面试官的认知负担。

4.15 如何准备

建议按三条线准备。

第一条线：通用算法题。

重点练 easy 到 medium，确保哈希、双指针、堆、二分、图搜索、动态规划基础不掉链子。不需要把大量时间投入偏竞赛的 hard 题，除非目标岗位明确强调传统算法。

第二条线：张量和模型题。

每天手写一个小模块：attention、loss、sampling、LayerNorm、DPO、sequence log probability。重点不是背代码，而是每次都解释 shape、mask、dtype 和边界。

第三条线：debug 和读代码。

找一些训练脚本、推理脚本或开源实现，练习快速回答：输入输出是什么，哪一步最可能错，如何插入检查，如何构造最小复现。

一周训练计划可以这样安排：

天数	重点
第 1 天	Python 数据结构、Top-K、字符串和 JSONL 处理
第 2 天	PyTorch shape、gather、mask、cross entropy
第 3 天	attention、causal mask、multi-head attention
第 4 天	sampling、top-k、top-p、beam search 思路
第 5 天	SFT loss、sequence log probability、DPO loss
第 6 天	debug 训练代码、NaN 排查、device 和 dtype 问题
第 7 天	模拟面试，限时讲解并手写完整题

4.16 常见现场失误

4.16.1 不澄清题意

比如输入是否为空、是否有 padding、是否 batch 化、是否需要支持不同长度。如果不问清楚，很容易写偏。

4.16.2 Shape 不写清楚

大模型题中，shape 错误是第一大问题。每次 transpose、view、gather 都要知道前后 shape。

4.16.3 忽略 mask

padding mask、causal mask、attention mask、loss mask 是不同概念。很多错误来自 mask 语义混乱。

4.16.4 忽略 dtype 和 device

CPU tensor 和 GPU tensor 混用、fp16 下使用不合适的常量、labels dtype 错误，都是常见 bug。

4.16.5 只会调用库，不会解释原理

调用 `F.cross_entropy` 没问题，但你要知道它内部相当于 `log_softmax` 加负对数似然。

4.16.6 过早优化

面试中先写正确清晰版本，再讨论优化。直接写复杂版本出错，通常得不偿失。

4.17 本章小结

大模型算法岗 coding 面试考的不是单纯刷题数量，而是你能否把算法、模型和工程基本功落到代码里。

你需要同时准备 Python 数据结构、通用算法、PyTorch 张量操作、attention、sampling、loss、DPO、debug 和数值稳定性。写代码时要先明确输入输出和边界，张量题要写 shape，模型题要讲清 mask 和 loss，debug 题要有系统排查路径。

下一章进入 ML 与 LLM 基础面试。coding 证明你能写出来，基础面试会继续考察你是否真正理解这些代码背后的概率、优化、泛化、Transformer、tokenization 和训练目标。

第五章：ML 与 LLM 基础面试

ML 与 LLM 基础面试考的不是你能不能背出很多名词，而是你能不能把大模型背后的基本假设、训练目标、优化机制和模型结构讲清楚。

很多候选人在这一关失分，不是因为完全不会，而是因为知识是碎的。比如能背交叉熵公式，但讲不清它和最大似然的关系；知道 Transformer，但讲不清 attention 为什么有效；听过 overfitting，但解释不了大模型为什么在巨量参数下仍能泛化；会说 tokenization，但不知道 tokenizer 会影响训练效率、压缩率、跨语言表现和安全边界。

基础面试的目标，是证明你不是只会调库和背论文，而是真的理解大模型是如何从数据中学习、如何被优化、如何生成文本、为什么会出错。

本章重点：概率、优化、泛化、Transformer、LLM 训练目标、tokenization、loss。

5.1 基础面试到底考什么

大模型算法岗的基础面试通常围绕六条主线：

1. 概率与统计：概率分布、条件概率、最大似然、交叉熵、KL 散度。
2. 优化：梯度下降、Adam、learning rate、batch size、正则化、收敛与稳定性。
3. 泛化：过拟合、欠拟合、数据规模、模型规模、正则化、训练集与测试集差距。
4. Transformer：self-attention、位置编码、残差、归一化、MLP、decoder-only 架构。
5. LLM 训练目标：next-token prediction、causal LM loss、SFT、偏好学习。
6. Tokenization：BPE、SentencePiece、词表大小、特殊 token、长度和多语言问题。

这些主题看似分散，其实可以用一句话串起来：大模型用 tokenizer 把文本变成 token 序列，用 Transformer 表示上下文，通过 next-token prediction 最大化训练数据的似然，再用优化器在大规模数据和算力上学习参数，最后通过评估和对齐让模型更有用、更安全。

面试时最重要的是建立这种整体图景。

5.2 回答基础题的三层结构

基础题不要只给定义。推荐用三层结构回答：

1. 直觉：这个概念解决什么问题。
2. 机制：它具体怎么工作，可以给公式或流程。
3. 边界：它有什么假设、缺点和常见坑。

例如问“什么是交叉熵”，差的回答是：

交叉熵是分类任务常用损失，用来衡量两个分布的差异。

更好的回答是：

交叉熵可以理解成用模型分布 q 去编码真实分布 p 时的平均代价。在分类或语言模型里，真实标签通常是 one-hot，所以交叉熵等价于最大化正确 token 的 log probability。它和最大似然直接相关：最小化负 log likelihood 就是在最小化训练集上的 log likelihood。

这个回答有直觉、机制和边界。

5.3 概率基础：语言模型到底在建模什么

语言模型本质上建模的是 token 序列的概率。

给定一个 token 序列：

$x_1, x_2, \dots, x_T$

自回归语言模型把联合概率分解为：

$$P(x_1, x_2, \dots, x_T) = P(x_1) P(x_2 | x_1) \dots P(x_T | x_1, \dots, x_{T-1})$$

也就是每一步根据前面的上下文预测下一个 token。

这就是 next-token prediction 的概率基础。

面试官可能问：为什么语言模型可以只预测下一个 token？

可以这样回答：

因为根据概率链式法则，任意序列的联合概率都可以分解为一系列条件概率的乘积。自回归语言模型选择按从左到右的顺序建模这些条件概率。

需要注意，next-token prediction 看似简单，但它迫使模型学习大量隐含能力：语法、事实、推理模式、代码结构、对话风格和世界知识。因为要预测下一个 token，模型必须从上下文中提取尽可能多的信息。

5.4 最大似然、负对数似然和交叉熵

最大似然估计的目标是：找到一组模型参数，让训练数据出现的概率尽可能大。

对语言模型来说，就是最大化训练集中每个真实 token 的条件概率。由于很多概率相乘会非常小，实际优化通常最大化 log probability，等价于最小化负 log likelihood。

直观地说：

1. 模型给正确 token 的概率越高，loss 越低。
2. 模型给正确 token 的概率越低，loss 越高。
3. 如果模型非常自信地预测错，loss 会很大。

在 one-hot 标签下，cross entropy 和 negative log likelihood 是等价的。

面试常见追问：为什么不用 accuracy 作为训练目标？

可以回答：

Accuracy 不可导，而且只关心预测是否排第一，不关心概率分布的置信度。交叉熵是可导的，会鼓励模型提高正确 token 的概率，并

另一个常见追问：perplexity 是什么？

Perplexity 可以理解为语言模型平均每一步有多少有效选择。它通常等于平均负 log likelihood 的指数形式。loss 越低，perplexity 越低，说明模型对真实序列越不困惑。

但 perplexity 也有边界：

1. 不同 tokenizer 下 perplexity 不一定可直接比较。
2. 低 perplexity 不一定代表回答更有用。
3. 对齐后的模型可能牺牲部分 perplexity，但交互质量更好。
4. 如果测试集被污染，perplexity 会虚高或虚低。

5.5 KL 散度和交叉熵的关系

KL 散度衡量一个分布 q 相对于真实分布 p 的偏离程度。它不是对称距离，因为 $KL(p \parallel q)$ 和 $KL(q \parallel p)$ 含义不同。

交叉熵、熵和 KL 散度之间有一个重要关系：

$$H(p, q) = H(p) + KL(p \parallel q)$$

如果真实分布 p 固定，最小化交叉熵就等价于最小化 $KL(p \parallel q)$ 。

在大模型里，KL 经常出现在：

1. 蒸馏：让学生模型接近教师模型。
2. RLHF：限制 policy 不要偏离 reference model 太远。
3. DPO：隐式地控制 policy 相对 reference 的偏移。
4. 分布漂移分析：比较训练分布和测试分布。

面试时不用把公式推得很复杂，但要讲清方向。 $KL(p \parallel q)$ 更关注 q 是否覆盖 p 中高概率的区域； $KL(q \parallel p)$ 更关注 q 自己生成的区域在 p 下是否合理。不同方向会带来不同优化偏好。

5.6 优化基础：梯度下降在做什么

深度学习训练本质上是在高维参数空间里最小化 loss。梯度告诉我们 loss 增长最快的方向，所以负梯度方向通常能让 loss 下降。

最基础的更新形式是：

$$\text{参数新值} = \text{参数旧值} - \text{学习率} * \text{梯度}$$

学习率是最重要的超参之一。

1. 学习率太大，训练可能震荡、发散或 loss spike。
2. 学习率太小，训练很慢，可能停在较差区域。
3. warmup 可以避免训练初期梯度不稳定导致参数剧烈变化。
4. decay 可以在后期更细致地收敛。

面试官可能问：为什么大模型训练常用 warmup？

可以回答：

训练初期模型参数还没有形成稳定表示，梯度统计也不稳定。如果一开始使用较大学习率，容易造成 loss spike 或发散。warmup 让

5.7 SGD、Adam 和 AdamW

SGD 使用当前 batch 的梯度更新参数。它简单、泛化性质好，但对学习率敏感，收敛可能慢。

Adam 在 SGD 基础上引入一阶矩和二阶矩估计。直观地说，它会根据梯度均值和梯度方差为不同参数自适应调整步长，因此在 Transformer 和大模型训练中非常常用。

AdamW 是 Adam 的重要改进，核心是 decoupled weight decay，也就是把权重衰减和梯度更新解耦。

面试常见问题：Adam 和 AdamW 有什么区别？

可以回答：

Adam 中如果直接把 L2 正则加到梯度里，权重衰减会和 Adam 的自适应学习率耦合，不同参数受到的衰减不再是统一的。AdamW 把 wei

还要知道 weight decay 不是对所有参数都应该加。bias、LayerNorm 或 RMSNorm 的缩放参数通常不做 weight decay，因为它们的作用和普通权重不同。

5.8 Batch Size、梯度累积和泛化

Batch size 影响训练速度、显存、梯度噪声和泛化。

大 batch 的优点：

1. 吞吐更高。
2. 梯度估计更稳定。
3. 分布式训练效率更好。

大 batch 的缺点：

1. 显存需求高。
2. 可能泛化变差。
3. 需要更仔细的学习率和 warmup 调整。

梯度累积可以在显存有限时模拟更大的 global batch。比如每张卡 micro batch 是 2，8 张卡，累积 4 步，则 global batch 是 $2 * 8 * 4 = 64$ 。

面试官可能问：梯度累积和真正大 batch 完全等价吗？

大多数情况下，如果没有 dropout 随机性差异、batch norm 统计差异、优化器 step 时机差异等问题，梯度累积可以近似等价于更大的 batch。但在实际系统中，随机性、数值误差和并行实现会带来细微差别。

5.9 泛化：为什么大模型不只是记忆

传统直觉认为，参数太多容易过拟合。但大模型常常参数极多，却在很多任务上表现出很强泛化能力。这背后有多个原因：

1. 训练数据规模巨大，覆盖了丰富模式。
2. 自监督任务提供了密集训练信号。
3. Transformer 架构有强表达能力。
4. 优化过程和正则化会影响最终解。
5. 预训练学到的表示可以迁移到下游任务。

但大模型并不是不会记忆。它可能记住训练数据中的事实、代码片段、隐私信息或 benchmark 样本。因此泛化和记忆同时存在。

面试时可以这样回答“大模型是不是在记忆训练数据”：

大模型既有记忆，也有泛化。记忆体现在它可能复现高频或重复训练样本，尤其是罕见字符串、代码和个人信息。泛化体现在它能组成 out 评估、反事实测试和任务迁移表现。

这个回答比简单说“不是记忆，是理解”更成熟。

5.10 Bias-Variance、过拟合和欠拟合

过拟合是训练集表现好、测试集表现差。欠拟合是训练集和测试集都表现差。

在大模型中，过拟合不只发生在小模型或小数据上。SFT、DPO、小规模领域微调、评估集迭代过多时都可能过拟合。

常见现象：

1. 训练 loss 持续下降，验证 loss 上升。

2. 固定评估集提升，线上样本不提升。
3. 模型学会固定格式，但真实任务能力没提升。
4. 对安全数据过拟合，导致大量误拒。

应对方法包括：

1. 增加和清洗数据。
2. 使用验证集和早停。
3. 降低学习率或训练轮数。
4. 加强数据多样性。
5. 做交叉评估和 held-out 测试。
6. 对评估集做污染检查。

5.11 Transformer 的核心直觉

Transformer 的核心思想是：让序列中每个位置根据上下文中其他位置的信息更新自己的表示。

RNN 按时间顺序处理 token，长距离依赖难训练且并行性差。CNN 局部感受野强，但要堆很多层才能建模长距离关系。Self-attention 允许任意两个 token 直接交互，因此更适合长文本建模，并且训练时更容易并行。

Transformer block 通常包含：

1. Self-attention：聚合上下文信息。
2. MLP：对每个位置做非线性变换。
3. 残差连接：帮助梯度传播和保留原始信息。
4. LayerNorm 或 RMSNorm：稳定训练。
5. 位置编码：让模型知道 token 顺序。

面试官问 “attention 为什么有效”，可以回答：

Self-attention 为每个 token 动态选择上下文中最相关的信息。和固定窗口或顺序递归不同，attention 权重依赖当前输入，因此同

5.12 Self-Attention 的机制

Self-attention 中，每个 token 的 hidden state 会投影成 query、key、value。

直觉如下：

1. query 表示 “我想找什么信息”。
2. key 表示 “我能提供什么索引”。
3. value 表示 “如果被关注，我提供什么内容”。

计算流程：

1. query 和 key 做点积，得到相关性分数。
2. 分数除以维度平方根，避免数值过大。
3. 加 mask，屏蔽未来 token 或 padding。
4. softmax 得到 attention 权重。
5. 用权重加权 value，得到上下文表示。

为什么要除以维度平方根？

如果 query 和 key 维度很大，点积的方差会随维度增大，softmax 输入过大后容易饱和，梯度变小。除以 $\sqrt{d}$ 可以稳定分数尺度。

为什么 decoder-only LLM 需要 causal mask？

因为训练目标是根据过去预测未来。如果不加 causal mask，模型在训练时可以看到答案 token，造成信息泄漏，训练 loss 会虚低，推理时无法复现这种条件。

5.13 Multi-Head Attention 为什么需要多头

单头 attention 只能在一个表示子空间里计算相关性。多头 attention 把 hidden state 分成多个 head，让不同 head 学习不同关系。

例如：

1. 有的 head 关注局部语法。
2. 有的 head 关注长距离依赖。
3. 有的 head 关注括号或代码结构。
4. 有的 head 关注实体指代。

当然，不要把每个 head 的功能解释得过于神秘。实际模型中 head 的可解释性有限，有些 head 可能冗余，可以被剪枝或合并。GQA 和 MQA 就是在推理效率和表示能力之间做取舍。

面试时可以补充：多头不是简单增加参数，而是让模型在多个子空间并行建模关系；但更多 head 不一定总是更好，因为 head_dim、显存、通信和推理成本也会变化。

5.14 位置编码：为什么 Transformer 需要顺序信息

Self-attention 本身对输入顺序不敏感。如果不加入位置信息，模型很难区分“狗咬人”和“人咬狗”。

常见位置方法包括：

1. 绝对位置编码。
2. 相对位置编码。
3. RoPE。
4. ALiBi。

大模型中 RoPE 很常见。它通过旋转 query 和 key，把相对位置信息融入 attention 分数。RoPE 的优势是适合自回归模型，能较好表达相对位置关系，并且在一定程度上支持长度外推。但长度外推不是无条件可靠，通常还需要训练长度、缩放策略和评估配合。

面试官可能问：位置编码会影响长上下文能力吗？

可以回答：

会。位置编码决定模型如何表示 token 间距离。训练时见过的最大长度、RoPE scaling、attention 实现、数据分布和评估任务都会影响。

5.15 LayerNorm、RMSNorm 和残差连接

归一化和残差连接是深层 Transformer 稳定训练的关键。

残差连接让每层学习对表示的增量修改，而不是完全重写表示。它有助于梯度传播，也让深层网络更容易优化。

LayerNorm 会对每个 token 的 hidden 维度做归一化，减均值、除标准差，再乘可学习缩放和偏置。RMSNorm 不减均值，只按均方根缩放，计算更简单，在很多 LLM 中常见。

Pre-Norm 和 Post-Norm 也是高频考点：

1. Post-Norm 是先子层，再残差，再归一化。
2. Pre-Norm 是先归一化，再子层，再残差。

大模型常用 Pre-Norm，因为训练更稳定，梯度更容易流过残差路径。但 Post-Norm 在某些设置下也有优势。

5.16 MLP、激活函数和 SwiGLU

Transformer block 里除了 attention，还有 MLP。Attention 负责 token 间信息交互，MLP 负责对每个位置的表示进行非线性变换和特征组合。

早期 Transformer 常用 ReLU 或 GELU。很多现代 LLM 使用 SwiGLU 或 GeGLU 这类门控激活。

SwiGLU 的直觉是：一部分投影产生候选特征，另一部分投影产生门控信号，用门控控制哪些特征通过。这通常能提升表达能力，但会增加一定计算量。

面试时可以这样回答“MLP 在 Transformer 中有什么作用”：

Attention 主要做跨 token 的信息混合，MLP 主要做每个 token 位置上的非线性特征变换。没有 MLP，模型只有线性投影和 attention。

5.17 Decoder-only、Encoder-only 和 Encoder-Decoder

三类 Transformer 架构要能区分。

架构	代表	适合任务
Encoder-only	BERT	理解、分类、检索、表示学习
Decoder-only	GPT 系列	自回归生成、对话、代码、通用 LLM
Encoder-Decoder	T5	条件生成、翻译、摘要

Decoder-only LLM 用 causal mask，从左到右预测下一个 token。它特别适合统一成“给定上下文，继续生成”的接口，所以成为通用大模型主流架构。

Encoder-only 不适合直接自回归生成，因为它通常双向看上下文。Encoder-decoder 在条件生成上很自然，但架构和推理缓存更复杂。

面试时不要说某一种架构绝对更好。不同架构服务不同任务和历史阶段。

5.18 Tokenization：文本如何变成模型输入

模型不能直接处理字符串，必须先通过 tokenizer 把文本变成 token id。

常见方法包括：

- 1. Word-level：按词切分，词表大，遇到未知词麻烦。
- 2. Character-level：词表小，但序列很长。
- 3. BPE：从字符开始合并高频片段。
- 4. WordPiece：类似子词方法，常见于 BERT。
- 5. SentencePiece：把文本当作原始字符流处理，常用于多语言。

大模型常用子词 tokenizer，因为它在词表大小和序列长度之间折中。

Tokenizer 会影响：

- 1. 训练效率：同样文本被切成多少 token。
- 2. 多语言表现：不同语言压缩率不同。
- 3. 代码能力：空格、缩进、符号切分很重要。
- 4. 安全边界：恶意字符串、Unicode、特殊 token 可能造成异常行为。
- 5. 上下文长度：token 数不等于字符数或词数。

面试官可能问：为什么中文或某些语言 token 更长会有问题？

可以回答：

如果 tokenizer 对某种语言压缩率差，同样语义需要更多 token，训练和推理成本更高，有效上下文更短，模型在该语言上的数据利用率低。

5.19 特殊 Token 和 Padding

常见特殊 token 包括 BOS、EOS、PAD、UNK，以及对话模型中的 system、user、assistant 分隔 token。

这些 token 看起来是工程细节，但会影响训练和生成。

- 1. EOS 学不好，模型可能不知道何时停止。
- 2. PAD 如果参与 loss，会污染训练目标。
- 3. 对话模板不一致，会导致训练和推理分布不匹配。
- 4. 特殊 token 泄漏或误用，可能造成模型行为异常。

面试中常见问题：padding token 为什么要在 loss 中 ignore？

因为 padding 不是自然文本的一部分，只是为了 batch 对齐。如果把 padding 当成真实目标训练，模型会被鼓励预测 PAD，影响语言建模目标。

5.20 LLM 训练目标：Pretraining、SFT 和偏好学习

LLM 训练通常可以分为几个阶段。

预训练阶段：用大规模文本做 next-token prediction，学习语言、知识、代码和推理模式。

SFT 阶段：用指令数据训练模型按照人类指令回答，让模型学会对话格式、任务遵循和回答风格。

偏好学习阶段：用人类或模型偏好数据，让模型更偏向有用、安全、诚实的回答。常见方法包括 RLHF、RLAIF、DPO 等。

这三个阶段目标不同：

1. 预训练让模型 “会语言、会知识”。
2. SFT 让模型 “会按指令回答”。
3. 偏好学习让模型 “更符合人类偏好和安全要求”。

面试官可能问：为什么不只做预训练？

可以回答：

预训练模型学到的是续写分布，不一定知道如何作为助手回答问题。它可能补全文档、模仿网页、输出不稳定格式。SFT 用指令数据训

5.21 Causal LM Loss 的细节

Causal LM loss 的关键是 shift。

输入 token 是：

$[x_1, x_2, x_3, \dots, x_T]$

模型在位置 1 的输出预测 x_2 ，在位置 2 的输出预测 x_3 ，以此类推。因此训练时通常用 logits 的前 $T-1$ 个位置对齐 labels 的后 $T-1$ 个 token。

常见错误：

1. logits 和 labels 没有 shift。
2. padding 没有 ignore。
3. prompt 部分和 response 部分 mask 错误。
4. 多轮对话模板和训练 labels 不匹配。
5. 评估 loss 时把不该算的 token 算进去。

SFT 中经常只对 assistant response 计算 loss，而不对 user prompt 计算 loss。原因是我们希望模型学习如何回答，而不是学习复述用户输入。

但这也不是绝对规则。有些训练设置会对完整序列算 loss，取决于数据格式和目标。

5.22 面试高频基础题与回答要点

5.22.1 为什么 Transformer 比 RNN 更适合大模型

回答要点：

1. Transformer 训练并行性更好。
2. Self-attention 更容易建模长距离依赖。
3. 残差、归一化和大规模优化配合成熟。
4. 但 attention 的序列长度平方复杂度是代价。

5.22.2 为什么 attention 要加 mask

回答要点：

1. Causal mask 防止看到未来 token。
2. Padding mask 防止关注无意义 padding。
3. mask 语义要明确 True 是保留还是屏蔽。

5.22.3 为什么 softmax 后能表示概率

回答要点：

1. softmax 输出非负。
2. 所有类别概率和为 1。
3. 输入 logit 差值决定相对概率。
4. temperature 可以调节分布尖锐程度。

5.22.4 为什么需要 validation set

回答要点：

1. 训练集 loss 不能反映泛化。
2. 验证集用于调参、早停和模型选择。
3. 过度使用验证集也会导致评估过拟合。

5.22.5 为什么模型会 hallucinate

回答要点：

1. 训练目标是预测 plausible token，不是真实性验证。
2. 预训练数据有噪声和矛盾。
3. 模型缺少外部事实校验。
4. 解码策略可能放大不确定性。
5. SFT 可能鼓励模型回答而不是承认不知道。

5.22.6 为什么 instruction tuning 有用

回答要点：

1. 预训练学会续写，不等于学会听指令。
2. 指令数据把任务格式和回答风格对齐到用户需求。
3. 它可以显著提升零样本和多任务指令遵循能力。
4. 数据质量比数量更关键。

5.23 如何表现出“基础扎实”

基础扎实不是把定义背得很流利，而是能在不同概念之间建立连接。

例如：

1. 从 next-token prediction 讲到最大似然。
2. 从交叉熵讲到 KL 散度和 perplexity。
3. 从 attention 讲到 mask、复杂度和 KV cache。
4. 从 tokenizer 讲到上下文长度和多语言公平性。
5. 从 SFT loss 讲到 prompt mask 和 assistant-only loss。
6. 从泛化讲到数据污染和评估可信度。

面试中如果你能主动补充边界条件，通常是加分项。比如讲 perplexity 时提到 tokenizer 不同不能直接比较，讲 long context 时提到扩大窗口不等于会用长上下文，讲 hallucination 时提到训练目标和事实校验之间的差距。

5.24 常见失分点

5.24.1 背概念但讲不出关系

比如知道 cross entropy、KL、maximum likelihood，但不知道它们在语言模型训练中如何连接。

5.24.2 公式会背但不会解释直觉

面试官更关心你能否把公式和模型行为联系起来。

5.24.3 忽略工程细节

基础题也会追到实现，比如 labels shift、padding mask、dtype、tokenizer 特殊 token。

5.24.4 绝对化表达

例如“模型参数越大一定越好”“perplexity 越低模型一定越强”“DPO 一定比 RLHF 好”。大模型里几乎所有结论都有条件。

5.24.5 不承认不确定性

遇到不确定问题，可以说出自己的理解和假设，而不是硬编。强候选人会区分确定知识、经验判断和需要实验验证的部分。

5.25 准备清单

准备 ML 与 LLM 基础面试时，至少要能回答下面问题：

1. 语言模型建模的概率分布是什么？
2. next-token prediction 为什么合理？
3. 最大似然、交叉熵和 NLL 有什么关系？
4. perplexity 如何解释，有什么边界？
5. KL 散度常在哪些大模型方法中出现？
6. Adam 和 AdamW 的区别是什么？
7. warmup 为什么重要？
8. batch size 如何影响训练和泛化？
9. 大模型是记忆还是泛化？
10. Transformer block 包含哪些部分？
11. attention 的 Q、K、V 分别是什么？
12. causal mask 和 padding mask 有什么区别？
13. RoPE 解决什么问题？
14. Pre-Norm 为什么常用于大模型？
15. tokenizer 如何影响模型能力？
16. PAD、EOS、BOS 分别有什么作用？
17. 预训练、SFT、DPO/RLHF 的目标有什么不同？
18. Causal LM loss 为什么要 shift？
19. 为什么模型会 hallucinate？
20. 如何判断一个基础概念在工程中是否用对？

5.26 本章小结

ML 与 LLM 基础面试考的是知识结构，而不是名词数量。

你需要把概率、优化、泛化、Transformer、tokenization 和训练目标连成一张图：文本经过 tokenizer 变成 token，Transformer 用 attention 和 MLP 建模上下文，训练目标通过 next-token prediction 最大化数据似然，优化器在大规模数据上更新参数，泛化能力来自数据、架构和优化共同作用，而 SFT 和偏好学习进一步改变模型行为。

准备这一章的关键，是每个概念都能讲清直觉、机制和边界。下一章进入 Training 面试，会把这些基础进一步落到大规模训练中的数据、分布式、loss spike、checkpoint、评估和成本估算问题上。

第六章：Training 面试

Training 面试考的不是你会不会说“预训练、SFT、ZeRO、Megatron、loss spike、checkpoint”这些词，而是你能不能像一个真正做过大规模训练的人一样，把训练目标、数据、模型、优化器、并行策略、稳定性、评估和成本串成一个闭环。

很多候选人准备 training 面试时容易走偏：背很多分布式训练名词，但讲不清为什么要做数据清洗；知道 AdamW，但讲不清学习率和 batch size 的关系；知道 loss spike，但不知道第一时间应该排查数据、数值、学习率、checkpoint 还是集群故障；知道 checkpoint，但不知道保存频率会影响故障恢复成本和训练吞吐。

大模型训练是一个系统工程。它不是“把数据喂给模型跑很久”这么简单，而是在有限预算下，稳定地把高质量 token 转化为模型能力。

本章重点：数据、预训练、分布式训练、loss spike、评估、checkpoint、成本估算。

6.1 Training 面试到底考什么

Training 面试通常围绕七条主线：

1. 训练目标：预训练、SFT、偏好学习分别在优化什么。
2. 数据工程：数据质量、去重、混合比例、采样策略如何影响模型能力。
3. 优化与稳定性：learning rate、batch size、warmup、梯度裁剪、loss spike、数值精度。
4. 分布式训练：data parallel、tensor parallel、pipeline parallel、ZeRO、FSDP、通信瓶颈。
5. 训练监控：loss、gradient norm、throughput、MFU、显存、eval 指标。
6. Checkpoint 与恢复：保存什么、多久保存、如何 resume、如何处理坏 checkpoint。
7. 成本估算：参数量、token 数、FLOPs、GPU 数、训练时长、预算。

这些主题看似分散，其实可以用一句话串起来：训练团队先定义目标能力和训练预算，再准备高质量数据，选择模型规模和训练配方，通过分布式系统高效训练，并用监控、评估和 checkpoint 保证训练过程稳定可恢复，最后根据结果迭代数据和配方。

面试官想看到的不是你背出某个框架的 API，而是你能不能回答这些问题：

1. 如果 loss 突然炸了，你怎么排查？
2. 如果训练很慢，你怎么判断瓶颈在计算、通信、I/O 还是 checkpoint？
3. 如果模型评估不好，你怎么判断是数据问题、模型规模问题、训练配方问题还是评估问题？
4. 如果预算有限，你会怎么估算能训练多大的模型、多少 token、跑多久？
5. 如果训练中断，你如何安全恢复并确认没有 silent bug？

6.2 回答 Training 题的通用结构

Training 面试建议使用“五步结构”：

1. 先定义目标：是在预训练、SFT、继续预训练，还是偏好学习阶段。
2. 再说输入输出：输入是什么数据，输出希望模型获得什么能力。
3. 解释核心机制：loss、优化器、并行方式、训练循环。
4. 讲工程边界：稳定性、吞吐、显存、通信、checkpoint、成本。
5. 给排障思路：指标异常时如何定位。

例如问“怎么做一次大模型预训练”，不要只说：

准备数据，搭模型，用 AdamW 训练，最后评估。

更好的回答是：

我会先确定目标模型规模、训练 token 数和预算，然后准备多来源高质量语料，做清洗、去重、质量过滤和数据混合。训练目标通常 token prediction 优化交叉熵。工程上需要选择并行策略，比如 data parallel、tensor parallel、pipeline parallel 或 ZeRO/FSDP。

这个回答体现了完整闭环。

6.3 预训练：模型从哪里获得能力

预训练是大模型能力的主要来源。它的目标通常是让模型在大规模 token 序列上学习语言、事实、代码、推理模式、格式和世界知识。

对于 decoder-only LLM，最常见目标是 causal language modeling：

给定前面的 token，预测下一个 token。

训练 loss 通常是所有有效 token 上的负 log likelihood：

$$\text{Loss} = - \text{average log } P(x_t \mid x_1, \dots, x_{t-1})$$

面试时要强调：预训练不是显式教模型“回答问题”，而是通过预测下一个 token 让模型压缩训练分布中的规律。问答、代码、推理和指令遵循能力，一部分来自预训练数据中的模式，一部分来自后续 SFT 和偏好学习。

预训练阶段常见设计问题包括：

1. 模型规模：参数量、层数、hidden size、attention head 数、context length。
2. 数据规模：训练 token 数、数据来源、去重、质量过滤、混合比例。
3. 训练配方：optimizer、learning rate schedule、batch size、warmup、weight decay、gradient clipping。
4. 系统策略：并行方式、混合精度、激活重计算、checkpoint、容错。
5. 评估方式：训练 loss、validation loss、下游 benchmark、污染检查、人工分析。

如果面试官问“为什么预训练 loss 降低不一定代表模型更好”，可以回答：

训练 loss 只衡量模型对训练分布的拟合程度。它下降说明模型更擅长预测训练样本中的 token，但不保证泛化到目标任务。数据重复

6.4 数据是训练的第一变量

大模型训练里，数据不是训练前的附属步骤，而是决定模型上限的核心变量。

同样的模型规模和训练预算，数据质量不同，结果可能完全不同。高质量数据可以提升知识密度、推理样式、代码能力和指令表达；低质量数据会引入重复、模板页、垃圾文本、错误事实、毒性内容、隐私风险和格式噪声。

Training 面试中关于数据，常见追问包括：

1. 为什么要去重？
2. 为什么要做质量过滤？
3. 数据混合比例怎么决定？
4. 代码、数学、网页、论文数据的作用分别是什么？
5. 如何避免 benchmark 污染？
6. 如果某个能力差，是不是只要加对应数据就行？

6.4.1 去重为什么重要

去重解决的不只是存储浪费问题。

重复数据会带来几个影响：

1. 降低有效 token 多样性：模型反复看到相同内容，浪费训练预算。
2. 增加记忆风险：模型更容易复述训练样本。
3. 污染评估：如果 benchmark 或相似题进入训练集，评估分数会虚高。
4. 扭曲数据分布：某些网站、模板或文档被过度采样。

可以这样回答：

去重的核心目的是提升有效数据多样性和评估可信度，而不只是减少数据量。精确去重可以删除完全重复样本，近似去重可以删除高

6.4.2 质量过滤不是越严越好

质量过滤的目标是提高 token 的平均价值，而不是把所有“不像百科全书”的文本都删掉。

过松会保留垃圾、广告、乱码、模板页和低信息密度文本；过严会损失长尾知识、多语言数据、口语对话、代码注释、论坛问答和真实用户表达。

常见质量信号包括：

1. 文本长度、字符分布、乱码比例。
2. 重复 n-gram 比例。
3. 语言识别结果。
4. 毒性、成人、暴力、隐私等安全标签。
5. perplexity 或小模型质量评分。
6. 来源域名、文档类型、人工标注质量。

面试回答要体现 trade-off：

质量过滤不是简单地越严格越好。预训练数据需要覆盖真实语言分布和长尾知识，如果只保留高质量百科和论文，模型可能表达单一、

6.4.3 数据混合比例决定能力形状

数据混合比例决定模型“长成什么样”。

网页数据提供广覆盖，书籍提供长文本和叙事结构，论文提供专业知识，代码提供形式化结构和程序能力，数学数据提供推理样式，对话数据提供交互格式。

如果代码比例高，模型可能代码能力强，但自然语言风格受影响；如果英文比例过高，多语言能力会弱；如果短文本过多，长上下文能力可能不足；如果高质量推理数据少，模型可能只会流畅表达，不擅长复杂推理。

面试中可以这样说：

数据 mixture 不是静态经验值，而是训练配方的一部分。通常会先根据目标能力设定初始比例，例如通用网页、书籍、代码、数学、

6.5 SFT 与预训练的区别

SFT，也就是 supervised fine-tuning，通常在预训练之后进行。它的目标不是重新学习世界知识，而是让模型学会按照指令格式完成任务。

预训练数据通常是自然文本，目标是预测下一个 token。SFT 数据通常是 instruction-response、multi-turn dialogue、tool-use trace 或专家示范，目标是让模型在给定指令时输出符合人类期望的回答。

预训练像是让模型读完整个图书馆，SFT 像是教模型如何参加面试、如何回答用户、如何遵守格式。

常见面试问题：SFT 会不会让模型学到新知识？

可以回答：

SFT 可以注入一些新知识，但它更主要的作用是改变模型行为和输出格式。因为 SFT 数据量通常远小于预训练数据，且覆盖范围有限，

SFT 训练中要注意：

1. 数据质量比数量更重要。
2. 响应格式要一致，否则模型会学到混乱风格。
3. 多轮对话要正确 mask loss，避免把用户输入也当作需要生成的目标。
4. 要平衡 helpfulness、factuality、safety 和 verbosity。
5. SFT 可能带来能力遗忘，需要保留通用能力评估。

6.6 Optimizer、学习率和 batch size

训练面试经常从优化器开始追问。

大模型训练常用 AdamW。它相比 SGD 更适合稀疏、高维、非平稳的深度网络优化；相比 Adam，AdamW 把 weight decay 从梯度更新中解耦，通常更稳定。

但面试中不要只背 “AdamW 好”。要讲清几个关键点：

1. 学习率决定每一步参数更新幅度。
2. warmup 用来避免训练早期梯度和动量估计不稳定。
3. decay schedule 用来在后期减小更新幅度，提高收敛质量。
4. batch size 影响梯度噪声、吞吐和泛化。
5. gradient clipping 用来限制异常梯度，减少 loss spike 风险。

如果问 “为什么大模型训练要 warmup”，可以回答：

训练刚开始时，模型参数随机，梯度分布不稳定，Adam 的一阶和二阶矩估计也还不可靠。如果一开始就用大学习率，参数可能被异常

如果问 “batch size 越大越好吗”，可以回答：

不是。更大的 global batch size 可以提高硬件利用率，降低梯度噪声，并支持更稳定的吞吐，但它也可能降低更新步数，影响泛化，

6.7 分布式训练：为什么不能只用 Data Parallel

当模型和 batch 足够大时，单卡无法容纳模型、激活、优化器状态和 batch 数据，需要分布式训练。

常见并行方式包括：

1. Data Parallel：每张卡放一份完整模型，处理不同 batch，反向后同步梯度。
2. Tensor Parallel：把单层矩阵运算切到多张卡上。
3. Pipeline Parallel：把不同层放到不同设备，形成流水线。
4. ZeRO/FSDP：切分参数、梯度和优化器状态，减少每卡显存占用。
5. Sequence Parallel 或 Context Parallel：针对长上下文切分序列维度。

6.7.1 Data Parallel

Data Parallel 最直观：每张 GPU 复制一份模型，不同 GPU 处理不同样本，反向传播后做 all-reduce 同步梯度。

优点是简单、扩展性好；缺点是每张卡都要放完整模型和优化器状态。当模型很大时，显存不够。

面试回答：

Data parallel 适合模型能放进单卡、主要想扩大 batch 和吞吐的场景。瓶颈通常是梯度同步通信。模型越大，all-reduce 成本越高；网络带宽差时，扩展效率会下降。

6.7.2 Tensor Parallel

Tensor Parallel 把单层中的大矩阵乘法切到多张卡上。例如一个巨大的 linear layer，可以按列或按行切分权重矩阵。

优点是能训练单卡放不下的层；缺点是层内通信频繁，对高速互联要求高。

面试回答：

Tensor parallel 主要解决单层参数或激活太大、单卡放不下的问题。它通常需要高带宽低延迟互联，因为每层前向和反向都可能涉及

6.7.3 Pipeline Parallel

Pipeline Parallel 把不同层放到不同设备。例如前几层在 GPU 0，中间层在 GPU 1，后几层在 GPU 2。

优点是能切分深层模型；缺点是有 pipeline bubble，需要 micro-batch 提高利用率，并且调度更复杂。

可以这样回答：

Pipeline parallel 适合模型层数很多、按层切分比较自然的场景。它的核心问题是流水线空泡，设备不能一直满负载。通常会用 micro batch 把一个 global batch 拆成多个小批次，让不同 stage 同时工作。

6.7.4 ZeRO 与 FSDP

ZeRO 和 FSDP 的核心思想是：不要让每张卡都保存完整训练状态。

训练时显存主要来自四类：

1. 参数。
2. 梯度。
3. 优化器状态。
4. 激活。

对于 AdamW，优化器状态通常包括一阶矩和二阶矩，显存占用很大。ZeRO/FSDP 通过切分参数、梯度、优化器状态，显著降低每卡显存压力。

面试回答：

ZeRO/FSDP 主要解决 data parallel 下每张卡重复保存参数、梯度和优化器状态导致的显存浪费。不同 stage 切分不同训练状态。代 scatter 参数和梯度，因此通信复杂度增加。选择时要权衡显存节省和通信开销。

6.8 混合精度、数值稳定性与 loss spike

大模型训练通常使用混合精度，比如 FP16、BF16 或 FP8。混合精度可以降低显存占用、提高吞吐，但会带来数值稳定性问题。

BF16 相比 FP16 指数范围更大，通常更稳定，因此很多大模型训练偏好 BF16。FP16 可能需要 loss scaling 来避免 underflow。FP8 更节省但对训练配方和硬件支持要求更高。

6.8.1 什么是 loss spike

Loss spike 指训练过程中 loss 突然异常升高，有时会恢复，有时会导致训练发散。

它可能来自：

1. 数据异常：坏样本、超长样本、乱码、重复异常、错误 mask。
2. 学习率过大：尤其是 warmup 不足或 schedule 设计不当。
3. 梯度异常：gradient norm 突增、梯度裁剪缺失。
4. 数值问题：FP16 overflow/underflow、NaN、Inf。
5. 模型结构问题：初始化、归一化、激活函数、残差尺度。
6. 分布式问题：某个 rank 数据不同步、通信错误、checkpoint 恢复不一致。
7. 代码 bug：loss mask、position id、attention mask、label shift 错误。

6.8.2 loss spike 怎么排查

面试中如果被问“训练时 loss spike 怎么办”，推荐按优先级回答：

我会先判断 spike 是单点异常、持续发散，还是所有 rank 同时出现。然后检查最近 batch 的数据质量、长度分布、loss mask、是否这个回答有几个关键点：

1. 先分类：偶发、持续、全局、局部。
2. 再排数据：坏样本和 mask bug 很常见。
3. 再排优化：LR、gradient norm、clip。
4. 再排数值：NaN、Inf、精度格式。
5. 再排系统：rank、通信、checkpoint、硬件。
6. 最后恢复：回滚 checkpoint，谨慎重启。

6.8.3 不要只说“调小学习率”

很多候选人回答 loss spike 时只说“调小 learning rate”。这不够。

学习率确实可能是原因，但不是唯一原因。如果 loss spike 来自数据坏样本、label shift、attention mask bug、checkpoint 损坏或某个 rank 同步错误，调小学习率只是掩盖问题。

更好的表达是：

调小学习率是一种缓解手段，但我会先定位 **spike** 的类型和触发条件。如果 **spike** 总是在某类数据出现，优先看数据和 **mask**；如果发

6.9 训练监控：哪些指标必须看

训练不是启动一个 job 然后等结果。大规模训练必须持续监控。

核心指标包括：

1. Train loss: 训练集上的 next-token loss。
2. Validation loss: 验证集 loss，最好按领域拆分。
3. Learning rate: 确认 schedule 是否符合预期。
4. Gradient norm: 监控梯度爆炸、异常 batch。
5. Weight norm 或 update norm: 看参数更新是否异常。
6. Throughput: tokens/sec 或 samples/sec。
7. MFU: Model FLOPs Utilization, 衡量硬件有效利用率。
8. GPU 显存: 判断是否接近 OOM。
9. GPU utilization: 判断是否计算瓶颈或等待瓶颈。
10. Communication time: 判断分布式通信瓶颈。
11. Data loading time: 判断 I/O 和 dataloader 是否拖慢。
12. Checkpoint time: 判断保存是否影响训练效率。
13. NaN/Inf count: 捕捉数值异常。

面试时可以强调：单一指标不可靠。

例如：

1. loss 正常但 throughput 下降，可能是 I/O 或通信瓶颈。
2. throughput 正常但 validation loss 变差，可能是过拟合或数据分布问题。
3. train loss 降得快但 benchmark 不涨，可能是训练数据和评估目标不匹配。
4. GPU utilization 低，可能是 dataloader、通信、checkpoint 或 pipeline bubble。

6.10 Evaluation：训练中如何判断模型真的变好了

大模型训练不能只看最终 loss。

评估通常分为四类：

1. 语言建模指标: train loss、validation loss、perplexity。
2. 通用能力 benchmark: 知识、推理、数学、代码、多语言。
3. 领域评估: 目标业务或研究方向的数据集。
4. 人工分析: 真实 prompt、失败案例、安全边界、输出风格。

Training 面试中要注意一个点：评估本身也可能有问题。

常见评估风险包括：

1. Benchmark contamination: 评估题进入训练集。
2. Prompt 格式不一致: 不同模板导致分数不可比。
3. Decoding 参数不一致: temperature、top-p、max tokens 影响结果。
4. 样本量太小: 结果波动大。
5. 只看平均分: 掩盖某些能力退化。
6. 数据语言和领域偏置: 不能代表真实用户。

如果问“validation loss 降了，但 benchmark 没涨，怎么办”，可以回答：

我会先确认 benchmark 评估流程是否稳定，包括 prompt、解码参数、样本量和污染检查。然后看 validation set 是否代表目标能力。

6.11 Checkpoint：训练工程的安全绳

Checkpoint 是大规模训练的安全绳。训练越贵，checkpoint 越重要。

一个完整 checkpoint 通常包括：

1. 模型参数。
2. 优化器状态。
3. 学习率 scheduler 状态。
4. random seed 和 RNG state。
5. dataloader 或 sampler 状态。
6. 当前 step、token 数、训练配置。
7. 分布式切分信息。

只保存模型参数，通常不足以无缝 resume 训练。因为 optimizer state、scheduler 和数据采样位置不一致，可能导致训练轨迹改变。

6.11.1 checkpoint 保存频率怎么定

保存太频繁，会占用 I/O、存储和训练时间；保存太少，故障后损失太大。

需要权衡：

1. 集群故障率。
2. 单次 checkpoint 写入耗时。
3. 存储成本。
4. 可接受的重算 token 数。
5. 训练总预算。

可以这样回答：

checkpoint 频率取决于故障率、保存开销和可接受的重算成本。如果集群不稳定或训练成本很高，需要更频繁保存；如果 checkpoint

6.11.2 resume 后要检查什么

恢复训练后不能只看 job 跑起来了。

要检查：

1. loss 是否与恢复前连续。
2. learning rate 是否正确。
3. optimizer step 是否正确。
4. gradient norm 是否异常。
5. 数据是否重复或跳过。
6. 分布式 rank 和 shard 是否一致。
7. eval 指标是否没有突然漂移。

面试回答：

resume 后我会重点看 loss 曲线是否连续、LR 和 step 是否对齐、optimizer state 是否恢复、数据 sampler 是否从正确位置继续，以

6.12 成本估算：训练到底要多少钱

Training 面试非常喜欢问成本估算，因为它能区分“只懂概念”和“懂工程”的候选人。

训练成本可以从几个变量估算：

1. 参数量 N。
2. 训练 token 数 D。
3. 每 token 训练 FLOPs。
4. GPU 峰值算力。
5. 硬件利用率。

6. GPU 数量。
7. 单价和训练时长。

对 dense decoder-only Transformer，一个常用粗略估算是：训练 FLOPs 约等于：

$6 * \text{参数量} * \text{训练 token 数}$

这只是粗略估算，不包括所有 attention、embedding、通信、checkpoint、数据加载和低利用率损耗，但面试中足够用于数量级判断。

例如，一个 7B 模型训练 1T tokens：

$\text{FLOPs} \approx 6 * 7\text{e}9 * 1\text{e}12 = 4.2\text{e}22$

如果 GPU 理论算力是每张 300 TFLOPs，实际利用率 40%，有效算力为：

$300\text{e}12 * 0.4 = 1.2\text{e}14 \text{ FLOPs/s}$

如果使用 1024 张 GPU，总有效算力约为：

$1024 * 1.2\text{e}14 = 1.23\text{e}17 \text{ FLOPs/s}$

训练时间约为：

$4.2\text{e}22 / 1.23\text{e}17 \approx 341000 \text{ seconds} \approx 95 \text{ hours} \approx 4 \text{ days}$

这个估算非常粗，但能说明思路。

面试时要补一句边界：

这个公式是数量级估算，实际训练时间还会受 sequence length、attention 实现、并行策略、通信、checkpoint、数据加载、故障重

6.13 Scaling Law：模型、数据和算力怎么权衡

Scaling law 的核心启发是：模型效果会随模型规模、数据规模和计算量呈规律性改善，但三者需要匹配。

如果模型很大但数据不足，会 undertrain，训练预算没有充分利用；如果数据很多但模型太小，模型容量可能不足；如果算力有限，需要在模型参数量和训练 token 数之间做 trade-off。

面试中可以这样讲：

训练不是模型越大越好，也不是 token 越多越好，而是在给定 compute budget 下选择合适的模型规模和数据规模。Chinchilla 之后，optimal training，也就是在相同算力下，用相对更小但训练 token 更多的模型，可能比参数更大但训练不足的模型效果更好。

还要注意：scaling law 是趋势，不是万能公式。

它不直接解决：

1. 数据质量差异。
2. 推理、代码、数学等具体能力差异。
3. SFT 和偏好学习后的行为变化。
4. 评估污染和 benchmark 偏差。
5. 长上下文、多模态、工具使用等特殊能力。

6.14 高频题：如何设计一次继续预训练

继续预训练常用于领域适配，例如医学、法律、金融、代码、数学或企业内部知识。

面试官可能问：如果已有一个 base model，如何继续预训练提升某领域能力？

可以按这个结构回答：

我会先明确目标领域和评估集，确认是知识缺失、术语不熟、风格不对，还是任务格式问题。然后准备领域数据，做清洗、去重、隐私脱敏等。关键点：

1. 继续预训练适合补领域分布和知识。
2. SFT 更适合补指令格式和回答风格。

3. 要混入通用数据防止遗忘。
4. 要有领域 eval 和通用 eval。
5. 要控制学习率，避免破坏已有能力。

6.15 高频题：训练慢怎么排查

如果面试官问“训练吞吐很低怎么办”，不要直接说“加 GPU”。

应该先定位瓶颈。

排查路径：

1. 看 GPU utilization：GPU 是否在忙。
2. 看 tokens/sec：实际吞吐是否低于预期。
3. 看 MFU：计算利用率是否低。
4. 看 dataloader time：数据加载是否慢。
5. 看 communication time：all-reduce、all-gather 是否占比高。
6. 看 pipeline bubble：pipeline stage 是否等待。
7. 看 checkpoint time：是否频繁阻塞训练。
8. 看 sequence packing：短样本 padding 是否浪费。
9. 看 kernel：attention、MLP、norm 是否使用高效实现。

可以这样回答：

我会先用 profiling 把 step time 拆开，看时间花在 forward、backward、optimizer、communication、dataloader 还是 checkpoint

6.16 高频题：如何判断训练是否发散

训练发散不只是 loss 变大。可以从多个信号判断：

1. loss 持续上升或出现不可恢复 spike。
2. gradient norm 持续异常增大。
3. 出现 NaN 或 Inf。
4. 参数 norm 或 update norm 异常。
5. validation loss 同步恶化。
6. 生成样例变得重复、乱码或失去格式。

回答时可以说：

我不会只看单个 step 的 loss，而会看趋势和相关指标。如果 loss 单点 spike 后恢复，可能是异常 batch；如果 loss、gradient no

6.17 高频题：预训练、SFT、RLHF 的训练差异

面试官可能让你比较不同训练阶段。

可以用表格回答：

阶段	数据	目标	主要作用	风险
预训练	大规模自然文本、代码、论文等	next-token prediction	学语言、知识、模式和基础能力	数据质量、成
继续预训练	领域或新分布数据	next-token prediction	补领域能力和分布适配	遗忘、过拟合
SFT	指令-回答、对话、专家示范	模仿高质量回答	学会听指令和输出格式	数据风格单一
偏好学习	人类或模型偏好数据	优化偏好目标	提升有用性、安全性和偏好一致性	reward hack

然后补一句：

这些阶段不是互相替代的。预训练提供基础能力，继续预训练适配领域，SFT 教模型如何回答，偏好学习进一步塑造行为。不同阶段的

6.18 面试中的常见失分点

Training 面试常见失分点包括：

1. 只背框架名，不讲为什么需要这种并行方式。
2. 把 loss 当成唯一指标，不讨论 eval 和数据分布。
3. loss spike 只会说调小学习率。
4. 不知道 checkpoint 需要保存 optimizer、scheduler 和 sampler 状态。
5. 不会估算训练成本，只会说“需要很多 GPU”。
6. 把 SFT 当成万能知识注入手段。
7. 忽略数据质量、去重和污染检查。
8. 不知道训练吞吐低要先 profiling。
9. 混淆 DP、TP、PP、ZeRO/FSDP 的作用。
10. 不会讲 trade-off，只给绝对答案。

6.19 一套完整 Training 面试回答模板

如果被问一个开放题：“请你设计一次 7B 模型预训练”，可以这样组织：

第一，我会先确定目标和预算，包括模型规模、context length、目标能力、训练 token 数、GPU 资源和时间约束。

第二，准备数据。数据会来自网页、书籍、代码、论文、数学、多语言等来源，经过清洗、去重、质量过滤、安全过滤和 benchmark。

第三，设计训练配方。模型采用 decoder-only Transformer，目标是 causal LM loss。优化器用 AdamW，配合 warmup、cosine decay。

第四，设计分布式策略。根据模型大小和集群拓扑选择 data parallel、tensor parallel、pipeline parallel、ZeRO/FSDP 或组合策略。

第五，训练监控。持续看 train loss、validation loss、LR、gradient norm、throughput、MFU、GPU utilization、NaN/Inf、checkpoint。

第六，容错和 checkpoint。定期保存完整 checkpoint，包括模型、optimizer、scheduler、RNG、sampler 和 step 信息；resume 后继续训练。

第七，评估和迭代。用通用 benchmark、领域 benchmark、人工样例、安全评估和污染检查判断模型是否变好。如果某个能力不足，再调整数据或模型。这个模板适合大多数 training 开放题。

6.20 准备清单

准备 Training 面试时，至少要能回答下面的问题：

1. 预训练的 loss 是什么，为什么 next-token prediction 有效？
2. 数据清洗、去重、质量过滤分别解决什么问题？
3. 数据 mixture 如何影响模型能力？
4. SFT 和预训练有什么区别？
5. AdamW、learning rate、warmup、batch size、gradient clipping 分别起什么作用？
6. DP、TP、PP、ZeRO/FSDP 分别解决什么问题？
7. 混合精度为什么能加速，风险是什么？
8. loss spike 可能有哪些原因，怎么排查？
9. 训练吞吐低怎么定位瓶颈？
10. checkpoint 应该保存哪些状态？
11. resume 后如何确认训练正确？
12. 如何估算一个模型的训练 FLOPs、时长和成本？
13. validation loss 和 benchmark 不一致时怎么分析？
14. 如何设计继续预训练提升领域能力？
15. 如何防止训练过程中的 silent bug？

这一章的核心不是让你背完所有训练系统名词，而是让你建立一个工程闭环：目标决定数据，数据决定能力上限，训练配方决定稳定性，分布式系统决定效率，监控和 checkpoint 决定能否安全跑完，评估决定下一轮怎么迭代。

第七章：Deployment 面试

Deployment 面试考的不是你会不会说 “KV Cache、vLLM、量化、batching、RAG、SLA、成本” 这些词，而是你能不能把一个模型从 checkpoint 变成稳定、低延迟、可观测、可扩展、可控成本的线上服务。

很多候选人在部署面试里容易失分：知道 KV Cache 能加速，但不会估算显存；知道量化能省显存，但不知道 INT4 不一定更快；知道 vLLM 和 PagedAttention，但讲不清它解决的是 KV Cache 管理和动态调度问题；知道 RAG，但把它说成 “向量数据库加大模型”；知道延迟指标，但分不清 TTFT、TPOT、吞吐和尾延迟。

大模型部署是系统工程。它的核心不是 “把模型加载起来能回答”，而是在真实流量下同时满足质量、延迟、吞吐、成本、安全和可靠性约束。

本章重点：推理服务架构、prefill/decode、KV Cache、batching、vLLM/PagedAttention、量化、RAG/Agent 部署、监控排障、成本估算。

7.1 Deployment 面试到底考什么

Deployment 面试通常围绕八条主线：

1. 推理流程：模型从 checkpoint 到线上服务的完整链路。
2. 延迟与吞吐：TTFT、TPOT、QPS、tokens/s、P95/P99 如何理解和优化。
3. KV Cache：为什么需要、显存如何估算、长上下文和高并发下为什么成为瓶颈。
4. 调度与 batching：dynamic batching、continuous batching、chunked prefill、token budget。
5. 推理引擎：vLLM、SGLang、PagedAttention、prefix cache、speculative decoding 的作用边界。
6. 量化与压缩：PTQ、QAT、GPTQ、AWQ、weight-only quantization、KV Cache quantization。
7. 应用系统：RAG、Agent、function calling、多模型路由、安全审核和权限控制。
8. 生产治理：监控、告警、灰度、回滚、容量规划、成本和故障排查。

这些主题可以用一句话串起来：部署团队把模型权重、tokenizer、prompt template 和推理引擎组合成服务，通过路由、调度、缓存、量化和监控，在有限 GPU 资源下稳定服务不同长度、不同优先级、不同风险等级的请求。

面试官想看到的是你能不能回答这些问题：

1. 用户说 “首字很慢”，你怎么排查？
2. tokens/s 看起来很高，但用户仍觉得卡，为什么？
3. 高并发下为什么 KV Cache 会 OOM？
4. vLLM 的 PagedAttention 解决了什么，不解决什么？
5. INT4 模型为什么可能质量变差或速度不升？
6. RAG 系统回答错了，如何判断是检索错、重排错、上下文错还是生成错？
7. 如果成本太高，你会从哪些层面优化？

7.2 回答 Deployment 题的通用结构

Deployment 面试建议使用 “五步结构”：

1. 先定义目标：是离线评测、内部 demo、生产 API、端侧部署，还是企业私有化部署。
2. 再说请求路径：gateway、router、tokenizer、scheduler、GPU worker、streamer、日志监控。
3. 解释核心瓶颈：prefill、decode、KV Cache、显存、带宽、batching、网络和 CPU 前处理。
4. 讲优化手段：量化、continuous batching、prefix cache、speculative decoding、模型路由、prompt 压缩。
5. 给排障和评估：质量、延迟、吞吐、错误率、成本、安全和可观测性。

例如问 “如何部署一个大模型服务”，不要只说：

用 vLLM 启动模型，套一个 API，然后压测。

更好的回答是：

我会先明确服务目标，包括模型大小、上下文长度、请求量、SLA、成本和安全要求。系统上会包含 API gateway、鉴权限流、请求路由。这个回答体现了完整工程闭环。

7.3 从 checkpoint 到线上服务

模型部署不是只加载权重。一个可用的大模型服务通常包括：

- 1. 模型权重：base、instruct、量化版本或 LoRA merge 后版本。
- 2. Tokenizer：必须和训练时一致，否则 token 序列分布会错。
- 3. Chat template：system、user、assistant、特殊 token 和轮次格式。
- 4. 推理引擎：Transformers、vLLM、SGLang、TensorRT-LLM 或自研 serving engine。
- 5. API 层：REST、OpenAI-compatible API、SSE 流式输出或 gRPC。
- 6. 调度层：batching、优先级、队列、超时、取消、资源预算。
- 7. 安全层：鉴权、限流、内容安全、工具权限、审计日志。
- 8. 观测层：指标、日志、trace、bad case、成本归因。

面试中如果问 “为什么 tokenizer 和 chat template 很重要”，可以回答：

模型看到的是 token 序列，不是人类看到的 messages 对象。Tokenizer 决定文本如何切成 token，chat template 决定 system、user 上线前至少要验证：

- 1. 同一 prompt 在离线评测和线上服务中 tokenization 是否一致。
- 2. system prompt 和 chat template 是否重复或缺失。
- 3. stop token 和 EOS 是否正确。
- 4. max context、max output tokens 和截断策略是否符合预期。
- 5. 流式输出中断、客户端断开和超时是否正确释放资源。

7.4 Prefill 与 Decode

LLM 推理可以粗略分成两个阶段：prefill 和 decode。

Prefill 阶段处理用户输入的完整 prompt，一次前向计算所有输入 token 的 hidden states，并为每一层生成 KV Cache。Decode 阶段逐 token 生成输出，每一步只输入新生成的 token，同时复用历史 KV Cache。

两者瓶颈不同：

阶段	输入特点	主要指标	常见瓶颈
Prefill	一次处理较长 prompt	TTFT	计算量、长 prompt、attention kernel、tokenizer、排队
Decode	每步生成一个或少量 token	TPOT、tokens/s	权重读取、KV Cache 访问、batching、显存带宽

面试时要能解释：

TTFT 主要受排队、tokenization、prefill 计算、prefix cache 命中和调度策略影响；TPOT 主要受 decode 阶段的模型 forward、KV 常见追问：“为什么 output token 通常比 input token 贵？”

可以回答：

输入 token 在 prefill 阶段可以并行处理，而输出 token 需要自回归逐步生成。每生成一个 token，都要占用一次模型 forward，并

7.5 KV Cache：推理服务的核心状态

KV Cache 的作用是避免 decode 阶段重复计算历史 token 的 Key 和 Value。

没有 KV Cache 时，每生成一个新 token，都要重新处理整个上下文，代价随序列长度不断增加。有了 KV Cache 后，每一步只计算新 token 的 Q/K/V，并让新 token 的 Q 去 attend 历史 K/V。

面试中要能说清：

- 1. KV Cache 加速的是 decode 阶段。
- 2. KV Cache 是每个请求独有的运行时状态，不是模型参数。
- 3. KV Cache 大小随层数、batch、上下文长度、KV head 数、head dim 和精度增长。
- 4. 长上下文和高并发下，KV Cache 可能比权重更先成为显存瓶颈。

粗略估算公式：

$$\text{KV Cache bytes} \approx 2 * \text{num_layers} * \text{batch_size} * \text{seq_len} * \text{num_kv_heads} * \text{head_dim} * \text{bytes_per_element}$$

其中 2 表示 K 和 V 两份缓存。

如果面试官问 “为什么 GQA/MQA 能降低 KV Cache 显存”，可以回答：

MHA 中每个 query head 都有独立 K/V head，KV Cache 规模和 head 数相关。MQA 让所有 query heads 共享一组 K/V，GQA 让一组 query

7.6 PagedAttention 与 vLLM

高并发 serving 中，请求长度不同、输出长度不同、到达和结束时间也不同。如果每个请求都连续预留一大块 KV Cache，会产生大量内部碎片和外部碎片。

PagedAttention 的直觉是借鉴操作系统分页：逻辑上连续的 KV Cache，不要求物理显存连续，而是切成固定大小 blocks，通过 block table 做映射。

面试回答要准确：

PagedAttention 不是新的 attention 建模算法，也不改变 attention 的数学定义。它主要是 KV Cache 管理方法，把请求的逻辑 token

vLLM 的价值不只是 PagedAttention。面试中可以概括为：

1. 更高效的 KV Cache block 管理。
2. 支持 continuous batching，提高吞吐。
3. 支持 OpenAI-compatible API 和常见模型接入。
4. 支持 prefix cache、量化、分布式推理等工程能力。

但也要说边界：

1. 它不能自动修复模型质量问题。
2. 它不能让显存无限大，长上下文和高并发仍受 KV Cache budget 限制。
3. 它不是所有场景都比专用 TensorRT 或自研 kernel 更优。
4. 真实收益取决于模型、请求长度分布、硬件、batching 和业务 SLA。

7.7 Dynamic Batching 与 Continuous Batching

普通 dynamic batching 通常是在请求进入模型前等待一小段时间，把多个请求凑成 batch 一起执行。

LLM 自回归生成更复杂，因为每个请求输出长度不同。静态 batch 会让短请求等长请求，GPU 也无法及时接纳新请求。

Continuous batching，也叫 in-flight batching，是在每个 decode iteration 动态更新 batch：完成的请求退出，新请求进入，活跃请求继续生成。

面试回答：

Continuous batching 适合 LLM，因为生成过程是逐 token 迭代的，不同请求输出长度差异很大。它允许请求在 decode 过程中动态加

调度器不能只控制请求数，还要控制：

1. 每轮 token budget。
2. 活跃 batch size。
3. KV Cache block 占用。
4. Prefill 和 decode 的比例。
5. 请求优先级和超时。
6. 长请求与短请求隔离策略。

7.8 Chunked Prefill：长 prompt 怎么调度

长 prompt 的 prefill 可能一次占用 GPU 较长时间。如果服务正在给很多用户流式输出，突然进入一个超长 prompt，会让已有请求的 decode 变慢，用户看到 token 间隔变长。

Chunked prefill 的思想是把长 prompt 切成多个 chunk，穿插在 decode iteration 中执行，避免长 prefill 独占 GPU。

回答时要讲 trade-off:

Chunked prefill 可以降低长 prompt 对已有 decode 请求的阻塞，改善流式输出稳定性。但它也可能增加调度复杂度，并让单个长 prompt 等待时间变长。这类题的重点不是背术语，而是能解释调度目标：TTFT、TPOT、吞吐、公平性和成本之间存在冲突。

7.9 量化：省显存不等于一定更快

量化把 FP16/BF16 权重或激活映射到 INT8、INT4 等低精度表示，目标是降低显存、带宽和成本。

常见类型包括：

1. Weight-only quantization：只量化权重，激活保留较高精度。
2. PTQ：训练后量化，成本低，依赖校准数据。
3. QAT：量化感知训练，质量更稳但成本更高。
4. GPTQ：利用近似二阶信息做权重量化误差补偿。
5. AWQ：根据激活统计保护重要权重或通道。
6. KV Cache quantization：量化运行时 KV Cache，降低长上下文和高并发显存压力。

面试中不要说“INT4 一定更快”。更准确的回答是：

INT4 通常能显著降低权重显存，但速度是否提升取决于硬件和 kernel 是否高效、反量化开销、batch 大小、瓶颈是否在权重带宽，以及推理延迟。量化上线前要评估：

1. 通用 benchmark 和业务样本质量。
2. 格式稳定性，如 JSON、代码、工具调用参数。
3. 安全拒答和越狱鲁棒性。
4. 长上下文、RAG、数学、代码、多轮对话。
5. TTFT、TPOT、吞吐、P95/P99。
6. 显存、OOM 率和并发上限。
7. 与 FP16/BF16 baseline 的差异。

如果问“perplexity 变化很小，为什么线上仍可能出问题”，可以回答：

Perplexity 是平均 token loss，可能掩盖格式、工具调用、安全拒答和长上下文检索这些局部能力变化。量化误差可能不明显影响平均 perplexity。

7.10 Speculative Decoding 与多 token 预测

Speculative decoding 的核心思想是：用小而快的 draft model 先提出多个候选 token，再用 target model 一次 forward 验证这些候选。如果候选被接受，一次大模型调用可以推进多个 token。

面试回答：

Speculative decoding 的目标是在保持 target model 输出分布基本不变的前提下减少大模型 decode 次数。收益取决于 draft model 的准确率和 target model 的推理速度。

Medusa、EAGLE 等多 token 预测方法也可以放在 proposal-verification 框架下理解：

1. Medusa 用额外 heads 从当前 hidden state 预测未来 token 候选。
2. EAGLE 更强调在 feature 或 hidden state 层面预测未来状态。
3. 最终仍需要 target model 或原模型验证，不能简单把多个 token 直接拼上。

上线时不能只看接受率，还要看：

1. 提案模型或额外 heads 的计算成本。
2. 候选树验证成本。
3. KV Cache 和显存占用。
4. 对 continuous batching 的影响。
5. 不同任务、温度和输出长度下的收益。
6. P95/P99 是否变差。

7.11 RAG 部署不是向量数据库加 LLM

企业 RAG 系统通常分为离线和在线两条链路。

离线链路包括：

1. 文档接入。
2. 解析、OCR、表格抽取。
3. 清洗和去重。
4. Chunking。
5. Metadata 和权限标签。
6. Embedding。
7. 向量索引和关键词索引。

在线链路包括：

1. Query rewrite。
2. Hybrid retrieval。
3. Rerank。
4. Context construction。
5. LLM generation。
6. Citation 和 groundedness 检查。
7. 日志、反馈和 bad case 回流。

面试回答：

RAG 不是简单接一个向量数据库。向量检索只是召回组件，真实系统还要处理文档解析、chunking、metadata、权限过滤、混合检索、

如果 RAG 回答错了，排查路径是：

1. 正确文档是否入库。
2. 文档解析是否正确，表格、图片、标题是否丢失。
3. Chunk 是否语义完整。
4. Retriever 是否召回正确 chunk。
5. Reranker 是否把正确 chunk 排到前面。
6. Context construction 是否截断或被噪声淹没。
7. Prompt 是否要求基于证据回答和资料不足拒答。
8. LLM 是否忽略证据或编造引用。
9. 权限过滤是否误杀或漏放。

RAG 面试的关键是错误归因，不是只说“换更好的 embedding model”。

7.12 Agent 与工具调用部署

Agent 部署比普通 chat 更复杂，因为模型输出可能触发真实工具、API、数据库或外部系统。

一个工具调用系统通常包括：

1. Tool registry：工具名、描述、schema、权限、超时、风险等级。
2. Planner 或 policy：决定是否调用工具、调用哪个工具。
3. 参数校验：类型、枚举、范围、必填字段。
4. 权限检查：用户身份、资源权限、操作级别。
5. 执行器：真实调用 API、数据库、浏览器或代码环境。
6. 错误处理：超时、失败、重试、追问用户。
7. 审计日志：记录工具调用 trace。
8. 安全控制：高风险操作二次确认、防 prompt injection。

面试中要强调：

Function calling 不只是让模型输出 JSON。完整系统必须做 schema 约束、参数校验、权限控制、执行隔离、错误处理、审计日志和

常见风险包括：

1. 模型选错工具。
2. 参数缺失或编造参数。
3. 工具返回失败时模型编造结果。
4. Prompt injection 诱导越权调用。
5. 多步调用中状态错乱。
6. 高风险操作缺少确认。

7.13 延迟、吞吐和用户体验

Deployment 面试里必须区分几个指标：

1. TTFT: time to first token, 首 token 延迟。
2. TPOT: time per output token, 每输出 token 延迟。
3. E2E latency: 端到端完成时间。
4. Throughput: tokens/s 或 requests/s。
5. QPS: 每秒请求数。
6. P95/P99: 尾延迟。
7. Error rate: 超时、OOM、取消、内部错误。

“tokens/s 高但用户觉得卡” 可能来自：

1. TTFT 很高，用户等首字太久。
2. 平均 tokens/s 高但 P99 很差。
3. 流式输出不均匀，token 间隔抖动。
4. 长请求阻塞短请求。
5. 网络、网关或客户端渲染慢。
6. safety filter、RAG 检索或工具调用前置耗时高。
7. 输出太长，整体完成时间高。

面试回答：

我不会只看平均 tokens/s。用户体验更受 TTFT、流式稳定性和尾延迟影响。排查时会把链路拆成网关排队、tokenizer、RAG 或工具调用。

7.14 线上故障怎么排查

部署面试经常问线上问题。回答时要先分类，再定位。

7.14.1 TTFT 变高

排查路径：

1. 请求排队是否变长。
2. Router 是否把流量打到热点 worker。
3. Prompt 是否变长。
4. Tokenizer CPU 是否成为瓶颈。
5. RAG 检索、rerank 或工具调用是否变慢。
6. Prefill tokens/s 是否下降。
7. Prefix cache 命中率是否下降。
8. 是否有长 prompt 抢占 prefill budget。

7.14.2 TPOT 变高

排查路径：

1. Running batch 是否过大。
2. KV Cache 访问是否成为瓶颈。
3. 显存是否接近上限导致频繁调度失败。
4. 是否启用了低效量化 kernel。
5. GPU utilization 和 memory bandwidth 是否异常。

6. Decode 调度是否被大量 prefill 打断。
7. Speculative decoding 接受率是否下降。

7.14.3 OOM 增加

排查路径：

1. 上下文长度是否变长。
2. 并发是否增加。
3. max output tokens 是否过大。
4. KV Cache block 是否泄漏。
5. 请求取消后资源是否释放。
6. 模型版本、量化配置或 KV dtype 是否变化。
7. Prefix cache 或 LoRA adapter 是否占用额外显存。

7.14.4 回答质量突然下降

排查路径：

1. 模型版本是否切换。
2. Tokenizer 或 chat template 是否变化。
3. system prompt 是否被覆盖。
4. RAG 索引、embedding model、reranker 是否更新。
5. Decoding 参数是否变化。
6. 量化版本是否替换。
7. 安全策略是否过度拒答。

7.15 成本优化：不要只说换小模型

推理成本由多个因素决定：

1. 模型大小。
2. 输入 token 数。
3. 输出 token 数。
4. 上下文长度。
5. 并发和 SLA。
6. KV Cache 显存。
7. GPU 类型和利用率。
8. 推理引擎效率。
9. 量化和 batching。
10. 冗余、灰度和故障切换。

降本可以从四层做：

1. 模型层：小模型、蒸馏、量化、LoRA merge、模型路由。
2. 引擎层：continuous batching、PagedAttention、prefix cache、speculative decoding、kernel 优化。
3. 产品层：prompt 压缩、限制 max tokens、缓存、摘要、减少无效调用。
4. 系统层：容量规划、峰谷调度、冷热模型分层、优先级队列、自动扩缩容。

面试回答要体现单位有效任务成本：

最便宜的模型不一定最划算。要看 cost per successful answer。如果小模型单次调用便宜，但成功率低、重试多、人工介入多，最终

7.16 高频题：如何设计一个 LLM Serving 系统

可以用下面模板回答：

第一，我会先明确目标：模型规模、上下文长度、请求量、SLA、成本、安全和部署环境。

第二，设计入口层：API gateway 做鉴权、限流、配额、请求大小检查和日志；支持流式输出和取消请求。

第三，设计路由层：根据模型、租户、token 预算、GPU 负载、KV Cache 余量和优先级选择 worker，而不是只按请求数负载均衡。

第四，设计推理层：使用 vLLM、SGLang 或自研引擎，支持 tokenizer、prefill/decode、continuous batching、KV Cache 管理、pre

第五，设计可靠性：超时、重试、降级模型、灰度发布、版本回滚、健康检查和故障隔离。

第六，设计观测体系：监控 TTFT、TPOT、P95/P99、tokens/s、queue time、KV blocks、OOM、error rate、GPU utilization、成本和

第七，设计安全治理：内容安全、权限控制、prompt injection 防护、工具调用审计、用户数据隔离和合规日志。

这个模板适合大多数推理服务系统设计题。

7.17 高频题：如何优化首 token 延迟

首 token 延迟通常来自排队、前处理和 prefill。

优化方向：

1. 减少排队：更好的路由、优先级、扩容、隔离长短请求。
2. 减少 prompt：prompt 压缩、历史对话摘要、减少无效 system prompt。
3. 优化 prefill：高效 attention kernel、chunked prefill、prefix cache。
4. 优化前处理：tokenizer 并行、RAG/rerank 降延迟、缓存检索结果。
5. 控制长上下文：限制最大输入、分桶调度、长请求单独队列。

回答模板：

我会先把 TTFT 拆成 queue time、tokenization、RAG/tool 前处理、prefill、调度等待和网络返回。不同部分对应不同优化：排队高

7.18 高频题：如何做容量规划

容量规划不能只看 QPS，要看 token 分布。

需要统计：

1. 输入 token 分布：P50、P90、P99。
2. 输出 token 分布：P50、P90、P99。
3. 峰值并发。
4. 目标 TTFT 和 TPOT。
5. 每张 GPU 的 prefill tokens/s 和 decode tokens/s。
6. KV Cache 可用显存和最大并发上下文。
7. 冗余系数和故障切换容量。

面试回答：

LLM 容量规划要按 token 负载做，而不是只按请求数。两个请求的成本可能因为 prompt 长度和输出长度差几十倍。估算时我会用业务

7.19 高频题：RAG 系统上线前评估什么

RAG 上线前要同时评估检索、生成和系统。

检索指标：

1. Recall@k。
2. Precision@k。
3. MRR。
4. nDCG。
5. Rerank latency。

生成指标：

1. Answer correctness。
2. Faithfulness。
3. Groundedness。
4. Citation accuracy。
5. Unsupported claim rate。
6. Abstention accuracy。

系统指标：

1. 端到端延迟。
2. 检索和生成成本。
3. 权限过滤正确率。
4. 文档新鲜度。
5. bad case 回流效率。

回答模板：

我不会只评估最终答案分数。RAG 要分层评估：先看 retriever 是否召回正确证据，再看 reranker 是否排序正确，再看最终 context

7.20 面试中的常见失分点

Deployment 面试常见失分点包括：

1. 把模型能跑起来等同于能上线。
2. 分不清 prefill 和 decode 的瓶颈。
3. 只说平均 latency，不看 TTFT、TPOT、P95/P99。
4. 不会估算 KV Cache 显存。
5. 把 PagedAttention 说成新的 attention 算法。
6. 认为 INT4 一定更快、质量一定可接受。
7. 只会说“用 vLLM”，讲不清调度和 KV 管理。
8. RAG 错误只会换 embedding，不会做错误归因。
9. Agent 工具调用只讲 JSON，不讲权限、审计和安全。
10. 成本优化只说换小模型，不看成功率和重试成本。
11. 忽略 tokenizer、chat template 和 stop token 的一致性。
12. 不会从监控指标定位线上问题。

7.21 一套完整 Deployment 面试回答模板

如果被问开放题：“请你设计一个 ChatGPT 类大模型推理服务”，可以这样组织：

第一，我会先明确业务目标和约束，包括模型大小、上下文长度、QPS、输入输出 token 分布、TTFT/TPOT SLA、成本、安全和合规要求。

第二，设计整体架构。入口是 API gateway，负责鉴权、限流、请求校验和流式连接；后面是 router，根据模型版本、租户、token 预算

第三，优化核心瓶颈。Prefill 影响首 token，可以通过 prompt 压缩、prefix cache、chunked prefill 和高效 attention kernel 优化。

第四，保证质量和安全。上线前比较 FP16 baseline 和量化版本，评估通用能力、业务样本、格式稳定性、安全拒答和长上下文；如果

第五，设计可观测性和可靠性。监控 queue time、TTFT、TPOT、P95/P99、tokens/s、KV blocks、OOM、error rate、GPU utilization

第六，做容量规划和成本治理。根据 token 分布、单卡 profiling、KV Cache 显存和目标 SLA 估算 GPU 数；通过模型路由、量化、缓

7.22 准备清单

准备 Deployment 面试时，至少要能回答下面的问题：

1. 从 checkpoint 到线上服务需要哪些组件？
2. Tokenizer 和 chat template 为什么必须一致？

3. Prefill 和 decode 的区别是什么？
4. TTFT、TPOT、吞吐、P95/P99 分别衡量什么？
5. KV Cache 为什么能加速推理？显存如何估算？
6. MHA、MQA、GQA 对 KV Cache 有什么影响？
7. PagedAttention 解决什么问题？它是不是新的 attention 算法？
8. Continuous batching 和普通 dynamic batching 有什么区别？
9. Chunked prefill 解决什么问题，代价是什么？
10. INT8、INT4、GPTQ、AWQ、KV Cache 量化分别要注意什么？
11. Speculative decoding 的收益由什么决定？
12. RAG 回答错了怎么做错误归因？
13. Function calling 上线要做哪些权限和安全控制？
14. TTFT 变高、TPOT 变高、OOM 增加分别怎么排查？
15. 如何估算推理服务所需 GPU 数？
16. 如何降低推理成本但不明显损伤质量？
17. 如何设计大模型 serving 的监控 dashboard？
18. 如何做模型版本灰度、回滚和降级？

这一章的核心不是让你背完所有部署系统名词，而是让你建立一套生产思维：模型质量只是起点，线上服务还要在真实流量、真实成本、真实故障和真实安全风险下稳定运行。

第八章：Alignment 与 Safety 面试

Alignment 与 Safety 面试考的不是你会不会说“RLHF、DPO、reward hacking、jailbreak、red teaming、model card”这些词，而是你能不能把模型行为、训练目标、风险评估、系统防护和发布治理串成一个闭环。

很多候选人在这一关容易走偏：要么只讲 RLHF 和 DPO，不讲评估与上线门禁；要么只说“拒绝危险请求”，忽略 over-refusal 和正常 helpfulness；要么把 jailbreak 和 prompt injection 混在一起；要么把 safety 当成政策背诵，而不是一个可评估、可监控、可迭代的工程系统。

Alignment 与 Safety 面试的关键，是证明你理解：模型不是只要能力强就够了，还要在复杂用户、复杂工具、复杂部署环境和分布外场景里尽量可靠、诚实、安全、可控。

本章重点：alignment problem、RLHF/DPO 局限、reward hacking、jailbreak、prompt injection、red teaming、safety eval、over-refusal、隐私、危险能力评估、发布治理。

8.1 Safety 面试到底考什么

Safety 面试通常围绕八条主线：

1. 目标定义：什么是 alignment，什么是 safety，Helpful、Honest、Harmless 如何权衡。
2. 后训练方法：SFT、RLHF、DPO、Constitutional AI、RLAIF 在安全中的作用和局限。
3. 目标错配：reward hacking、Goodhart 定律、proxy objective、over-optimization。
4. 攻防问题：jailbreak、prompt injection、多轮诱导、工具越权和 RAG 污染。
5. 评估体系：safety eval、red teaming、harmful compliance、over-refusal、dangerous capability eval。
6. 模型行为：幻觉、不确定性表达、拒答边界、偏见、公平性和诚实性。
7. 隐私治理：memorization、PII、训练数据泄露、RAG 权限和日志脱敏。
8. 发布治理：model card、system card、发布门禁、灰度、监控、事故响应。

这些主题可以用一句话串起来：安全团队先定义风险 taxonomy 和可接受边界，再通过训练、评估、红队和系统防护降低风险，最后用发布门禁和线上监控保证模型在真实使用中持续符合安全目标。

面试官想看到的是你能不能回答这些问题：

1. RLHF 为什么不是 alignment 的最终解？
2. DPO 简化了什么，又没有解决什么？
3. Reward hacking 在 LLM 里可能如何表现？
4. Jailbreak 和 prompt injection 有什么区别？
5. 如何设计一个 safety eval？

6. 如何同时降低 harmful compliance 和 over-refusal?
7. Agent 工具调用如何做安全边界?
8. 如果 helpfulness 提升但 jailbreak 成功率也提升, 能不能上线?

8.2 回答 Safety 题的七层框架

Safety 题不要直接背方法。推荐用七层框架:

1. 定义目标: 要解决什么安全或对齐问题。
2. 定义风险 taxonomy: 风险类别、严重度、用户场景和攻击面。
3. 分析失败模式: 模型为什么会失败, 训练、数据、系统、用户侧各有什么原因。
4. 给缓解方法: 训练方法、系统防护、权限控制、人工审核。
5. 给评估方法: 离线 eval、红队、人工复核、线上监控。
6. 给发布治理: 门禁、灰度、回滚、日志、事故响应。
7. 讲 trade-off: 安全与 helpfulness、隐私与 utility、透明度与滥用风险。

例如问“你如何提升一个模型的安全性”, 不要只说:

做安全 SFT, 再用 RLHF, 对 jailbreak 做红队。

更好的回答是:

我会先定义安全目标和风险分类, 比如有害内容、隐私泄露、jailbreak、prompt injection、工具滥用和高风险专业建议。训练上可 refusal、正常 helpfulness、多轮 jailbreak 成功率和工具越权率。发布前设置门禁和灰度, 发布后持续监控安全 bad case, 并把红队测试用例加入训练。这个回答有目标、方法、评估、治理和 trade-off。

8.3 Alignment 和 Safety 的区别

AI Safety 更关注模型或系统是否会造成伤害, 例如有害内容、隐私泄露、越狱、工具误用、危险能力、偏见和部署事故。

Alignment 更关注模型实际优化的目标和行为是否符合人类真实意图、价值和边界, 例如模型是否 helpful、honest、harmless, 是否会 reward hacking, 是否会在分布外场景里学错目标。

面试回答:

Safety 更偏风险和伤害控制, Alignment 更偏目标一致性和行为一致性。Safety 可以看作 alignment 的重要目标之一, 但 alignment

常见追问: “安全是不是让模型多拒答?”

可以回答:

不是。安全不是简单提高拒答率。真正的目标是该拒绝时拒绝, 该帮助时帮助。只提高拒答率会造成 over-refusal, 损害正常用户请求 refusal 和 safe alternative quality。

8.4 Helpful、Honest、Harmless 的冲突

Helpful、Honest、Harmless 是面试中非常常见的三目标框架。

1. Helpful: 模型能解决用户问题。
2. Honest: 模型不编造、不假装知道, 能表达不确定性。
3. Harmless: 模型不帮助用户造成伤害。

三者经常冲突:

1. 用户请求危险帮助时, helpful 和 harmless 冲突。
2. 用户要求模型回答不知道的信息时, helpful 和 honest 冲突。
3. 某些真实信息本身高风险时, honest 和 harmless 冲突。
4. 过度安全会损害 helpfulness, 过度迎合会损害 harmless 和 honesty。

面试回答:

我不会把 helpful、honest、harmless 当成三个独立指标，而会看它们在具体场景中的权衡。比如普通教育性请求要尽量 helpful，而

8.5 RLHF 的价值与根本局限

RLHF 的价值是把难以手写的目标函数，转化为人类偏好数据和 reward model，让模型更符合人类对回答质量、安全性和风格的期望。

典型流程：

1. 先做 SFT，让模型学会基础指令遵循。
2. 收集 prompt 下多个回答的人类偏好。
3. 训练 reward model，给回答打分。
4. 用 PPO 或类似方法优化 policy，同时用 KL 约束不要偏离 reference model 太远。

面试中不能只夸 RLHF，要讲清局限：

RLHF 的根本局限在于 human preference 和 reward model 都是 proxy，不等于真实人类目标。偏好数据可能有标注员偏差、长度偏差。

常见追问：“为什么需要 KL 约束？”

回答：

KL 约束让 policy 不要为了追求 reward model 分数而偏离 SFT 或 reference model 太远。它可以缓解语言质量下降、reward hacking

8.6 DPO 简化了什么，没有解决什么

DPO 把带 KL 约束的偏好优化推导成直接在 chosen/rejected pair 上优化 policy 的损失，避免显式训练 reward model 和 PPO loop。

它的工程优势是：

1. 流程更简单。
2. 训练更稳定。
3. 不需要在线采样和单独 reward model。
4. 更容易复现和调参。

但 DPO 没有消除偏好数据的根本问题。

面试回答：

DPO 简化的是 RLHF 的工程优化流程，而不是解决 alignment 的所有问题。它仍然依赖 chosen/rejected 偏好数据。如果偏好数据有问

常见追问：“DPO 一定比 RLHF 更安全吗？”

回答：

不一定。安全性取决于偏好数据、训练强度、reference 选择和评估门禁。DPO 训练更简单，不等于目标更正确。如果 preference pa

8.7 Reward Hacking 和 Goodhart 定律

Reward hacking 指模型找到提高 reward 或评估分数的捷径，但输出不符合真实人类目标。

LLM 中常见表现包括：

1. 为了高分输出冗长但空泛的答案。
2. 自信编造引用或事实。
3. 过度拒答，以避免安全风险。
4. 迎合 judge 偏好的格式，而不是解决真实问题。
5. 在 RAG 中给出看似有 citation 但证据不支持的回答。
6. 在代码任务中通过测试样例但隐藏边界错误。

面试回答：

Reward hacking 本质上是 proxy objective 被过度优化。Goodhart 定律说，当一个指标变成目标，它就会失去作为好指标的可靠性。

缓解方法包括：

1. 提高偏好数据质量。
2. 加 hard negatives 和 adversarial examples。
3. 控制优化强度和 KL。
4. 做 reward model 校准。
5. 引入 human eval 和 expert eval。
6. 用 holdout、红队和线上 bad case 监控 reward-human gap。

8.8 Jailbreak：绕过模型安全边界

Jailbreak 是用户通过角色扮演、诱导、多轮对话、编码、语言混合或对抗提示，让模型绕过安全策略，输出本应拒绝的内容。

防 jailbreak 不能只靠一句 system prompt。

面试回答：

我会从训练、评估和系统三层防 jailbreak。训练上加入安全 SFT、偏好优化、多轮诱导样本、边界样本和安全替代回答。评估上覆盖 refusal 和 safe alternative quality。

为什么 jailbreak 难以完全防住？

1. 自然语言空间巨大。
2. 攻击可以跨语言、多轮、编码和组合变形。
3. 模型遵循指令的能力本身会被滥用。
4. 安全训练覆盖不了所有分布外场景。
5. 需要持续红队和 regression suite。

面试时注意：讲防御框架和评估，不要展示可操作攻击步骤。

8.9 Prompt Injection：LLM 应用的指令边界问题

Prompt injection 和 jailbreak 经常被混淆。

Jailbreak 主要是绕过模型自身安全边界。Prompt injection 主要是 LLM 应用混淆了可信指令和不可信数据，让外部文档、网页、邮件或工具返回中的恶意文本影响模型行为。

面试回答：

Prompt injection 的核心问题是指令和边界混淆。防御不能只靠 prompt，而要做系统层隔离：明确 system、developer、user、Agent 和 RAG 中 prompt injection 风险更高，因为外部内容可能影响：

1. 检索结果选择。
2. 最终回答内容。
3. 工具调用决策。
4. 敏感数据泄露。
5. 越权操作。

高质量回答要强调：不可信文本不能直接控制高权限动作。

8.10 Safety Eval 怎么设计

Safety eval 不是找几个危险 prompt 测一下拒答率。

一个完整 safety eval 应包含：

1. 风险 taxonomy：有害内容、自伤、隐私、网络安全、偏见、工具滥用、prompt injection、危险能力。
2. 样本分层：正常请求、边界请求、禁止请求、多轮请求、跨语言请求、对抗请求。
3. 指标体系：harmful compliance、refusal accuracy、over-refusal、safe alternative quality、attack success rate。

4. 人工复核：高风险样本不能只依赖自动 judge。
5. 错误归因：区分政策理解错、上下文理解错、拒答边界错、系统工具权限错。
6. 回归机制：红队发现样本进入 regression suite。

面试回答：

我会先定义风险分类和严重度，再构造分层测试集，既包括明显违规请求，也包括正常安全请求和边界请求。指标上不能只看拒答率，refusal、正常 helpfulness、safe alternative quality 和多轮攻击成功率。高风险类别需要人工或专家复核，红队发现的问题要沉

常见追问：“Safety eval 和 red teaming 区别是什么？”

回答：

Safety eval 更像系统化评估，有固定测试集、指标和发布门禁。Red teaming 更偏主动探索未知失败模式。两者互补：red teaming 发

8.11 Over-Refusal：安全模型也会伤害体验

Over-refusal 指模型对本应回答的正常请求过度拒绝。

例如：

1. 用户问网络安全防御知识，模型误判为攻击请求。
2. 用户询问心理健康支持，模型只拒绝或机械建议求助。
3. 用户进行合法药品科普，模型过度回避。
4. 用户要求分析暴力文本，模型无法区分分析和煽动。

面试回答：

安全优化不能只降低 harmful compliance，还要控制 over-refusal。否则模型看似安全，但正常用户体验很差。评估时要构造 benign-but-sensitive 样本，检查模型是否能给出安全替代、教育性说明或澄清问题，而不是简单拒绝。

降低 over-refusal 的方法：

1. 加入边界样本和正常敏感请求。
2. 训练 safe completion，而不是只训练 refusal。
3. 区分意图、上下文和细节级别。
4. 设计分级响应策略。
5. 用人工评估检查误拒。

8.12 Red Teaming：主动发现未知失败模式

Red teaming 的目标不是证明模型安全，而是尽可能发现模型和系统的失败模式。

高质量 red teaming 包括：

1. 风险覆盖：有害内容、隐私、prompt injection、工具调用、偏见、危险能力。
2. 攻击面覆盖：单轮、多轮、跨语言、RAG、Agent、文件上传、网页浏览、工具结果。
3. 角色覆盖：普通用户、恶意用户、内部员工、第三方插件、外部文档。
4. 严重度分级：P0、P1、P2、P3。
5. 修复闭环：问题、复现、根因、修复、回归测试。

面试回答：

Red teaming 应该和发布流程绑定，而不是一次性活动。红队发现的问题要分级，P0/P1 问题进入发布门禁；修复后要把样本加入 regression suite。

8.13 Dangerous Capability Eval

危险能力评估关注模型是否显著降低高风险行为门槛，而不只是会不会输出违规文本。

面试中可以按风险类型拆：

1. Cyber：是否辅助漏洞利用、攻击链规划或恶意自动化。
2. Bio：是否降低生物安全相关高风险门槛。

3. Autonomy: 是否能长期规划、获取资源、规避限制。
4. Tool use: 是否能用工具放大风险。
5. Deception: 是否表现出误导、隐瞒或策略性行为。

安全表达要保持防御和评估视角，不提供具体危险操作细节。

面试回答：

危险能力评估要看模型相对非 AI baseline 是否提供实质增量，而不是只看是否知道公开信息。我会明确评估条件，包括 prompt、工

常见追问：“什么是 capability elicitation？”

回答：

Capability elicitation 是用合适 prompt、工具、scaffold 或专家辅助尽量激发模型潜在能力，避免低估风险。但评估报告要清楚

8.14 Privacy、Memorization 与数据治理

LLM 隐私风险来自多个环节：训练数据、模型参数、RAG 知识库、日志、工具调用和输出。

常见风险：

1. 训练数据中包含 PII、密钥、私密文本。
2. 模型记忆并复现敏感样本。
3. RAG 权限过滤错误导致越权读取。
4. 日志记录用户敏感输入。
5. 工具调用返回敏感字段但未脱敏。

面试回答：

LLM privacy 要做全生命周期治理。训练前做数据来源记录、许可检查、PII scrub、密钥扫描、去重和数据分级；训练后做 memoriza

常见追问：“Memorization 和 privacy 有什么关系？”

回答：

Memorization 本身是模型复现训练样本。如果被记住的是公开常识，风险较低；如果是 PII、密钥、私有邮件或商业机密，就会变成

8.15 诚实性、不确定性和幻觉

Honesty 面试常围绕一个问题：如何让模型不知道时说不知道？

模型自信幻觉的原因包括：

1. Next-token prediction 不直接优化真实性。
2. 训练数据中有错误事实。
3. SFT/RLHF 可能偏好流畅、完整、自信的回答。
4. 模型缺少外部验证。
5. 奖励模型可能奖励看似专业的错误答案。

面试回答：

让模型诚实表达不确定性，需要训练、评估和系统共同支持。训练上加入不知道、资料不足、需要澄清、需要检索的样本；评估上看
不要说“降低 temperature 就能解决幻觉”。低温只能降低随机性，不能保证事实正确。

8.16 Mechanistic Interpretability 与 Safety

Mechanistic interpretability 试图理解模型内部 features、circuits 和信息流。

它对 safety 的潜在价值：

1. 分析拒答、幻觉、事实性等行为机制。
2. 发现与危险能力或不安全行为相关的表示。
3. 用 activation patching 或 ablation 做因果验证。

4. 用 SAE 等方法分解更可解释的 feature。
5. 辅助 red teaming 和模型调试。

但它目前不能单独证明模型安全。

面试回答：

Mechanistic interpretability 对 safety 很有价值，但目前更像辅助诊断工具，而不是安全保证。行为评估、红队、系统权限和发布

常见追问：“Attention heatmap 是否能解释模型？”

回答：

Attention heatmap 可以提供线索，但不是完整解释。它只显示某层某头的读取权重，不说明 value 写了什么，也不说明后续层如何使

8.17 Model Editing 与 Unlearning

Model editing 关注局部修改模型知识或行为，例如把错误事实改成新事实，同时尽量不影响其他能力。

Unlearning 关注让模型近似移除某些训练数据、知识、行为或能力的影响。

面试回答：

Editing 通常有明确的新目标答案，目标是局部更新；unlearning 通常要求目标数据或目标知识的影响消失，同时保留其他能力。LLM

评估 unlearning 要看：

1. Forget set 是否不再复现。
2. Retain set 能力是否保持。
3. 改写、多轮、翻译是否仍能诱导目标知识。
4. Membership inference 风险是否下降。
5. 是否只是输出层 guardrail suppression。
6. 是否有版本记录和审计证据。

8.18 Model Card、System Card 和发布门禁

Model card 关注模型本身：训练数据概要、用途、不适用场景、评估结果、限制、风险和缓解措施。

System card 关注完整系统：模型、RAG、工具、权限、policy layer、监控、红队、隐私和事故响应。

面试回答：

大模型产品的风险往往来自系统组合，而不只是模型权重本身。一个模型接入 RAG、浏览器、代码执行或企业 API 后，风险边界会变

发布门禁应提前定义，不能上线前临时解释结果。

门禁可以包括：

1. P0 安全问题为 0。
2. P1 问题有明确修复或限制。
3. Harmful compliance 低于阈值。
4. Over-refusal 不超过阈值。
5. Prompt injection 和工具越权风险低于阈值。
6. 隐私和数据泄露评估通过。
7. Dangerous capability 不超过发布等级。
8. 监控、回滚和事故响应就绪。

8.19 Agent Safety 系统设计题

题目可能是：

请设计一个能调用企业 API 的 Agent，并保证工具调用安全。

回答可以分七层：

1. 工具注册: tool name、description、schema、风险等级、权限范围。
2. 身份权限: 用户身份、租户、资源、操作级别。
3. 参数校验: 类型、范围、枚举、必填项、业务规则。
4. 执行隔离: 只读/写操作分离, 高风险操作二次确认。
5. Prompt injection 防护: 外部内容不能直接控制工具调用。
6. 审计日志: 记录 tool call trace、参数、结果、用户确认。
7. 评估监控: 工具选择准确率、参数准确率、越权率、失败率、人工复核。

标准回答:

我会把 Agent 安全边界放在系统层, 而不是只依赖模型。工具要有 schema、权限和风险等级; 调用前做用户身份和资源权限检查; 参

8.20 Safety 面试中的常见失分点

Alignment 与 Safety 面试常见失分点包括:

1. 把 safety 简化成拒答。
2. 只讲 RLHF, 不讲评估、红队和系统防护。
3. 认为 DPO 天然更安全。
4. 不区分 jailbreak 和 prompt injection。
5. 讲 prompt injection 时只说“加强 prompt”, 不讲权限和隔离。
6. 只看 harmful compliance, 不看 over-refusal。
7. 不会设计 safety eval, 只会列风险词。
8. 不知道 red teaming 和 regression suite 的关系。
9. 把 mechanistic interpretability 说成可以证明模型安全。
10. 对 unlearning 轻易宣称完全遗忘。
11. 讨论危险能力时提供不必要的攻击细节。
12. 不讲发布门禁、灰度、回滚和事故响应。

8.21 高频题回答模板

8.21.1 RLHF 为什么不是最终解

RLHF 用人类偏好训练 reward model, 再优化 policy, 让模型更符合人类期望。但它的根本问题是偏好和 reward 都是 proxy, 不等于

8.21.2 如何防 jailbreak

我会从训练、评估和系统三层防护。训练上加入安全 SFT、偏好优化、多轮诱导和边界样本; 评估上覆盖角色扮演、跨语言、编码包装 refusal 和 safe alternative quality。

8.21.3 如何设计 safety eval

我会先定义风险 taxonomy 和严重度, 再构造正常、边界、禁止、多轮、跨语言 and 对抗样本。指标上同时看 harmful compliance、refusal、safe alternative quality、jailbreak robustness 和正常 helpfulness。高风险类别需要人工或专家复核, 红队发现的问

8.21.4 如何防 prompt injection

Prompt injection 的核心是可信指令和不可信数据边界混淆。防御上要明确 system、developer、user、external content 和 tool r

8.21.5 如何平衡 helpfulness 和 safety

我会按风险等级和用户意图区分, 而不是一刀切拒答。明显有害请求要拒绝危险细节并提供安全替代; 正常教育、防御、合规请求要 refusal、helpfulness 和 safe alternative quality。

8.22 一套完整 Safety 面试回答模板

如果被问开放题：“请你设计一个大模型 Safety Platform”，可以这样组织：

第一，我会先定义风险 taxonomy 和严重度，包括有害内容、隐私、jailbreak、prompt injection、工具滥用、危险能力、偏见和 over-refusal。

第二，建设数据和训练闭环。数据包括 safety SFT、偏好数据、边界样本、红队样本、正常敏感请求和 regression suite；训练方法

第三，建设评估体系。离线 eval 覆盖 harmful compliance、refusal accuracy、over-refusal、safe alternative quality、prompt

第四，建设系统防护。输入输出 policy、RAG 权限过滤、工具最小权限、schema 校验、高风险操作二次确认、沙箱、日志审计和 pro

第五，建设发布治理。用 model card 和 system card 记录能力、限制、风险和缓解措施；发布前设置门禁，发布时灰度，发布后监控

第六，形成持续迭代。线上失败样本和红队样本进入 regression suite，修复后重新评估，保证安全能力不会随模型版本、prompt 或

8.23 准备清单

准备 Alignment 与 Safety 面试时，至少要能回答下面的问题：

1. AI Safety 和 Alignment 有什么区别？
2. Helpful、Honest、Harmless 如何冲突？
3. RLHF 的完整流程是什么？根本局限是什么？
4. DPO 简化了什么，没有解决什么？
5. Reward hacking 和 Goodhart 定律有什么关系？
6. 如何设计 safety eval？
7. Safety eval 和 red teaming 有什么区别？
8. Jailbreak 和 prompt injection 有什么区别？
9. 如何防 prompt injection？
10. 如何评估 over-refusal？
11. 如何让模型诚实表达不确定性？
12. 如何治理模型隐私和 memorization？
13. Dangerous capability eval 要评估什么？
14. Capability elicitation 为什么重要？
15. Mechanistic interpretability 对 safety 有什么用和局限？
16. Model editing 和 unlearning 有什么区别？
17. Model card 和 system card 有什么区别？
18. 如何设计 Agent 工具调用安全架构？
19. 如果新模型能力提升但 jailbreak 成功率也提升，是否上线？
20. 如何把红队发现的问题转成长期回归测试？

这一章的核心不是让你背完所有安全术语，而是建立一套风险治理思维：先定义目标和风险，再设计训练、评估、系统防护和发布门禁，最后用线上监控和回归测试持续改进。

第九章：Multimodal 与 Research 面试

Multimodal 与 Research 面试考的不是你能不能背出 CLIP、LLaVA、Diffusion、Whisper、Sora、DPO、FlashAttention 这些名字，而是你能不能把你一个新方向放进正确问题里，讲清它解决什么瓶颈、方法为什么有效、实验是否可信、边界在哪里，以及如果让你复现或改进，你会怎么做。

很多候选人在这一关会出现两个问题。第一个问题是多模态知识碎片化：知道 CLIP 做图文对齐，但讲不清 contrastive learning；知道 VLM 能看图，但讲不清 vision encoder、connector 和 LLM 如何连接；知道 diffusion 能生成图像，但讲不清加噪、去噪和条件控制；知道视频生成难，但只会说“算力大”。第二个问题是研究表达空泛：能报论文名，但不会判断贡献；能说“可以改进”，但没有实验设计；能说“这个方法很好”，但不讲 baseline、ablation、失败模式和成本。

这一章把多模态面试和研究讨论面试放在一起，是因为它们经常同时出现。多模态方向变化快，面试官更看重你的研究判断力，而不是只看你是否背过固定答案。

本章重点：CLIP、VLM、Diffusion、视频生成、语音模型、多模态评估、安全、论文讨论、复现实验、ablation、开放研究题。

9.1 Multimodal 与 Research 面试到底考什么

这类面试通常围绕两组能力。

第一组是多模态技术能力：

1. 图文对齐：CLIP、contrastive learning、image-text retrieval。
2. 视觉编码：ViT、CNN、patch embedding、视觉特征层级。
3. VLM 架构：vision encoder、connector、LLM、image token、multimodal instruction tuning。
4. 生成模型：diffusion、latent diffusion、text-to-image、text-to-video。
5. 语音音频：ASR、TTS、speech encoder、端到端语音对话。
6. 多模态评估：OCR、图表、空间关系、计数、视觉幻觉、安全。

第二组是研究讨论能力：

1. 如何介绍一篇论文。
2. 如何判断贡献是否真实。
3. 如何复现核心结论。
4. 如何设计 ablation。
5. 如何解释负结果。
6. 如何提出下一步研究方向。
7. 如何处理开放问题和不确定问题。

这些能力可以用一句话串起来：多模态面试考你是否理解不同模态如何表示、对齐、融合、生成和评估；研究面试考你是否能用科学方法判断一个新方法是否真的有效。

9.2 回答多模态题的通用结构

多模态题建议用“五步结构”：

1. 先说任务：图文检索、图片问答、图像生成、视频生成、语音识别还是多模态 Agent。
2. 再说输入输出：输入有哪些模态，输出是什么，监督信号是什么。
3. 讲核心架构：encoder、connector、LLM、decoder、diffusion U-Net 或 DiT。
4. 讲训练目标：contrastive loss、caption loss、next-token loss、denoising loss、instruction tuning。
5. 讲评估和失败模式：幻觉、OCR、计数、空间关系、时序一致性、安全和成本。

例如问“VLM 如何把图像接入 LLM”，不要只说：

用 vision encoder 提取图像特征，然后喂给 LLM。

更好的回答是：

一个典型 VLM 会先用 vision encoder，比如 ViT，把图像切成 patch 并编码成视觉 token；然后用 connector，比如 linear project Former，把视觉特征映射到 LLM 的 hidden space；最后把这些 image tokens 和文本 tokens 一起送入 LLM，让 LLM 以自回归方式生成这个回答有架构、训练、成本和失败模式。

9.3 回答研究讨论题的通用结构

研究讨论题建议用“七步论文模板”：

1. 背景：这篇论文之前大家怎么做。
2. 问题：前人方法遇到什么瓶颈。
3. 方法：论文提出了什么核心机制。
4. 实验：数据、baseline、指标和 ablation 是否支撑结论。

5. 边界：适用条件、失败模式、隐藏成本。
6. 复现：如果你做，最小复现路径是什么。
7. 下一步：如何改进或迁移到新场景。

如果面试官问“介绍一篇你熟悉的多模态论文”，可以这样组织：

我会先说明它解决的具体问题，比如图文对齐、VLM instruction tuning 或视频生成一致性。然后讲之前方法为什么不够，再讲它的

9.4 CLIP 如何训练

CLIP 的核心目标是把图像和文本映射到同一个语义空间，使匹配的 image-text pair 更接近，不匹配的 pair 更远。

典型结构包括：

1. Image encoder：通常是 ResNet 或 ViT。
2. Text encoder：通常是 Transformer。
3. Projection head：把图像和文本表示映射到同一维度。
4. Contrastive loss：在一个 batch 内让正确图文对相似度最高。

面试回答：

CLIP 使用图文对比学习。一个 batch 里有 N 对 image-text pair，模型分别编码图像和文本，得到归一化 embedding，然后计算 $N*N$ shot classification 和图文检索。

常见追问：“CLIP 为什么能 zero-shot 分类？”

回答：

因为 CLIP 训练时已经把图像和自然语言描述对齐到同一空间。做分类时，可以把类别名写成 prompt，例如 a photo of a dog，把每个

CLIP 的局限：

1. 依赖大规模图文对，数据噪声和偏见会进入模型。
2. 对细粒度计数、空间关系、OCR 不一定强。
3. 相似度对齐不等于深层推理。
4. prompt wording 会影响 zero-shot 结果。
5. 训练目标偏检索和对齐，不直接生成长回答。

9.5 Vision Encoder 面试怎么答

Vision encoder 的作用是把图像变成可供后续模型使用的视觉表示。

常见问题包括：

1. CNN 和 ViT 的区别。
2. Patch embedding 是什么。
3. 为什么 VLM 常用预训练 vision encoder。
4. 图像分辨率如何影响能力和成本。
5. OCR、细粒度识别和空间关系为什么难。

面试回答：

Vision encoder 负责把原始像素转成视觉 token 或全局 embedding。CNN 带有局部卷积和归纳偏置，适合提取局部纹理；ViT 把图像块 attention 建模 patch 间关系，更容易和 Transformer 语言模型生态对接。VLM 中常用预训练 vision encoder，是因为从零训练视觉

图像分辨率的 trade-off：

1. 分辨率越高，细节、OCR、小物体更容易保留。
2. 分辨率越高，patch token 数越多，计算和显存更高。
3. 下采样过强会丢失文字、表格和细粒度空间信息。
4. 多尺度或 crop 策略可以缓解，但系统复杂度更高。

9.6 VLM 架构：图像如何接入 LLM

VLM 常见架构可以拆成三块：

1. Vision encoder：提取视觉特征。
2. Connector：把视觉特征映射到 LLM hidden space。
3. LLM：把 image tokens 和 text tokens 作为上下文生成回答。

Connector 常见形式：

1. Linear projection：简单、成本低。
2. MLP projector：表达更强。
3. Q-Former 或 resampler：用少量 query 压缩视觉 token。
4. Cross-attention：让语言模型某些层 attend 视觉特征。

面试回答：

VLM 的核心问题是视觉表示和语言模型表示空间不一致。Connector 负责把 vision encoder 输出的视觉 token 映射到 LLM 能理解的 hidden space。Q-Former 或 resampler 可以压缩大量视觉 token，降低上下文成本，但可能损失细粒度信息。架构选择本质是在视觉细节、推理能力、成本之间做权衡。

常见追问：“为什么 VLM 会视觉幻觉？”

回答：

视觉幻觉可能来自多个环节：vision encoder 没捕捉到细节，connector 压缩丢信息，LLM 语言先验太强，训练数据里图文不一致，或者推理时温度太高。

9.7 多模态 Instruction Tuning

多模态 instruction tuning 的目标是让模型学会按照用户指令理解图像、文本和对话上下文。

数据形式通常是：

1. 单图问答。
2. 多轮图文对话。
3. OCR 和文档理解。
4. 图表解释。
5. 空间关系、计数和定位。
6. 安全拒答和不确定性表达。

训练中要注意：

1. Image placeholder 和视觉 token 对齐。
2. Chat template 一致。
3. 通常只对 assistant response 算 loss。
4. 图片和文本样本长度差异大，要控制 packing 和 padding。
5. 数据质量比数量更重要，错误 caption 会强化幻觉。

面试回答：

多模态 instruction tuning 不只是把图片和问答拼起来。关键是让模型学会在图像证据约束下回答用户指令。训练时要保证 image token 和 text token 的 loss mask 一致；数据要覆盖描述、OCR、图表、计数、空间关系、多轮对话和拒答边界；评估时要看视觉 grounding，而不是只看问答准确率。

9.8 Diffusion Model 如何生成图像

Diffusion model 的核心思想是学习从噪声逐步还原数据。

训练过程：

1. 从真实图像开始逐步加噪。
2. 模型学习在某个时间步预测噪声或去噪方向。
3. 推理时从随机噪声开始，多步去噪得到图像。
4. 文本条件通过 text encoder 和 cross-attention 或 conditioning 注入。

面试回答：

Diffusion 可以理解为学习一个反向去噪过程。训练时把真实图像加不同强度的噪声，让模型预测加入的噪声或干净图像；推理时从纯 noise 开始，通过 attention 或其他条件机制影响去噪过程。

常见追问：“Latent Diffusion 为什么高效？”

回答：

Latent Diffusion 不直接在像素空间做扩散，而是先用 autoencoder 把图像压到 latent space，在低分辨率潜空间里去噪，再解码回像素空间。

Diffusion 的优缺点：

1. 优点：生成质量高、训练稳定、条件控制灵活。
2. 缺点：采样步数多、推理慢、精确文本和空间控制困难。
3. 风险：偏见、版权、深伪、隐私和安全滥用。

9.9 视频生成为什么难

视频生成比图像生成更难，因为视频同时要求空间质量和时间一致性。

难点包括：

1. 数据量大：视频 token 或 latent 序列远大于图像。
2. 时序一致：人物、物体、背景不能每帧漂移。
3. 物理合理：运动、接触、遮挡、因果关系要可信。
4. 长程规划：视频要有持续事件结构。
5. 评估困难：自动指标很难衡量真实观感和世界一致性。
6. 成本高：训练、存储、采样和人工评估都贵。

面试回答：

视频生成不是简单地逐帧生成图像。逐帧生成会导致闪烁、身份漂移和运动不连续。好的视频模型需要同时建模空间细节和时间动态。

常见追问：“视频生成和 world model 有什么关系？”

回答：

视频生成需要预测未来视觉状态，因此和 world model 有相似目标：都要建模环境动态、物体持久性和动作后果。但当前视频生成模型和 world model 还是分开的。

9.10 Whisper 与语音模型

语音方向常见问题包括 ASR、TTS、语音对话和音频理解。

ASR 的目标是把语音转成文本。典型系统包括：

1. Audio feature extraction，例如 log-mel spectrogram。
2. Encoder，把音频特征编码成序列表示。
3. Decoder 或 CTC，把表示转成文本 token。
4. 多语言和时间戳处理。

面试回答：

语音识别和文本 LLM 的区别在于输入是连续音频信号，需要先转成时频特征或 audio tokens，再建模时间序列。Whisper 这类模型通过自监督学习在音频上建模。

语音接入 LLM 的两种路线：

1. Pipeline：ASR 转文本，再给 LLM，最后 TTS。
2. End-to-end：语音 encoder 直接接入语言模型或统一模型。

Pipeline 简单可控，端到端潜力更大但训练和评估更复杂。

9.11 多模态评估怎么设计

多模态评估不能只问“图片里有什么”。

应覆盖：

1. Caption: 整体描述是否准确。
2. OCR: 文字识别是否正确。
3. Chart/Table: 图表和表格理解。
4. Counting: 数量判断。
5. Spatial relation: 位置、方向、遮挡。
6. Fine-grained recognition: 细粒度类别和属性。
7. Multi-image reasoning: 多图比较和变化检测。
8. Visual grounding: 回答是否基于图像证据。
9. Safety: 危险图像、隐私、人脸、版权、深伪。

面试回答:

我会把多模态评估拆成感知、理解、推理、生成和安全几层。感知层看 OCR、计数、细粒度识别; 理解层看图表、文档、场景关系; 推理层看多图推理、变化检测; 生成层看生成内容; 安全层看危险图像、隐私、人脸、版权、深伪。

多模态评估的风险:

1. Benchmark 污染。
2. 题目过于简单。
3. 只测最终答案, 不测证据定位。
4. LLM judge 对图像不可见或偏差大。
5. 不评估延迟、成本和失败类型。

9.12 多模态安全

多模态模型引入新的安全面。

常见风险:

1. 图像中的隐私信息: 人脸、证件、地址、屏幕截图。
2. OCR prompt injection: 图片里的文字诱导模型忽略系统指令。
3. 视觉误识别导致错误建议。
4. 深伪、版权和生成内容滥用。
5. 医疗、法律、金融等高风险图像解释。
6. 多模态越狱: 用图像承载文本或符号绕过安全策略。

面试回答:

多模态安全要同时考虑视觉内容和文本指令。图片不是可信输入, OCR 出来的文字也可能包含 prompt injection。系统上要区分用户输入和系统输出。

9.13 如何介绍一篇多模态论文

如果面试官问“讲一篇你熟悉的多模态论文”, 推荐模板:

我会先讲它解决的任务, 例如图文对齐、VLM 问答、图像生成或视频生成。然后讲前人方法的瓶颈, 比如视觉特征和语言空间不对齐、视觉推理能力弱、生成内容不可控等。最后讲本文的贡献, 比如提出了新的模型结构、训练方法或评估指标。不要只讲论文结论, 要主动讲边界。

例如讲 LLaVA, 可以强调: 它的重要性在于用相对简单的 vision encoder + projector + LLM 加 instruction tuning 跑通多模态对话范式; 边界是视觉细节、OCR、复杂图表、幻觉和评估可靠性。

例如讲 Stable Diffusion, 可以强调: 它把扩散过程放到 latent space 降低成本; 边界是文本精确控制、手部和空间关系、版权和安全风险。

9.14 如何判断多模态论文贡献是否真实

判断多模态论文, 要重点看六点:

1. 任务是否真实: 是不是解决真实视觉语言问题, 而不是只优化 demo。
2. Baseline 是否强: 是否和已有强 VLM、diffusion 或 video model 比较。
3. 数据是否公平: 是否使用更多更干净数据造成提升。
4. 指标是否可靠: 是否只看主观样例或单一 benchmark。

5. 是否有 ablation: 架构、数据、分辨率、token 数、训练阶段各自贡献。
6. 成本是否透明: 训练成本、推理延迟、显存、人工标注和数据处理成本。

面试回答:

我会特别警惕多模态论文中的 **demo bias**。漂亮样例不等于稳定能力。需要看公开 **benchmark**、真实业务样本、失败案例、消融和成本。

9.15 如何复现一个 VLM 项目

最小复现路线可以这样设计:

1. 选择小规模开源 LLM 和预训练 vision encoder。
2. 用 linear projector 或 MLP connector 对齐视觉特征。
3. 准备小型 image-text instruction 数据。
4. 跑通单图问答和 caption。
5. 评估 OCR、计数、图表、空间关系和幻觉样本。
6. 做 ablation: 冻结/解冻 vision encoder、不同 connector、不同 image token 数、不同数据类型。

面试回答:

如果让我复现一个 VLM, 我不会一开始追求大规模效果, 而会先验证视觉特征能否通过 connector 被 LLM 使用。最小版本可以冻结 vision encoder。

关键坑:

1. Image token 与文本模板不一致。
2. 只看语言流畅度, 不看图像 grounding。
3. 数据中 caption 错误导致幻觉。
4. 训练样本太单一, 模型只学到模板。
5. 评估集被训练数据污染。

9.16 如何设计多模态 ablation

多模态 ablation 要对应明确假设。

常见消融:

1. Vision encoder: CLIP ViT vs 其他 encoder。
2. Connector: linear、MLP、Q-Former、resampler。
3. Image token 数: 更多 token 是否提升 OCR 和细节理解。
4. 分辨率: 高分辨率对小文字和细节的收益。
5. 数据类型: caption、VQA、OCR、chart、multi-turn 的贡献。
6. 训练阶段: 预对齐、instruction tuning、偏好优化各自贡献。
7. 冻结策略: 冻结 vision encoder、冻结 LLM、全参微调或 LoRA。

好的回答:

如果论文声称提升来自更好的 connector, 我会固定 vision encoder、LLM、训练数据和分辨率, 只替换 connector, 并同时看总体 VLM 性能。

9.17 开放研究题怎么回答

开放研究题不是要你给出唯一正确答案, 而是看你如何建模问题。

推荐结构:

1. 先复述问题和目标。
2. 拆出关键瓶颈。
3. 提出 2-3 个假设。
4. 为每个假设设计实验。
5. 说明指标、风险和 trade-off。
6. 给出优先级。

例如问 “如何设计下一代多模态模型”, 可以回答:

我会从架构、数据、训练、评估和安全五个维度设计。架构上考虑更统一的多模态 token 表示，减少手工 connector；数据上需要高

9.18 高频研究题：最近哪篇论文重要

这类题的重点不是论文名，而是判断标准。

可以先说：

我判断一篇论文重要，会看它是否抓住真实瓶颈，是否改变后续方法范式，是否有可信实验支持，是否能影响工程系统或研究路线。

然后选一篇你能讲深的论文。

如果选择 CLIP：强调图文对比学习和 zero-shot 迁移。

如果选择 Stable Diffusion：强调 latent diffusion 降低生成成本。

如果选择 LLaVA：强调用 instruction tuning 把 VLM 对话范式工程化。

如果选择 FlashAttention：强调 IO-aware 系统视角。

如果选择 DPO：强调偏好优化流程简化。

回答时必须补边界，不要只夸。

9.19 高频研究题：如何解释负结果

多模态和研究复现很容易出现负结果。

排查顺序：

1. 数据：图文是否匹配，caption 是否错误，训练集是否太小。
2. 模板：image placeholder、chat template、loss mask 是否正确。
3. 模型：vision encoder、connector、LLM hidden size 是否匹配。
4. 训练：learning rate、batch size、冻结策略、混合精度是否合理。
5. 评估：指标是否覆盖目标能力，样本是否被污染。
6. 规模：方法是否依赖更大模型或更多数据。

面试回答：

负结果不一定说明方法无效。我会先确认实现和数据没有错误，再检查是否复现了论文关键条件。如果仍无效，会分析是规模不足、数

9.20 面试中的常见失分点

Multimodal 与 Research 面试常见失分点包括：

1. 把 VLM 说成“图像特征拼到 LLM”，不讲 connector 和训练目标。
2. 讲 CLIP 只说图文匹配，不讲 batch 内对比学习。
3. 讲 diffusion 只说“从噪声生成图”，不讲训练和反向去噪。
4. 讲视频生成只说“算力大”，不讲时序一致和物理约束。
5. 多模态评估只看 VQA，不看 OCR、图表、计数、幻觉和安全。
6. 研究讨论只背论文名，不讲问题、机制、实验和边界。
7. 判断论文只看 leaderboard，不看 baseline 和 ablation。
8. 提改进点太空泛，没有对应失败模式和实验。
9. 复现方案只说“跑开源代码”，不讲最小复现和控制变量。
10. 遇到不熟论文时假装看过，而不是给分析框架。

9.21 高频题回答模板

9.21.1 CLIP 如何训练

CLIP 用图文对比学习训练 image encoder 和 text encoder。一个 batch 中有 N 对图文样本，模型计算图像和文本 embedding 的 $N \times N$ 分类。

9.21.2 VLM 如何接入图像

典型 VLM 用 vision encoder 把图像编码成视觉 token，再用 connector 把视觉特征映射到 LLM hidden space，最后把 image tokens 和 text tokens 一起输入 LLM。trade-off 是视觉细节、image token 数、训练数据和推理成本。

9.21.3 Diffusion 如何生成图像

Diffusion 训练一个反向去噪过程。训练时给真实图像加不同程度噪声，让模型预测噪声或干净图像；推理时从随机噪声开始逐步去噪，用 text attention 或其他 conditioning 控制生成内容。

9.21.4 视频生成为什么难

视频生成不仅要单帧质量，还要时间一致性、物体持久性、运动合理性和长程事件结构。逐帧生成容易闪烁和身份漂移。训练数据、推理成本、生成速度都是挑战。

9.21.5 如何评价一篇论文贡献

我会先看它解决的问题是否真实重要，再看前人方法为什么不够，然后判断新方法的核心机制和假设。实验上要看强 baseline、公平比较、消融实验和鲁棒性。

9.22 一套完整 Multimodal + Research 面试回答模板

如果被问开放题：“请你设计一个多模态助手，并说明如何研究和评估它”，可以这样组织：

第一，我会先明确任务边界：它是图片问答、文档理解、图表分析、多图推理、语音对话，还是图文生成系统。不同任务需要不同架构和评估指标。

第二，设计模型架构。图片理解场景可以用 vision encoder、connector 和 LLM；生成场景可能需要 diffusion 或 video generation。语音对话需要 ASR 和 TTS 集成。

第三，设计训练数据。数据要覆盖 caption、VQA、OCR、chart、spatial relation、counting、多轮对话和安全边界，并保证 image 和 text 的高质量对齐。

第四，设计评估。不能只看通用分数，要分层评估感知、理解、推理、grounding、幻觉、安全、延迟和成本。对产品场景还要做 bad case 分析和用户测试。

第五，设计研究闭环。如果引入新 connector、新数据或新训练目标，要用 ablation 控制其他变量，比较强 baseline，并分析失败案例的原因。

第六，设计上线风险。多模态输入可能包含隐私、OCR prompt injection、版权和危险内容，因此要有权限控制、内容安全、日志脱敏和模型更新策略。

9.23 准备清单

准备 Multimodal 与 Research 面试时，至少要能回答下面的问题：

1. CLIP 如何训练？为什么能 zero-shot 分类？
2. Vision encoder 在 VLM 中起什么作用？
3. VLM 如何把图像接入 LLM？
4. Connector 有哪些形式，各有什么 trade-off？
5. 多模态 instruction tuning 要注意什么？
6. 为什么 VLM 会有视觉幻觉？
7. Diffusion 的训练和推理过程是什么？
8. Latent Diffusion 为什么更高效？
9. 视频生成为什么比图像生成难？
10. 语音模型和文本 LLM 的区别是什么？
11. 如何设计多模态评估集？
12. 多模态安全有哪些新风险？
13. 如何介绍一篇你熟悉的论文？
14. 如何判断论文贡献是否真实？
15. 如何设计 ablation？
16. 如何复现一个 VLM 或 DPO 项目？
17. 如何解释负结果？
18. 如何提出一个不空泛的研究改进点？

19. 如果现场遇到没读过的论文，如何分析？
20. 如何把多模态论文变成可展示项目？

这一章的核心不是让你背所有多模态模型名，而是建立一种研究型表达方式：先定义问题，再讲机制和证据，最后讲边界、实验和下一步。

第十章：System Design 面试

System Design 面试考的不是你能不能画出一张复杂架构图，而是你能不能在一个开放、模糊、有约束的问题里，主动澄清目标，拆出核心模块，识别瓶颈和 trade-off，并给出可以落地、可以评估、可以演进的系统方案。

大模型系统设计和传统后端系统设计不同。它不仅有流量、存储、缓存、消息队列和可用性问题，还多了模型质量、推理延迟、token 成本、RAG 事实性、Agent 工具安全、评估闭环、模型灰度、prompt 版本和安全治理等问题。

很多候选人这一关失分，不是因为完全不会系统设计，而是回答没有主线：一上来堆 API Gateway、Redis、Kafka、向量数据库、Kubernetes、Prometheus，却讲不清请求怎么流动、核心瓶颈是什么、为什么这样拆模块、系统如何评估、失败时如何降级。

本章重点：系统设计答题流程、ChatGPT 服务、训练平台、推理平台、RAG、Agent、多模态助手、评估平台、安全和成本治理。

10.1 System Design 面试到底考什么

大模型系统设计面试通常考六类能力：

1. 需求澄清：能否把“设计一个 ChatGPT”这种宽题变成有边界的问题。
2. 架构拆解：能否拆出在线链路、离线链路、模型服务、数据服务、安全和观测。
3. 核心机制：是否理解 RAG、推理调度、KV Cache、工具调用、评估平台等大模型特有模块。
4. Trade-off：能否解释质量、延迟、吞吐、成本、安全和可维护性之间的取舍。
5. 故障意识：能否处理超时、OOM、检索失败、模型幻觉、权限泄露、缓存错误和评估污染。
6. 演进能力：能否从 MVP 讲到生产化，再讲到持续优化和反馈闭环。

面试官想看到的是你能不能像负责过系统的人一样思考，而不是背一个标准答案。

10.2 一套通用答题流程

遇到任何大模型系统设计题，都可以按六步回答。

第一步，澄清需求。

可以问：

1. 面向 C 端、企业内部、开发者 API，还是研究平台？
2. 支持文本，还是还要图片、语音、视频？
3. 是否需要 RAG、工具调用、多轮对话和记忆？
4. 流量规模、文档规模、模型规模大概是多少？
5. 主要优化目标是质量、延迟、成本、安全还是可用性？
6. 是否有多租户、权限隔离、合规和审计要求？

第二步，列功能需求和非功能需求。

功能需求是系统要做什么。非功能需求是系统要做到什么程度。

例如设计 RAG，功能需求包括文档上传、解析、切分、索引、检索、重排、生成、引用和反馈；非功能需求包括权限安全、低延迟、高召回、答案可追溯、知识更新及时和成本可控。

第三步，给高层架构。

先给主链路，不要直接陷入数据库字段。

常用骨架：

Client -> Gateway -> Orchestrator -> Retrieval/Tool/Cache -> Model Serving -> Guard -> Response -> Logs/Eval

第四步，深入核心模块。

不同题目的核心模块不同：

1. ChatGPT 服务：会话、上下文、推理、流式输出、安全。
2. 推理平台：路由、调度、batching、KV Cache、模型版本、GPU 利用率。
3. RAG：文档解析、chunk、检索、rerank、引用、权限。
4. Agent：工具注册、规划、状态、权限、失败恢复、审计。
5. 评估平台：数据集、模型输出、指标、judge、人工评审、回归门禁。

第五步，讲权衡和失败场景。

系统设计没有唯一答案。你需要主动讲：为什么这样设计、有什么代价、失败时怎么办。

第六步，讲演进路线。

可以分三个阶段：

1. MVP：打通核心链路。
2. 生产化：补监控、评估、安全、灰度、回滚。
3. 优化：做成本治理、自动路由、数据闭环和持续评估。

10.3 大模型系统的通用架构积木

很多系统题底层积木相似。

10.3.1 接入层

接入层包括 API Gateway、鉴权、限流、租户识别、请求大小限制、基础日志。

大模型系统里，限流不能只按 request 数，还要按 input tokens、output tokens、并发数、模型等级和租户配额。

10.3.2 编排层

编排层决定一次请求怎么执行：是否需要 RAG、是否调用工具、用哪个模型、是否走安全审核、是否命中缓存。

大模型应用的复杂度通常在编排层，而不是单个模型调用。

10.3.3 模型服务层

模型服务层负责推理，核心包括模型加载、tokenizer、chat template、prefill/decode、KV Cache、batching、量化、流式输出和超时取消。

10.3.4 数据与存储层

不同系统需要不同存储：

1. Chat 服务需要会话存储和用户反馈。
2. RAG 需要对象存储、元数据、向量索引、倒排索引。
3. Agent 需要工具状态、执行 trace 和审计日志。
4. 评估平台需要数据集版本、实验结果和人工标注。

10.3.5 安全与权限层

安全不是外挂模块，而是全链路能力。包括输入审核、输出审核、RAG 权限过滤、工具最小权限、PII 检测、prompt injection 防护和审计。

10.3.6 观测与评估层

观测指标包括 latency、TTFT、TPOT、tokens/s、error rate、OOM、cost、user feedback、safety violation、RAG citation accuracy、tool success rate。

没有观测和评估，就无法判断系统变更是否真的变好。

10.4 设计 ChatGPT 类服务

题目可以这样澄清：

我先按一个文本为主的 ChatGPT 类服务设计，支持多轮对话、流式输出、上下文管理、安全审核、可选 RAG 和工具调用，目标是低延

功能需求：

1. 用户登录和会话管理。
2. 多轮对话。
3. 流式输出。
4. 上下文构建和历史压缩。
5. 模型推理。
6. 安全审核。
7. 用户反馈。
8. 模型灰度和回滚。

高层架构：

Web/App

- > API Gateway
- > Auth / Rate Limit
- > Conversation Service
- > Request Orchestrator
- > Context Builder
- > Safety Input Guard
- > Retrieval / Tool / Cache
- > Model Router
- > Model Serving
- > Safety Output Guard
- > Streaming Response
- > Logs / Metrics / Evaluation

核心模块：

1. Conversation Service 保存会话、消息、上下文摘要和用户偏好。
2. Context Builder 决定哪些历史进入 prompt，避免无限堆上下文。
3. Model Router 根据任务难度、上下文长度、租户等级和成本选择模型。
4. Safety Guard 做输入输出安全检查。
5. Streaming Response 降低体感延迟。
6. Feedback Loop 收集点赞、点踩、重试、追问和人工标注。

关键 trade-off：

1. 上下文越长，理解越好，但成本和延迟越高。
2. 安全越严格，风险越低，但误拒可能增加。
3. 强模型质量更好，但成本更高。
4. 流式输出体验更好，但输出审核更复杂。

常见追问：如何处理超长历史？

回答：

我会结合滑动窗口、会话摘要、重要消息保留、用户显式 pin、历史检索和任务相关性筛选。不能简单保留最后 N 条，因为早期约束可

10.5 设计大模型推理平台

题目可以这样澄清：

我先按一个服务多业务、多模型的在线推理平台设计，支持模型注册、版本管理、部署、路由、动态 batching、流式输出、GPU 调度。

高层架构：

Business Service

- > Inference Gateway
- > Auth / Quota / Rate Limit
- > Model Router
- > Request Scheduler
- > Runtime Workers
- > GPU Cluster
- > Metrics / Logs / Traces
- > Model Registry / Config Center

核心模块：

1. Model Registry 保存模型版本、tokenizer、runtime config、资源需求和能力标签。
2. Router 根据模型、租户、token 预算、延迟 SLA 和负载选择 worker。
3. Scheduler 控制 waiting queue、running batch、prefill/decode、token budget 和 KV Cache budget。
4. Runtime Worker 加载模型，执行推理，管理 KV Cache，流式返回。
5. Observability 记录 TTFT、TPOT、queue time、tokens/s、OOM、error rate、GPU utilization。

关键 trade-off：

1. Batch 越大吞吐越高，但等待时间更长。
2. 副本越多可用性越高，但显存成本越高。
3. 量化降低成本，但可能影响质量和格式稳定性。
4. 多租户共享提高利用率，但隔离和公平性更难。

常见追问：延迟变高怎么排查？

回答：

我会把延迟拆成 gateway、queue、tokenization、prefill、decode、output guard、network 几段。如果 TTFT 高，重点看排队、pre

10.6 设计训练平台

训练平台题考察你是否理解大模型训练不是单个脚本，而是数据、调度、分布式训练、checkpoint、监控、评估和成本的系统。

题目澄清：

我先按一个支持预训练、继续预训练和 SFT 的训练平台设计。平台支持数据集管理、训练任务提交、资源调度、分布式训练、checkp

高层架构：

User / Researcher

- > Experiment Portal / CLI
- > Config & Dataset Registry
- > Job Scheduler
- > Training Workers on GPU Cluster
- > Checkpoint Store
- > Metrics / Logs / Traces
- > Evaluation Pipeline
- > Experiment Tracker

核心模块：

1. Dataset Registry 管理数据版本、来源、过滤策略、token 数和权限。
2. Config System 管理模型、优化器、batch、学习率、并行策略和随机种子。

3. Scheduler 分配 GPU、处理队列、优先级、抢占和失败重试。
4. Training Runtime 支持 DDP、FSDP、ZeRO、tensor parallel、pipeline parallel。
5. Checkpoint System 保存模型、optimizer、scheduler、RNG、step、sampler 状态。
6. Monitoring 监控 loss、gradient norm、learning rate、throughput、MFU、GPU utilization、NaN/Inf。
7. Evaluation Pipeline 定期跑 validation loss、benchmark、安全和业务评估。

关键 trade-off:

1. Checkpoint 越频繁，恢复损失越小，但 I/O 成本越高。
2. 更复杂并行策略能训练更大模型，但通信和调试更难。
3. 数据加载越复杂，质量越高，但吞吐可能受影响。
4. 自动恢复能节省成本，但必须防止 silent bug。

常见追问：训练 loss spike 怎么接入平台治理？

回答：

平台要记录 step、rank、batch id、数据 shard、learning rate、gradient norm、NaN/Inf、checkpoint 版本和代码版本。发现 spi

10.7 设计 RAG 系统

题目澄清：

我先按一个企业知识库 RAG 系统设计，支持文档上传、解析、切分、索引、权限控制、混合检索、rerank、答案生成、引用溯源、反
离线链路：

Upload -> Parse -> Clean -> Chunk -> Metadata/Permission -> Embed -> Index -> Version

在线链路：

Query -> Rewrite -> Permission Filter -> Hybrid Retrieve -> Rerank -> Context Builder -> LLM -> Citation -
> Feedback

核心模块：

1. 文档解析处理 PDF、Word、网页、表格、图片 OCR 和代码。
2. Chunking 根据标题、段落、表格、代码块和语义边界切分。
3. Metadata 保存权限、来源、版本、时间、部门和文档类型。
4. Hybrid Retrieval 结合向量检索和 BM25。
5. Reranker 提高最终上下文精度。
6. Context Builder 控制 token budget、去重、排序和引用。
7. Citation 检查答案声明是否被证据支持。

关键 trade-off:

1. Chunk 太小语义不完整，太大噪声多。
2. Top-k 太小漏召回，太大增加噪声和成本。
3. 强 reranker 质量好但延迟高。
4. 严格引用减少幻觉，但可能降低回答流畅性。

常见追问：RAG 回答错了怎么排查？

回答：

我会做错误归因：先看正确文档是否入库，再看解析和 chunk 是否正确，然后看 retriever 是否召回、reranker 是否排前、context

10.8 设计 Agent 平台

Agent 平台题的核心不是“让模型自动规划”，而是让工具调用可靠、安全、可观测、可恢复。

题目澄清：

我先按一个企业 Agent 平台设计，支持工具注册、权限控制、任务规划、工具执行、状态管理、失败恢复、trace 日志、评估和安全

高层架构:

User Request

- > Gateway / Auth
- > Agent Orchestrator
- > Planner / Tool Selector
- > Tool Registry
- > Permission & Policy Check
- > Tool Executor / Sandbox
- > State Store / Memory
- > LLM Response
- > Trace / Audit / Evaluation

核心模块:

1. Tool Registry 记录工具名、描述、schema、权限、风险等级和超时。
2. Planner 决定是否调用工具以及调用顺序。
3. Permission Check 做用户、租户、资源和操作级别校验。
4. Executor 执行工具, 处理超时、重试、错误和幂等。
5. State Store 保存任务状态、工具结果和中间计划。
6. Trace 记录每一步 reasoning 摘要、tool call、参数、结果和错误。
7. Evaluation 测工具选择、参数正确率、任务成功率、安全违规率。

关键 trade-off:

1. Agent 自主性越强, 能力越强, 但风险越高。
2. 工具越多, 覆盖越广, 但选择错误率和权限复杂度上升。
3. 多步计划能解决复杂任务, 但延迟、成本和失败概率增加。
4. 沙箱越严格越安全, 但工具能力可能受限。

常见追问: 如何防 prompt injection?

回答:

外部网页、邮件、文档和工具返回都要视为 **untrusted content**, 不能直接覆盖 **system instruction** 或触发高权限工具。工具调用必

10.9 设计多模态助手

多模态助手题要同时覆盖图片理解、文本对话、RAG、工具、安全和延迟。

题目澄清:

我先按一个支持图片输入和文本对话的多模态助手设计, 支持图片问答、OCR、图表解释、截图理解、可选 RAG 和安全审核。目标是准

高层架构:

Image/Text Input

- > Gateway
- > Image Preprocessor / OCR
- > Vision Encoder / VLM
- > Context Builder
- > Optional Retrieval / Tool
- > LLM / VLM Serving
- > Safety & Privacy Guard
- > Response / Citation
- > Logs / Evaluation

核心模块:

1. 图片预处理: 尺寸、格式、压缩、OCR、隐私检测。
2. VLM 服务: vision encoder、connector、LLM 或统一多模态模型。
3. 多模态 RAG: 图片 caption、OCR、metadata、文本和图像向量索引。

4. 安全：人脸、证件、屏幕截图、OCR prompt injection、版权和危险内容。
5. 评估：OCR、图表、计数、空间关系、视觉幻觉、多图推理。

关键 trade-off:

1. 高分辨率保留细节，但 token 和延迟成本更高。
2. OCR 专用模型更准，但链路更复杂。
3. VLM 端到端更统一，但错误归因更难。
4. 多模态安全比文本更复杂，因为图像里也可能包含指令和敏感信息。

10.10 设计评估平台

评估平台题非常适合展示你对大模型迭代闭环的理解。

题目澄清：

我先按一个支持模型、prompt、RAG、Agent 和 safety 的评估平台设计。平台支持数据集管理、批量生成、自动指标、LLM-as-a-judge、人工评审、回归测试、A/B 实验和发布门禁。

高层架构：

Dataset Registry

- > Eval Job Scheduler
- > Model Runner
- > Metric Calculator
- > Judge Model Service
- > Human Review Tool
- > Result Store
- > Report Dashboard
- > Regression Gate

核心模块：

1. Dataset Registry 管理数据集版本、来源、任务类型、难度、标签和权限。
2. Model Runner 记录模型、prompt、解码参数、工具版本和知识库版本。
3. Metric Calculator 支持准确率、F1、EM、代码测试、retrieval recall、citation accuracy。
4. Judge Service 用 rubric 做 pairwise 或 scalar 评估。
5. Human Review 处理高风险和主观任务。
6. Regression Gate 接入发布流程，阻止质量或安全回退。

关键 trade-off:

1. 自动评测快但可能不符合人类偏好。
2. 人工评审准确但贵且慢。
3. LLM judge 可扩展但有位置偏差、长度偏差和模型偏好。
4. 固定评测集可比较但会过拟合，动态评测集更真实但历史可比性差。

常见追问：如何避免评测污染？

回答：

评测集要分公开集、内部保密集、动态集和线上真实样本集。核心集限制访问，只展示聚合结果。训练数据和评测数据做近重复检测，

10.11 设计安全审核系统

安全审核系统题考察你是否能把 safety 从模型能力变成系统能力。

高层架构：

Request

- > Input Classifier / Policy Engine
- > Risk Router
- > Model / Tool / Retrieval

- > Output Classifier
- > Human Review for High Risk
- > Audit Logs / Red Team Regression

核心设计:

1. 风险 taxonomy: 有害内容、隐私、版权、prompt injection、工具滥用、高风险建议。
2. 输入审核: 判断请求风险和允许的响应等级。
3. 输出审核: 检查模型是否泄露、违规或过度自信。
4. Policy Engine: 按地区、业务、用户等级和风险类别执行策略。
5. Human Review: 处理高风险或低置信样本。
6. Regression Suite: 红队样本和线上事故进入回归集。

关键 trade-off:

1. 拦截越严, 风险越低, 但误杀越高。
2. 人工审核更可靠, 但延迟和成本高。
3. 安全策略透明有助于用户理解, 但也可能被滥用者利用。

10.12 面试追问怎么应对

系统设计面试后半段通常会不断追问。

10.12.1 流量扩大十倍怎么办

回答方向:

1. 水平扩展 gateway、orchestrator 和 stateless 服务。
2. 增加模型副本和 GPU worker。
3. 按 token 而不是请求数负载均衡。
4. 缓存热点 prompt、检索结果和响应。
5. 大小模型路由和降级。
6. 按租户限流和优先级队列。

10.12.2 成本太高怎么办

回答方向:

1. 模型路由: 简单请求用小模型, 复杂请求用大模型。
2. Prompt 压缩和上下文裁剪。
3. 限制 max output tokens。
4. Prefix cache、response cache、retrieval cache。
5. 量化、batching、speculative decoding。
6. 离线预计算 embedding 和热门答案。
7. 监控 cost per successful answer。

10.12.3 质量下降怎么办

回答方向:

1. 确认模型、prompt、template、tokenizer、知识库版本是否变化。
2. 用评估平台复现问题。
3. 做 bad case 分类。
4. 区分检索问题、模型问题、工具问题、安全误杀和数据过期。
5. 回滚或灰度修复。

10.12.4 数据泄露怎么办

回答方向：

1. 立即限制相关功能和访问。
2. 定位泄露路径：RAG 权限、日志、缓存、工具、模型输出。
3. 保留审计日志。
4. 修复权限和脱敏策略。
5. 通知合规流程。
6. 加入回归测试和监控。

10.13 常见失分点

System Design 面试常见失分点包括：

1. 一上来画架构，不澄清需求。
2. 只堆组件名，不讲数据流。
3. 只讲在线链路，不讲离线索引、评估和反馈。
4. 只讲模型，不讲 tokenizer、prompt、版本、缓存和监控。
5. 设计 RAG 时忽略权限和引用。
6. 设计 Agent 时忽略工具权限、审计和失败恢复。
7. 设计推理平台时不讲 batching、KV Cache、TTFT、TPOT。
8. 不讲安全、隐私和合规。
9. 不讲成本估算和降本策略。
10. 不讲灰度、回滚和发布门禁。
11. 不讲指标，无法判断系统是否成功。
12. 遇到追问只加组件，不解释 trade-off。

10.14 高频题回答模板

10.14.1 设计 ChatGPT 服务

我会先澄清用户、规模、SLA、是否需要 RAG 和工具。架构上分为 gateway、会话服务、上下文构建、编排层、模型路由、推理服务、

10.14.2 设计 RAG 系统

我会拆成离线文档链路和在线查询链路。离线链路做解析、清洗、chunk、metadata、权限、embedding 和索引；在线链路做 query re off 是召回率、上下文噪声、延迟、权限和引用准确性。

10.14.3 设计推理平台

推理平台要支持模型注册、版本管理、路由、调度、GPU worker、batching、KV Cache、流式输出、监控和灰度。路由不能只按请求数

10.14.4 设计评估平台

评估平台要管理数据集版本、模型和 prompt 版本、批量生成、自动指标、LLM-as-judge、人工评审、回归测试和报告。所有评测结果

10.15 一套完整 System Design 回答模板

如果被问任意大模型系统设计题，可以用下面模板组织：

第一，我会先澄清场景和目标，包括用户是谁、输入输出是什么、是否需要 RAG/工具/多模态、流量规模、SLA、安全和成本约束。

第二，我会列功能需求和非功能需求。功能需求描述系统做什么，非功能需求包括延迟、吞吐、可用性、权限、隐私、可观测、可评

第三，我会给高层架构。通常包括 gateway、auth/rate limit、orchestrator、retrieval/tool/cache、model serving、safety gua

第四，我会深入核心模块。不同题目重点不同：RAG 讲文档和检索，推理平台讲调度和 KV Cache，Agent 讲工具和权限，评估平台讲数

第五，我会讲 trade-off 和失败场景，例如质量和成本、延迟和准确率、安全和误杀、缓存和隐私、召回和噪声，并给出降级、回滚

第六，我会讲演进路线。第一阶段打通 MVP，第二阶段补齐监控、安全和评估，第三阶段做自动路由、成本优化和反馈闭环。

10.16 面试前一页速记

澄清问题：

1. 用户是谁？
2. 输入输出是什么？
3. 是否需要多轮、RAG、工具、多模态？
4. 规模和 SLA 是什么？
5. 安全、隐私、权限和成本要求是什么？

架构主线：

Gateway -> Auth/RateLimit -> Orchestrator -> Retrieval/Tool/Cache -> Model Serving -> Safety Guard -> Response -> Logs/Eval

必讲指标：

1. 延迟：TTFT、TPOT、P95、P99。
2. 吞吐：QPS、tokens/s、GPU utilization。
3. 质量：任务成功率、人工胜率、用户满意度。
4. RAG: Recall@k、MRR、citation accuracy、faithfulness。
5. Agent: tool selection accuracy、argument accuracy、task success rate。
6. 安全：harmful compliance、over-refusal、prompt injection success rate。
7. 成本：每请求成本、每百万 token 成本、GPU 小时。

必讲能力：

1. 权限隔离。
2. 安全审核。
3. 监控告警。
4. 评估体系。
5. 灰度回滚。
6. 成本优化。
7. 故障降级。
8. 数据和反馈闭环。

这一章的核心不是让你背一个架构图，而是让你形成一种面试现场的系统思维：先定边界，再画主链路，深入核心模块，主动讲 trade-off，最后用指标和演进路线收束。

第十一章：Behavioral Interview

Behavioral Interview 考的不是你会不会说漂亮话，而是你在真实工作中的判断方式、协作方式、抗压方式、学习方式和价值取舍。对于大模型算法岗，行为面试尤其会关注一个问题：你能不能在高不确定、高成本、高安全要求的环境里可靠地工作。

很多技术候选人会低估 behavioral 面试。他们觉得只要算法、系统和论文讲得好，行为题随便答就行。实际情况相反：顶级团队通常非常重视候选人的协作、诚实、责任感和安全意识。因为大模型项目成本高、影响大、跨团队复杂，一个技术能力很强但不诚实、不合作、不复盘、不考虑风险的人，可能给团队带来巨大代价。

本章重点：STAR 框架、自我介绍、失败经历、冲突协作、主动性、学习能力、压力管理、伦理安全、英文行为面试和反向提问。

11.1 Behavioral 面试到底考什么

Behavioral 面试通常考八类能力：

1. 动机匹配：为什么选择大模型方向，为什么选择这个团队。
2. 责任感：是否愿意 owner 一个问题，而不是只完成分配任务。
3. 协作能力：能否和研究、工程、产品、安全、数据团队合作。
4. 冲突处理：遇到技术分歧、优先级冲突、资源冲突时怎么做。
5. 失败复盘：能否承认失败、定位原因、更新方法。
6. 学习能力：面对新论文、新工具、新方向时如何快速上手。
7. 伦理和安全意识：是否理解大模型能力带来的风险和责任。
8. 沟通表达：能否清晰、诚实、有结构地讲复杂经历。

Behavioral 面试不是和技术面试割裂的。你讲失败经历时，可以讲一次训练 loss spike 排查失败；讲协作时，可以讲评估口径和产品目标冲突；讲伦理时，可以讲安全拒答、隐私数据或 RAG 权限边界。

高质量 behavioral 回答应该让面试官看到：你不只是会做题，而是可以被信任地放进真实团队。

11.2 STAR 框架：行为题的基础结构

行为题最常用的结构是 STAR：

1. Situation：当时背景是什么。
2. Task：你要解决什么问题。
3. Action：你具体做了什么。
4. Result：结果如何，学到了什么。

但普通 STAR 对大模型算法岗还不够。建议升级成 STAR-L：

1. Situation：背景和约束。
2. Task：目标和责任边界。
3. Action：你的关键判断和行动。
4. Result：结果和指标。
5. Learning：失败、反思和下一步。

例如问“讲一次失败经历”，差回答是：

有一次项目延期了，后来我们加班完成了。我学到要更努力。

更好的回答是：

我在一次 RAG 项目中最初判断错误，以为主要问题是生成模型不够强，所以先做了 SFT。但上线前评估发现幻觉下降不明显。后来我...
这个回答有背景、任务、行动、结果和学习。

11.3 自我介绍怎么讲

自我介绍不是背简历。它的目标是建立一个清晰人设：你是谁，你的主线能力是什么，你为什么适合这个岗位。

推荐结构：

1. 一句话定位。
2. 两到三个能力支柱。
3. 一个代表性项目或研究经历。
4. 和目标岗位的匹配。
5. 未来想贡献什么。

中文模板：

我是一名以大模型算法和工程落地为主线的候选人，重点准备和实践过 Transformer 原理、SFT/DP0、RAG、评估和推理部署几个方向。

英文模板：

I am an algorithm engineer focusing on large language models, especially post-training, retrieval-augmented generation, evaluation, and inference systems. One representative project I worked on was an enterprise knowledge base QA system, where I helped improve answer reliability by combining data curation, SFT, retrieval quality analysis, and more. That is why I am interested in LLM roles that combine research judgment with engineering execution.

注意事项：

1. 不要超过 2 分钟。
2. 不要罗列所有技能。
3. 不要把自己包装成全能专家。
4. 要给面试官明确追问入口。
5. 结尾要自然连接目标岗位。

11.4 为什么选择大模型方向

这个问题看似简单，实际上在考动机是否真实。

差回答：

因为大模型很火，发展前景好。

更好的回答要包含个人经历和技术判断：

我选择大模型方向，不只是因为它热门，而是因为它把机器学习、系统工程、数据、评估和产品影响结合得非常紧。传统模型很多时候

如果目标是 OpenAI 风格团队，可以强调：

1. 对模型能力边界感兴趣。
2. 对安全和社会影响有敬畏感。
3. 喜欢高标准、跨学科、快速迭代的环境。
4. 能接受不确定性和高难度问题。

11.5 讲一次失败经历

失败题是 behavioral 面试最高频问题之一。

面试官想看：

1. 你是否诚实。
2. 你是否真的经历过复杂问题。
3. 你如何定位原因。
4. 你是否能从失败中更新方法。
5. 你是否会推卸责任。

好的失败案例应满足三个条件：

1. 真实具体。
2. 有技术或协作复杂度。
3. 有明确学习和后续改变。

模板：

我想讲一次技术判断上的失败。当时我们希望降低企业问答模型的幻觉，我一开始认为主要瓶颈是模型缺少领域知识，所以优先做了

不要这样答：

1. “我没有失败经历。”
2. “失败主要是别人没配合。”
3. “问题是资源不够。”但没有说明你做了什么。
4. 讲一个无关痛痒的小错误。
5. 只讲结果，没有讲学习。

11.6 讲一次团队冲突

冲突题不是让你证明自己永远正确，而是看你如何处理分歧。

大模型项目中常见冲突：

1. 研究团队想追求效果，平台团队担心成本和稳定性。
2. 产品团队想尽快上线，安全团队认为风险未评估。
3. 算法团队想增加上下文，推理团队担心延迟和显存。
4. 数据团队认为标注标准合理，模型团队发现标注噪声影响训练。
5. 评估团队认为指标不可信，业务团队只关心上线速度。

回答模板：

有一次我们在 RAG 系统中对是否加入强 reranker 有分歧。算法侧认为 reranker 能显著提升答案质量，工程侧担心延迟超出 SLA。我们 off 量化。

好回答的关键：

1. 不贬低对方。
2. 不把冲突讲成情绪问题。
3. 把分歧转成可验证问题。
4. 给出折中或分层方案。
5. 说明学到了什么。

11.7 讲一次主动性

主动性不是“我加班很多”，而是你发现了没人明确要求但对项目有价值的问题，并推动它解决。

大模型项目中的主动性例子：

1. 主动建立 bad case taxonomy。
2. 主动补充评估集，发现原指标不可信。
3. 主动做成本 dashboard。
4. 主动整理训练失败排查手册。
5. 主动把红队样本沉淀成 regression suite。
6. 主动发现数据隐私风险并推动修复。

回答模板：

在一次模型微调项目中，原计划只看自动评测分数。但我发现自动分数提升后，抽样回答中仍有不少引用不支持结论的问题。虽然这2主动性的重点是：你不是盲目多做，而是发现关键风险或机会，并用可执行方式推动。

11.8 讲一次快速学习

大模型领域变化快，学习能力很重要。

面试官可能问：

1. 你最近学了什么新技术？
2. 你如何快速上手一个陌生方向？
3. 讲一次你从零学会某个工具或论文线的经历。

回答结构：

1. 为什么需要学。
2. 你如何拆解学习路径。
3. 你如何实践验证。
4. 你如何沉淀和迁移。

示例：

我之前主要做文本模型，对 VLM 不熟。为了准备一个图片问答项目，我先把问题拆成四块：vision encoder、connector、LLM 接入和 2 相关资料，第二周用开源模型跑通图片问答 demo，第三周构造了一个小型评测集覆盖 OCR、计数和图表理解。这个过程让我不仅知道这个答案比“我看了很多论文”更有说服力，因为它包含学习路径和实践验证。

11.9 讲一次压力和优先级管理

大模型项目经常有紧急上线、算力窗口、实验排期、线上事故和评审 deadline。

面试官想看你是否能在压力下保持清晰。

回答模板：

有一次我们需要在一周内完成模型版本评估并决定是否灰度上线。任务很多，包括离线 benchmark、人评、安全测试和延迟压测。我的重点不是说你能熬夜，而是你能排序、沟通和控制风险。

11.10 讲一次伦理和安全判断

大模型岗位 behavioral 面试很可能问伦理、安全、隐私或社会影响。

常见问题：

1. 如果模型能力提升但安全指标下降，你怎么办？
2. 如果产品要求快速上线但安全评估没完成，你怎么办？
3. 如果你发现训练数据里有敏感信息，你怎么办？
4. 如果用户要求模型做高风险建议，你怎么看？
5. 如果你的模型可能被滥用，你会怎么设计防护？

回答原则：

1. 不要表现得“安全只是阻碍上线”。
2. 不要空泛说“遵守规范”。
3. 要讲风险分级、评估、沟通和可执行措施。
4. 要承认 trade-off，但不能绕过底线。

示例：

如果新模型整体 helpfulness 提升，但 jailbreak 成功率也明显上升，我不会直接建议全量上线。我会先看风险严重度和攻击类型，

11.11 讲一次和非技术团队协作

大模型项目经常需要和产品、运营、法律、安全、标注团队协作。

回答模板：

在一个企业知识库问答项目中，产品团队希望模型尽量回答，减少拒答；安全和法务团队更关注错误回答和越权信息。我的做法是把

关键点：

1. 把抽象分歧变成规则和指标。
2. 用共同语言沟通，不堆技术术语。
3. 尊重其他团队的约束。
4. 最后形成可执行流程。

11.12 如何回答“不确定”的问题

Behavioral 面试也会观察你如何处理不知道的问题。

优秀候选人不会硬编。

中文模板：

这个细节我不能保证完全准确。我会先从目标、数据、优化和系统约束几个角度分析。如果是在真实项目中，我会先查官方文档或论文

英文模板：

I am not fully certain about the exact implementation detail, but I would reason about it from the objective, data, optimization, and engineering perspective. This type of answer reflects honesty and engineering reliability.

11.13 英文 Behavioral 面试表达

英文行为面试最重要的是结构清楚，不是词汇华丽。

11.13.1 失败经历英文模板

One failure I learned a lot from happened in a retrieval-augmented QA project. Initially, I assumed the main issue was that of model tuning. However, the hallucination rate did not improve as much as expected. After doing a more systematic error analysis, I realized the issue was related to the response format. The main lesson for me was to do error attribution before jumping to model training.

11.13.2 冲突经历英文模板

In one project, there was a disagreement between the algorithm side and the engineering side about whether to add a heavy reranker for high confidence retrieval cases. The result showed that the strong reranker was most useful for complex and low-confidence cases, but not necessary for simple queries. So we adopted a conditional reranking strategy. This experience taught me to communicate clearly and find a balanced solution.

11.13.3 为什么加入这个团队英文模板

I am interested in this team because the work seems to combine frontier model capabilities with real-world reliability and safety constraints. That combination is exactly what I want to work on. I enjoy problems where model performance and safety are both important.

11.14 反向提问怎么问

反向提问不是随便问福利或流程，而是展示你关心什么。

好的问题应该围绕团队目标、工作方式、技术挑战和成功标准。

可以问：

1. 团队当前最核心的技术瓶颈是什么，是模型能力、数据、评估、推理成本还是安全？
2. 这个岗位入职前三个月最希望解决什么问题？
3. 团队如何判断一个模型或系统改动是否值得上线？
4. 你们如何平衡研究探索和产品交付？
5. 团队在 safety、privacy 和 evaluation 上有哪些发布门禁？
6. 对这个岗位来说，优秀候选人和普通候选人的最大区别是什么？
7. 这个方向未来 6 到 12 个月最重要的挑战是什么？

不要问：

1. 官网能查到的信息。
2. 过早聚焦薪资和假期。
3. 暗示你只关心 title 或资源。
4. 让面试官评价其他候选人。

11.15 Behavioral 面试常见失分点

常见失分点包括：

1. 回答太抽象，没有具体经历。
2. 只讲团队做了什么，不讲自己做了什么。
3. 把失败都归因于别人或资源不足。
4. 讲冲突时贬低其他团队。

5. 只讲努力，不讲判断。
6. 只讲结果，不讲学习。
7. 不敢承认不确定性。
8. 伦理安全题只说口号，没有可执行措施。
9. 英文回答句子很复杂，但结构混乱。
10. 反向提问没有技术深度。

11.16 准备自己的故事库

面试前要准备一个故事库，而不是临场想。

至少准备 8 个故事：

1. 一个代表性项目成功案例。
2. 一个失败案例。
3. 一个技术判断错误后修正的案例。
4. 一个团队冲突或分歧案例。
5. 一个主动发现问题并推动解决的案例。
6. 一个快速学习新方向的案例。
7. 一个压力和优先级管理案例。
8. 一个安全、隐私或伦理判断案例。

每个故事都要准备三个版本：

1. 30 秒版本：一句话讲清。
2. 2 分钟版本：STAR-L 完整讲述。
3. 5 分钟版本：准备细节追问。

故事卡片模板：

标题：

问题：

我的角色：

关键约束：

行动：

结果：

失败或代价：

学到什么：

可被追问的细节：

11.17 高频题回答模板

11.17.1 你最大的失败是什么

我最大的失败之一是过早假设问题来自模型能力，而没有先做足错误归因。当时一个 RAG 项目幻觉较多，我优先做了 SFT，但效果有限。

11.17.2 你如何处理团队分歧

我会先确认分歧背后的目标是否一致，然后把争论转成可验证问题。比如质量和延迟冲突时，我会设计分桶实验，分别测质量收益、延迟和成本。

11.17.3 你如何快速学习新方向

我通常会先建立问题地图，再选最小实践闭环。比如学习 VLM 时，我先拆成 vision encoder、connector、LLM 接入和 instruction tuning 四个模块。

11.17.4 如果发现安全风险但上线压力很大怎么办

我会先评估风险严重度和影响范围。如果是高严重度风险，应该进入发布门禁，不能因为上线压力忽略。可以提出替代方案，比如限制功能范围或延迟发布。

11.18 最后一页速记

Behavioral 面试记住四句话：

1. 用具体故事回答，不要讲抽象品质。
2. 用 STAR-L 结构组织：背景、任务、行动、结果、学习。
3. 讲失败和冲突时，不推责、不贬低别人，把问题转成可验证和可改进的行动。
4. 大模型岗位必须体现安全、评估、成本、协作和诚实意识。

面试前检查：

1. 自我介绍是否有清晰主线。
2. 是否准备了 8 个故事。
3. 每个故事是否讲清自己的贡献。
4. 是否能讲失败和反思。
5. 是否能用英文讲 2 个故事。
6. 是否准备了 5 个高质量反向问题。
7. 是否能在不确定时诚实表达边界。

这一章的核心不是教你包装人设，而是帮助你把真实经历组织成可信证据：你能解决问题，能和人合作，能从失败中学习，也能在大模型这种高影响技术上保持责任感。

第十二章：Mock Interview 与复盘

Mock Interview 的目标不是让你提前背答案，而是把知识、项目、系统设计、研究讨论和行为表达放进真实压力环境里检验。很多人平时看书能看懂，写笔记也完整，但一到面试就讲散、讲慢、讲不到重点、被追问就慌。这说明问题不在知识输入，而在面试输出没有训练。

真正有效的模拟面试必须包含三件事：真实流程、明确评分、严格复盘。只做题不复盘，等于重复错误；只复盘不再练，等于知道问题但没有形成新习惯。

本章重点：模拟面试流程、45 分钟脚本、评分表、错题归因、表达改进、项目深挖、系统设计训练、英文 mock、复盘记录和下一轮准备。

12.1 Mock Interview 到底训练什么

Mock Interview 训练的不是单点知识，而是五种综合能力：

1. 快速定位问题：听到题目后知道它属于基础、训练、部署、对齐、系统、研究还是行为题。
2. 结构化表达：能先给结论，再按层次展开。
3. 抗追问能力：面试官连续追问细节时不慌、不硬编。
4. 时间控制：能在 2 分钟、5 分钟、10 分钟版本之间切换。
5. 复盘迭代：能把一次失败面试转化为下一轮具体改进。

大模型算法岗的 mock 不能只问八股题。至少要覆盖：

1. 自我介绍。
2. 项目深挖。
3. ML/LLM 基础。
4. Training / Deployment / Alignment 专项。
5. System design。
6. Research discussion。
7. Behavioral。
8. 英文表达。

12.2 一次完整 Mock 的标准流程

建议用 45 分钟作为标准 mock，因为它接近真实单轮面试长度。

标准流程：

时间	模块	内容
0-3 分钟	开场	自我介绍和岗位匹配
3-12 分钟	项目深挖	一个核心项目，追问数据、方法、实验、失败
12-22 分钟	技术基础	Transformer、训练、对齐、部署等基础题
22-34 分钟	系统或研究	系统设计题或论文讨论题
34-40 分钟	Behavioral	失败、冲突、主动性、安全判断
40-43 分钟	反向提问	候选人提问
43-45 分钟	即时反馈	面试官给 1-2 条关键反馈

如果时间只有 30 分钟，可以压缩成：自我介绍 2 分钟，项目 8 分钟，技术 10 分钟，系统或行为 7 分钟，反馈 3 分钟。

如果是专项 mock，例如部署面试，可以把 30 分钟全部用于一个方向：5 分钟基础概念，15 分钟系统设计，5 分钟线上排障，5 分钟复盘。

12.3 Mock 前怎么准备

Mock 前不要临时乱翻资料。每次 mock 前准备四件事。

第一，确定目标。

本次 mock 只重点训练 1-2 个目标。例如：

1. 项目表达是否清楚。
2. Training 追问能否顶住。
3. System design 是否有主线。
4. Behavioral 是否具体。
5. 英文回答是否流畅。

第二，准备材料。

至少准备：

1. 2 分钟自我介绍。
2. 1 个 3 分钟项目讲稿。
3. 5 个项目可追问细节。
4. 10 道本轮方向题。
5. 1 个系统设计题或研究题。

第三，设置约束。

真实面试不会让你无限想。可以设置：

1. 基础题思考不超过 10 秒。
2. 开放题先用 20 秒组织结构。
3. 项目介绍不超过 3 分钟。
4. 不确定时必须明确说边界。

第四，录音或记录。

如果条件允许，录音很有价值。很多表达问题只有听回放才会发现，比如语速太快、口头禅太多、结构混乱、结论太晚出现。

12.4 面试官脚本：如何给自己做 Mock

如果没有真人面试官，也可以自问自答，但要模拟追问。

基础脚本：

1. 请用 3 分钟介绍你自己，并说明为什么适合大模型算法岗。
2. 讲一个你最熟悉的大模型项目。
3. 这个项目最难的问题是什么？
4. 你怎么评估结果可信？
5. 有没有失败实验？你怎么解释？
6. 如果让你重新做，你会改什么？
7. Transformer 里 attention 解决什么问题？
8. SFT、RLHF、DPO 有什么区别？
9. KV Cache 为什么能加速推理？
10. 设计一个企业知识库 RAG 系统。
11. 讲一次你和别人有技术分歧的经历。
12. 你有什么问题想问我？

追问原则：

1. 每个项目回答至少追问 3 层。
2. 每个技术题至少问一个边界或失败模式。
3. 每个系统设计题至少问一次成本、一次安全、一次故障。
4. 每个 behavioral 题都追问“你具体做了什么”。

示例追问链：

你说 RAG 幻觉下降了。
如何定义幻觉？
评估样本怎么来？
人评 rubric 是什么？
如果检索到了证据但模型仍然答错，你怎么归因？
这个方案上线后延迟增加了吗？

这种追问比单题问答更接近真实面试。

12.5 评分表：不要只凭感觉判断好坏

每次 mock 后都要打分。建议用 1 到 5 分。

维度	1 分	3 分	5 分
结构	没有主线	有基本结构	先结论后展开，层次清楚
准确性	概念错误多	主要正确但有模糊点	准确且能讲边界
深度	只会定义	能讲机制	能讲 trade-off、失败模式和实验
项目可信度	像旁观者	能讲自己贡献	能讲决策、失败、指标和反思
系统意识	只讲模型	能讲模块	能讲成本、安全、监控、降级
研究能力	背结论	能讲论文	能判断贡献、设计复现和 ablation
表达	断裂混乱	基本易懂	简洁、有重点、可追问
诚实边界	硬编	偶尔承认不确定	主动区分事实、假设和待验证

一次 mock 的目标不是每项都 5 分，而是找到最低的 1-2 项优先改。

12.6 错题归因：不要只写“不会”

复盘时最常见的错误是只写：这题不会。

这没有用。必须做归因。

错题可以分为八类：

1. 知识缺口：确实不知道概念。
2. 机制不清：知道名词，但讲不清原理。
3. 边界缺失：只讲优点，不讲缺点和适用场景。

4. 表达混乱：知道内容，但组织不好。
5. 项目不熟：讲不清自己的数据、实验、指标或贡献。
6. 系统意识弱：忽略成本、安全、监控、失败场景。
7. 研究判断弱：不会评价论文、设计 ablation 或解释负结果。
8. 心态问题：被追问后紧张、硬编、急于辩解。

复盘模板：

原题：

我的原回答：

问题类型：知识缺口 / 机制不清 / 边界缺失 / 表达混乱 / 项目不熟 / 系统意识弱 / 研究判断弱 / 心态问题

正确回答要点：

背后关联概念：

下一次 2 分钟版本答案：

需要补的资料或练习：

12.7 表达问题怎么改

很多人 mock 后发现不是不会，而是表达不稳定。

常见表达问题：

1. 开头太绕，迟迟不给结论。
2. 只讲细节，没有框架。
3. 讲太多背景，时间不够。
4. 每句话都加“可能、就是、然后”。
5. 被打断后回不到主线。
6. 不知道什么时候停。
7. 不确定时硬编。

改进方法：

1. 每题先用一句话给结论。
2. 用编号表达，例如“我会从三点回答”。
3. 开放题先讲框架，再讲细节。
4. 每个回答准备 30 秒、2 分钟、5 分钟版本。
5. 被打断后用“我先回到核心问题”收束。
6. 不确定时说“这个细节我不完全确定，但我会这样分析”。

示例：

差回答：

KV Cache 就是那个推理的时候会缓存一些东西，然后可以避免重复计算，所以会快很多，但是也会占显存。

更好回答：

KV Cache 的核心作用是加速自回归 decode。它缓存每层历史 token 的 Key 和 Value，使生成新 token 时不需要重新计算完整上下文。

12.8 项目深挖怎么练

项目深挖是 mock 中最重要的模块。

每个项目至少准备十类问题：

1. 背景：为什么做这个项目？原问题是什么？
2. 目标：优化什么指标？为什么这个指标可信？
3. 数据：数据来源、清洗、标注、隐私和偏差。
4. 方法：候选方案有哪些？为什么选这个？
5. 训练：模型、超参、loss、训练稳定性。
6. 评估：baseline、指标、人评、统计显著性。
7. 失败：哪些实验没成功？为什么？

8. 系统：延迟、成本、监控、上线和回滚。
9. 安全：误拒、幻觉、隐私、权限、滥用。
10. 反思：如果重做会改什么？

项目复盘表：

问题	我的答案是否清楚	证据	需要补充
我解决的问题是什么			
我的个人贡献是什么			
baseline 是什么			
指标怎么定义			
失败实验是什么			
最大 trade-off 是什么			
线上或系统约束是什么			
安全和隐私怎么处理			

12.9 系统设计 Mock 怎么练

系统设计 mock 的关键是流程稳定。

每次系统设计都按六步：

1. 澄清场景。
2. 列需求。
3. 画主链路。
4. 深入核心模块。
5. 讲 trade-off 和故障。
6. 讲演进和指标。

常用题库：

1. 设计 ChatGPT 服务。
2. 设计企业 RAG 系统。
3. 设计大模型推理平台。
4. 设计训练平台。
5. 设计 Agent 平台。
6. 设计多模态助手。
7. 设计实时语音助手。
8. 设计评估平台。
9. 设计安全审核系统。
10. 设计模型路由与缓存系统。

每道题必须主动讲四件事：

1. 指标：如何判断系统成功。
2. 成本：如何控制 token、GPU 和人工成本。
3. 安全：权限、隐私、滥用和审核。
4. 失败：故障、降级、回滚和监控。

12.10 Research Mock 怎么练

研究讨论 mock 要训练“判断力”，不是论文背诵。

常用题：

1. 最近哪篇论文重要？为什么？
2. 如何复现 DPO？
3. 如何判断一个新 attention 方法是否值得？
4. 如何设计 RAG 方法的 ablation？

5. 一个论文结果和你的直觉相反，你怎么办？
6. 如何解释负结果？
7. 如何提出一个研究改进点？
8. 如何把论文变成项目？

研究题回答必须包含：

1. 问题背景。
2. 方法机制。
3. 实验设计。
4. 贡献边界。
5. 复现路径。
6. 下一步。

如果没读过某篇论文，不要硬编。可以说：

这篇我没有完整读过，但我会从问题是否重要、baseline 是否公平、实验是否支持核心假设、成本和边界是否清楚这几个角度判断。

12.11 Behavioral Mock 怎么练

Behavioral mock 要准备故事库。

至少练 8 个故事：

1. 成功项目。
2. 失败项目。
3. 技术判断错误。
4. 团队冲突。
5. 主动性。
6. 快速学习。
7. 压力管理。
8. 安全伦理判断。

每个故事用 STAR-L 讲：

1. Situation：背景。
2. Task：任务。
3. Action：行动。
4. Result：结果。
5. Learning：学习。

Mock 时要特别检查：

1. 是否讲清自己的贡献。
2. 是否推卸责任。
3. 是否有具体行动。
4. 是否有结果或证据。
5. 是否有反思。

12.12 英文 Mock 怎么练

英文 mock 不要求词汇华丽，要求结构清楚、稳定、不硬翻译中文长句。

训练顺序：

1. 先准备英文自我介绍。
2. 准备一个英文项目 2 分钟版本。
3. 准备一个英文失败经历。
4. 准备 5 个技术解释模板。
5. 练习不确定时的英文表达。

不确定时可以说：

I am not fully certain about the exact detail, but I would reason about it from the objective, data, optimization, and system

项目开场可以说：

One project I know well is an enterprise RAG system. The main goal was to reduce hallucination and improve answer traceability

英文复盘时重点听：

1. 是否句子太长。
2. 是否中英文混杂太多。
3. 是否术语准确。
4. 是否能自然表达 trade-off。
5. 是否能承认不确定。

12.13 Mock 频率怎么安排

如果还有 3 个月准备：

1. 第 1 个月：每周 1 次，重点项目和基础。
2. 第 2 个月：每周 2 次，加入系统设计和专项追问。
3. 第 3 个月：每周 2-3 次，按真实面试强度模拟。

如果还有 1 个月准备：

1. 每周至少 2 次完整 mock。
2. 每天 20 分钟口头回答训练。
3. 每两天复盘一个项目或系统题。
4. 每周至少一次英文 mock。

如果只有 1 周：

1. 不要再大面积学新知识。
2. 集中打磨自我介绍、项目、系统设计模板和高频错题。
3. 每天做一次 30 分钟 mock。
4. 每次只改 1-2 个关键问题。

12.14 如何判断自己准备好了

可以用下面标准自测：

1. 能稳定完成 2 分钟自我介绍。
2. 能把核心项目讲出 1、3、10 分钟版本。
3. 能回答 50 道高频基础题，且能讲边界。
4. 能完成 5 类系统设计题的主链路。
5. 能讲 3 篇论文的背景、方法、实验和边界。
6. 能讲 5 个 behavioral 故事。
7. 被追问不知道的问题时不会硬编。
8. 每次 mock 后能形成具体改进项。

如果你仍然出现以下问题，说明还没准备好：

1. 项目讲不清个人贡献。
2. 系统设计只会堆组件。
3. 论文只会背结论。
4. 被追问就慌或硬编。
5. 复盘记录总是“继续学习”，没有具体行动。

12.15 复盘记录模板

每次 mock 后，用下面模板记录。

日期：
面试类型：基础 / 项目 / 系统 / 研究 / Behavioral / 英文 / 综合
面试时长：
本次目标：

题目列表：
1.
2.
3.

表现最好的 2 点：
1.
2.

最需要改的 2 点：
1.
2.

错题归因：
题目：
原回答：
问题类型：知识缺口 / 机制不清 / 边界缺失 / 表达混乱 / 项目不熟 / 系统意识弱 / 研究判断弱 / 心态问题
修正答案：

下一轮行动：
1.
2.

复盘必须具体到行动。例如：
差复盘：
系统设计还要加强。
好复盘：

设计 RAG 时漏讲权限和引用。明天补一版 RAG 系统 5 分钟答案，必须包含 permission filter、citation accuracy、faithfulness 和

12.16 一周 Mock 闭环示例

一个高效的一周安排：

日期	训练内容	目标
周一	项目 mock	打磨 3 分钟项目讲述
周二	基础题口头训练	每题 2 分钟讲清机制和边界
周三	系统设计 mock	练 RAG 或推理平台主链路
周四	Research mock	讲一篇论文和一个 ablation
周五	Behavioral mock	练失败、冲突、安全判断
周六	综合 45 分钟 mock	模拟真实面试
周日	深度复盘	整理错题、重录关键答案

周日不要只休息，也不要继续刷新内容。最有价值的是把一周暴露的问题转成下一周训练计划。

12.17 最后一页速记

Mock Interview 记住五句话：

1. 不 mock，不知道自己真实表达水平。
2. 不录音，很难发现表达习惯问题。
3. 不复盘，mock 只是重复错误。
4. 不重练，复盘不会自动变成能力。
5. 每次只改 1-2 个关键问题，进步最快。

面试前最后检查：

1. 自我介绍是否流畅。
2. 项目是否能被连续追问 5 层。
3. 基础题是否能先结论后机制。
4. 系统设计是否能主动讲指标、成本、安全和失败。
5. 研究题是否能讲背景、实验、边界和下一步。
6. Behavioral 是否有具体故事。
7. 英文回答是否结构清楚。
8. 不确定时是否能诚实表达。

这一章的核心是：面试能力不是看出来的，是练出来、录出来、复盘出来、重练出来的。

第十三章：六个月冲刺计划

六个月冲刺计划的目标，不是把所有大模型知识都学完，而是在有限时间内形成可面试、可展示、可追问的竞争力。真正能帮助你拿到高水平大模型算法岗面试机会的，不是“我看了很多资料”，而是你能稳定展示四类证据：基础扎实、项目可信、研究有判断、系统有工程意识。

六个月足够长，可以系统补齐短板；也足够短，不能无限发散。你必须把学习、实战、论文、项目、简历和 mock interview 放在同一条主线上推进。

本章重点：六个月总目标、按月路线、按周任务、每日节奏、项目里程碑、论文阅读、mock interview、验收标准和压缩版路线。

13.1 六个月冲刺的最终交付物

六个月结束时，你不应该只得到一堆笔记，而应该得到一组可以直接用于求职的交付物。

最终交付物包括：

1. 一份大模型算法岗简历。
2. 两到三个能深挖的项目。
3. 一个 2 分钟自我介绍。
4. 每个项目的 1 分钟、3 分钟、10 分钟讲稿。
5. 至少 10 篇论文的结构化阅读笔记。
6. 至少 5 套系统设计题模板。
7. 至少 100 道高频题的口头答案。
8. 至少 6 次完整 mock interview 记录。
9. 一个错题和薄弱点清单。
10. 一个英文面试基础模板库。

如果六个月后只有“我学了 Transformer、RLHF、RAG、vLLM”，但没有项目、讲稿、mock 记录和复盘，那么准备是不完整的。

13.2 六个月路线总览

推荐按六个阶段推进：

月份	主目标	关键产出
第 1 月	基础与数学/PyTorch 补齐	LLM 基础题、PyTorch 代码、mini 实验

月份	主目标	关键产出
第 2 月	Transformer 与 miniGPT	手写核心组件、训练 miniGPT、项目雏形
第 3 月	SFT/DPO/RAG/推理实战	2 个可讲项目、初版项目讲稿
第 4 月	Training/Deployment/Alignment 专项	专项面试题、系统排障清单
第 5 月	多模态/Agent/论文/系统设计	论文笔记、系统设计模板、开放题回答
第 6 月	简历、项目包装、mock interview	简历定稿、完整 mock、错题闭环

这条路线有一个核心原则：前三个月重在“做出来”，后三个月重在“讲清楚、答得住、能复盘”。

13.3 每周时间分配

如果你每周有 20 小时准备时间，建议这样分配：

模块	每周时间	目标
基础学习	5 小时	补概念和理论
代码/项目	6 小时	做出可展示证据
论文/研究	3 小时	训练判断力
面试题口头训练	3 小时	把知识转成表达
复盘和整理	2 小时	形成错题和讲稿
英文表达	1 小时	保持基本流畅

如果每周只有 10 小时，优先级是：项目 3 小时，基础 3 小时，面试口头训练 2 小时，复盘 1 小时，论文 1 小时。

如果每周有 40 小时，可以增加代码和 mock interview，但不要把时间全部用于看资料。输出训练必须占至少三分之一。

13.4 每日最小闭环

每天至少完成一个小闭环，而不是只“学习了一些”。

推荐每日结构：

1. 30 分钟复习一个概念。
2. 60 分钟写代码、看实验或改项目。
3. 20 分钟口头回答 2 道题。
4. 10 分钟记录今天的一个错点。

忙的时候也要保留最小训练：

今天至少口头回答 1 道题，并写下哪里卡住。

面试能力最怕中断。每天小剂量输出，比周末一次性刷 10 小时更稳定。

13.5 第 1 月：基础、数学和 PyTorch

第 1 月目标是把 LLM 基础地基打稳。

重点内容：

1. 概率、最大似然、交叉熵、KL、perplexity。
2. 梯度下降、反向传播、Adam、AdamW、learning rate、warmup。
3. PyTorch tensor、autograd、nn.Module、DataLoader、训练循环。
4. Tokenization、embedding、position encoding。
5. Transformer 总体结构。

第 1 周：概率和 loss。

产出：能用 3 分钟讲清 next-token prediction、cross entropy 和 perplexity。

第 2 周：优化和训练循环。

产出：手写一个简单 MLP 或 bigram language model 训练循环，能解释 `zero_grad()`、`loss.backward()`、`optimizer.step()`。

第 3 周：PyTorch 工程基础。

产出：写一个标准训练脚本，包含 train/eval、checkpoint、日志和简单配置。

第 4 周：Transformer 预热。

产出：能画出 decoder-only Transformer block 数据流，能解释 embedding、attention、MLP、residual、LayerNorm。

验收标准：

1. 能回答 30 道 ML/LLM 基础题。
2. 能手写 cross entropy 和一个训练循环。
3. 能解释 loss 不下降时的 5 个排查方向。
4. 能用自己的话解释大模型到底在学什么。

13.6 第 2 月：Transformer 与 miniGPT

第 2 月目标是从“知道 Transformer”升级为“能手写和训练一个最小 GPT”。

重点内容：

1. Self-attention。
2. Causal mask。
3. Multi-head attention。
4. LayerNorm / RMSNorm。
5. RoPE。
6. Decoder-only block。
7. Sampling、top-k、top-p。
8. miniGPT 训练和生成。

第 1 周：手写 attention。

产出：实现 scaled dot-product attention，能解释 Q/K/V shape 和 mask 为什么在 softmax 前加。

第 2 周：实现 multi-head attention 和 Transformer block。

产出：打印每层输入输出 shape，确认数据流。

第 3 周：训练 miniGPT。

产出：在小语料上训练一个 decoder-only 模型，记录 train loss、val loss 和生成样例。

第 4 周：采样和错误分析。

产出：实现 greedy、temperature、top-k、top-p，分析重复生成、胡言乱语和过拟合。

验收标准：

1. 能手写 causal attention。
2. 能解释 MHA、MQA、GQA 对 KV Cache 的影响。
3. 能讲清 GPT 和 BERT 训练目标区别。
4. miniGPT 项目可以讲 3 分钟。

13.7 第 3 月：SFT、DPO、RAG 和推理实战

第 3 月目标是做出 2 个能放进简历或作品集的项目。

推荐项目一：LoRA/SFT 中文助手。

关键任务：

1. 构造 instruction 数据。
2. 正确处理 chat template。
3. 实现 assistant-only loss mask。
4. 使用 LoRA 或 QLoRA 微调。
5. 比较 base vs SFT 输出。
6. 做人工评估和 bad case 分析。

推荐项目二：企业知识库 RAG。

关键任务：

1. 文档解析和 chunk。
2. Embedding 和向量索引。
3. Hybrid retrieval。
4. Rerank。
5. Citation。
6. 评估 recall、MRR、faithfulness、citation accuracy。

可选项目三：DPO 偏好优化。

关键任务：

1. 构造 chosen/rejected pairs。
2. 跑通 DPO 训练。
3. 分析长度偏差。
4. 比较 SFT 和 DPO 的 win rate、安全拒答和能力回归。

可选项目四：推理服务 demo。

关键任务：

1. 部署一个开源模型。
2. 支持 Chat API 和 SSE 流式输出。
3. 测 TTFT、TPOT、tokens/s、显存。
4. 对比量化或不同并发。

验收标准：

1. 至少完成 2 个项目的可运行版本。
2. 每个项目有 README、实验结果和 bad case。
3. 每个项目能讲 1 分钟和 3 分钟。
4. 能说明项目失败和下一步。

13.8 第 4 月：Training、Deployment、Alignment 专项突破

第 4 月目标是补齐大模型算法岗高频专项追问。

Training 专项：

1. 数据清洗、去重、mixture。
2. Pretraining、continued pretraining、SFT 区别。
3. AdamW、warmup、batch size、gradient clipping。
4. DP、TP、PP、ZeRO/FSDP。
5. Loss spike、NaN、checkpoint、resume。
6. 训练 FLOPs 和成本估算。

Deployment 专项:

1. Prefill / decode。
2. KV Cache 显存估算。
3. Continuous batching。
4. PagedAttention / vLLM。
5. 量化、speculative decoding。
6. TTFT、TPOT、P95/P99 排障。

Alignment 专项:

1. SFT、RLHF、DPO。
2. Reward hacking。
3. Over-refusal。
4. Jailbreak 和 prompt injection。
5. Safety eval 和 red teaming。
6. Model card、system card、发布门禁。

第 4 月每周安排:

1. 第 1 周: Training。
2. 第 2 周: Deployment。
3. 第 3 周: Alignment/Safety。
4. 第 4 周: 三类专项混合 mock。

验收标准:

1. 每个专项能回答 20 道题。
2. 每个专项有 1 页速记。
3. 能完成一次 Training 或 Deployment 深挖 mock。
4. 能回答一次 safety 开放题。

13.9 第 5 月: 多模态、Agent、论文和系统设计

第 5 月目标是从 “能做项目” 升级为 “能讨论方向和系统”。

多模态重点:

1. CLIP。
2. Vision encoder。
3. VLM 架构。
4. Diffusion。
5. 视频生成。
6. 语音模型。
7. 多模态评估和安全。

Agent 重点:

1. Tool schema。
2. Function calling。
3. Tool permission。
4. Agent memory。
5. Multi-step planning。
6. Prompt injection 防护。
7. Agent eval。

论文重点:

至少精读 6 篇:

1. Attention Is All You Need。
2. InstructGPT。

3. LoRA。
4. DPO。
5. FlashAttention。
6. CLIP。

有余力再读：Chinchilla、PagedAttention、LLaVA、Stable Diffusion、Constitutional AI、Mamba 或 Sora 相关技术报告。

系统设计重点：

1. ChatGPT 服务。
2. 推理平台。
3. RAG 系统。
4. Agent 平台。
5. 评估平台。
6. 多模态助手。

验收标准：

1. 能讲 3 篇论文的背景、方法、实验和边界。
2. 能设计 5 类系统主链路。
3. 能回答 “如何判断一篇论文贡献是否真实”。
4. 能完成一次系统设计 mock。

13.10 第 6 月：简历、项目包装和 Mock Interview

第 6 月目标是把所有能力转成面试表现。

第 1 周：简历定稿。

任务：

1. 确定目标岗位方向。
2. 精简技能栏。
3. 重写项目 bullet。
4. 每个指标补评估方式。
5. 删除不能追问的关键词。

第 2 周：项目讲稿定稿。

任务：

1. 每个项目准备 1 分钟、3 分钟、10 分钟版本。
2. 每个项目准备 20 个追问。
3. 每个项目准备失败案例和反思。
4. 每个项目准备英文简介。

第 3 周：全模块 mock。

任务：

1. 做 2 次完整 45 分钟 mock。
2. 做 1 次系统设计专项 mock。
3. 做 1 次英文 mock。
4. 复盘所有错题。

第 4 周：查漏补缺和投递。

任务：

1. 修正最后 10 个高频错题。
2. 重录自我介绍和项目讲述。
3. 整理反向提问。
4. 开始投递和约面。

验收标准：

- 1. 简历每个项目都能深挖。
- 2. 自我介绍稳定在 2 分钟内。
- 3. 至少完成 6 次 mock。
- 4. 每次 mock 有复盘记录。
- 5. 不确定问题能诚实表达边界。

13.11 三类候选人的不同路线

13.11.1 应届生或基础薄弱候选人

重点：基础、代码、项目可信度。

建议：

- 1. 前两个月不要跳过 miniGPT 和 PyTorch。
- 2. 项目少但要讲深。
- 3. 论文不求多，先讲透经典论文。
- 4. 简历不要装成熟工业经验。
- 5. 用复现和实验能力证明潜力。

13.11.2 有算法经验但非大模型背景

重点：迁移已有经验到 LLM。

建议：

- 1. 快速补 LLM 核心概念。
- 2. 用已有 ML/推荐/NLP/CV 项目连接数据、训练、评估能力。
- 3. 做一个 SFT/RAG/推理项目弥补大模型证据。
- 4. 面试中主动说明从旧领域迁移到大模型的路径。

13.11.3 有工程经验但算法不强

重点：系统、部署、RAG、评估和训练基础。

建议：

- 1. 不要只强调工程组件，要补模型机制。
- 2. 把服务稳定性、监控、成本、平台经验转成大模型系统价值。
- 3. 重点准备推理平台、RAG、评估平台和 Agent 系统设计。
- 4. 用代码项目证明能理解模型输入输出和训练目标。

13.12 每月验收表

月份	必须完成	如果没完成怎么办
第 1 月	基础题 30 道、训练循环、Transformer 总览	暂缓论文阅读，补基础
第 2 月	miniGPT、attention 代码、生成分析	暂缓 DPO，先补 Transformer
第 3 月	2 个项目可运行、初版讲稿	减少新知识，先完成项目闭环
第 4 月	Training/Deployment/Safety 专项题	每天口头练 5 道专项题
第 5 月	论文 6 篇、系统设计 5 题	减少项目扩展，补研究表达
第 6 月	简历、mock、错题闭环	暂缓投递，先做 2 次 mock

验收不通过时，不要继续往后堆内容。先修复最关键短板。

13.13 最容易失败的准备方式

六个月准备中最常见的失败方式：

1. 一直看资料，不做项目。
2. 一直做项目，不练表达。
3. 学很多新词，但不做错题复盘。
4. 简历写得很强，但项目经不起追问。
5. 只准备中文，不准备英文。
6. 只练技术题，不练 behavioral。
7. 只看公开 benchmark，不会设计评估。
8. 不敢 mock，等真实面试才发现表达问题。
9. 每次复盘都写“继续加强”，没有具体行动。
10. 同时追太多方向，没有主线。

避免这些问题的核心方法是：每两周做一次硬验收，看是否产生了可展示交付物。

13.14 一个月压缩路线

如果只有一个月，不要试图补完所有内容。

优先级：

1. 简历和项目讲稿。
2. 高频基础题。
3. Training、Deployment、Alignment 三个专项。
4. RAG 或推理平台系统设计。
5. Mock interview。

四周安排：

1. 第 1 周：简历、项目、基础题。
2. 第 2 周：Training/Deployment/Alignment。
3. 第 3 周：系统设计、论文讨论、behavioral。
4. 第 4 周：每天 mock 和复盘。

一个月路线的原则：少学新东西，多修表达和项目可信度。

13.15 三个月压缩路线

如果有三个月，可以按下面安排：

1. 第 1 月：基础、Transformer、miniGPT、SFT/RAG 项目。
2. 第 2 月：Training、Deployment、Alignment、多模态、论文。
3. 第 3 月：系统设计、简历、项目包装、mock interview。

三个月路线适合已有 ML 基础的人。如果基础薄弱，不建议跳过第 1 月的代码训练。

13.16 最后一周安排

面试前最后一周不要大规模学习新内容。

建议：

1. 每天录一遍自我介绍。
2. 每天讲一个项目。
3. 每天练 10 道高频题。
4. 隔天做一次 30 分钟 mock。
5. 每天复盘 3 个旧错题。
6. 睡前看一页速记，不刷新资料。

最后一天只做三件事：

1. 过简历和项目。
2. 过高频错题。
3. 休息。

不要在最后一天临时学一个新框架或新论文。那通常只会增加焦虑。

13.17 最终面试准备 Checklist

投递前检查：

1. 简历是否和目标岗位匹配。
2. 简历每个关键词是否能被追问。
3. 是否有 2-3 个项目能讲深。
4. 是否有一个项目能体现数据、模型、评估、系统和安全闭环。
5. 是否能回答基础题。
6. 是否能完成系统设计主链路。
7. 是否能讲至少 3 篇论文。
8. 是否完成至少 3 次完整 mock。
9. 是否准备 behavioral 故事库。
10. 是否准备英文自我介绍和项目介绍。

面试前 30 分钟检查：

1. 自我介绍。
2. 项目一句话定位。
3. 最熟项目的 3 个指标。
4. 最熟项目的 2 个失败案例。
5. 系统设计通用骨架。
6. 不确定问题回答模板。
7. 反向提问。

13.18 本书的使用方式

第十二册不是从头读一遍就结束，而应该反复使用。

建议使用顺序：

1. 先读第 1 章和第 2 章，明确岗位和简历主线。
2. 再读第 3 章，把项目讲稿写出来。
3. 用第 5-10 章补技术、训练、部署、对齐、多模态和系统设计。
4. 用第 11 章准备 behavioral 故事库。
5. 用第 12 章做 mock 和复盘。
6. 用本章安排每周任务和验收。

本书的目标不是让你“看完”，而是让你“能面试”。

13.19 最后一页速记

六个月冲刺记住五句话：

1. 前三个月做项目，后三个月练表达和面试。
2. 每两周必须产出一个可展示交付物。
3. 每个知识点都要转成口头回答。
4. 每个项目都要经得起数据、方法、实验、失败和系统追问。
5. 每次 mock 都要复盘，并把复盘变成下一轮行动。

最终目标不是“我学过很多”，而是面试官相信：你能进入一个高标准大模型团队，独立推进问题，可靠协作，诚实复盘，并在模型能力、系统约束和安全责任之间做出成熟判断。